Mechatronics Engineering Department

CoTune Lab

Active Suspension & Rotary Inverted Pendulum

Graduation Project Thesis

Submitted in fulfillment of the requirements for the
B.Sc. degree in Mechatronics Engineering

Team Members:

Name	ID
Adham Ehab Saleh	2100679
Abdelrahman Hany Abdelrahman	2100359
Ahmed Mohamed Ramadan	2100323
Ahmed Yasser Hosny	2101101
Khalid Amin Abdullah	2100808
Omar Mohamed Fathy	2100503

Supervisors:

Dr. Mohamed Omar

Academic Year: 2025–2026

1 Elsarayat St., Abbaseya, 11517 Cairo, Egypt
Fax: (+20 2) 26850617
www.eng.asu.edu.eg

DECLARATION

We hereby certify that this Project submitted as part of our partial fulfilment of BSc in Mechatronics Engineering is entirely our own work, that we have exercised reasonable care to ensure its originality, and does not to the best of our knowledge breach any copyrighted materials, and have not been taken from the work of others and to the extent that such work has been cited and acknowledged within the text of our work.

Signed by:

Adham Ehab Saleh	
Abdelrahman Hany Abdelrahman	
Ahmed Mohamed Ramadan	
Ahmed Yasser Hosny	
Khaled Amin	
Omar Mohamed Fathy	

Date: 20th of June 2026

ACKNOWLEDGMENT

It is with genuine appreciation that we acknowledge the invaluable contributions of those who made this graduation project possible. First and foremost, we would like to express our profound gratitude to our project supervisor, **Dr. Mohamed Omar**. His continuous support, visionary guidance, and deep expertise provided the foundation for our work. His patience and encouragement kept us motivated, and his constructive feedback was vital to the successful completion of this research.

Furthermore, we owe a special debt of gratitude to **Eng. Mohab Ahmed** and **Eng. Abdullah Awadullah**. We cannot thank them enough for their constant help, practical insights, and hands-on assistance during the implementation phases. Their dedication to helping us navigate complex technical hurdles was a critical factor in bringing this project to life.

ABSTRACT

This thesis focuses on the development of new experimental platforms intended to extend the capabilities of the CoTune Control Laboratory, with the aim of enhancing hands-on learning in control systems education through real-hardware experimentation. The project introduces two laboratory-scale experiments—an active suspension system and a rotary inverted pendulum—designed to complement theoretical coursework with practical implementation and experimental validation. Both experimental setups were developed from the ground up, starting with mechanical design and hardware integration, followed by system modeling and real-time experimental operation. The platforms are designed to support remote and flexible experimentation, allowing users to configure parameters and observe system behavior through live visual feedback, without requiring physical presence in the laboratory.

The active suspension experiment represents a quarter-car model and enables the study of vertical vehicle dynamics, disturbance effects, and suspension performance using a compact laboratory-scale setup. The rotary inverted pendulum experiment represents a classical unstable system and serves as a benchmark platform for studying nonlinear dynamics, instability, and real-time feedback control. Together, these experimental platforms expand the range of control problems addressed within the CoTune Laboratory and provide a foundation for future integration into remote-access educational environments, supporting a closer connection between control theory and practical implementation.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1 INTRODUCTION

1.1 Background & Motivation

Traditional control laboratories are a core component of engineering education, and most engineering students gain some level of hands-on experience during their studies. However, these laboratories are typically constrained by physical presence, fixed schedules, limited hardware availability, and restricted experiment time. As a result, students are often unable to freely test control theories, explore parameter variations, or repeat experiments outside scheduled lab sessions.

In addition, increasing student numbers and limited laboratory resources make it difficult to provide equal experimental access to all students. These constraints reduce the effectiveness of laboratory-based learning, particularly in control engineering, where repeated experimentation and real-time observation are essential for developing intuition and practical understanding.

The primary limitation addressed in this work is not the absence of control laboratories, but rather the lack of remote, flexible, and continuously accessible experimental platforms that allow students to interact with real hardware without being physically present in the laboratory.

1.2 CoTune Laboratory Concept

The CoTune Control Laboratory was developed to overcome the limitations of conventional control labs by enabling remote experimentation on real physical systems. CoTune allows students to access laboratory experiments through an online platform, configure system and controller parameters, and observe real-time system behavior without requiring physical presence in the lab.

Instead of replacing traditional laboratories, CoTune extends their functionality by decoupling experimentation from location and time constraints. This approach allows students to test control theories, analyze system responses, and validate design choices using real hardware, while maintaining the realism and non-ideal behavior of physical systems.

The CoTune Laboratory is designed as a unified framework that integrated multiple experimental platforms, real-time control execution, live data acquisition, and visual feedback, providing consistent user experience across different experiments.

1.3 Remote Experimentation and Live Interaction Framework

A core feature of the CoTune Laboratory is the ability to perform experiments remotely without requiring physical presence in the lab. This capability allows students to interact with real experimental setups through an online platform, overcoming common limitations related to lab availability, scheduling, and physical access.

Using the CoTune web interface, students can select an experiment, set the relevant parameters, and run the experiment directly on the physical hardware. The system executes the experiment in real time, while measured signals and system responses are streamed back to the user. In parallel, a live camera feed provides continuous visual feedback, allowing students to observe the physical motion of the system as the experiment is running.

This interaction model preserves the essential characteristics of hands-on experimentation, including real-time dynamics and non-ideal behavior, while enabling experiments to be performed from any location. Screenshots of the web interface and live camera feed are included to illustrate the structure of the platform and the user interaction flow.

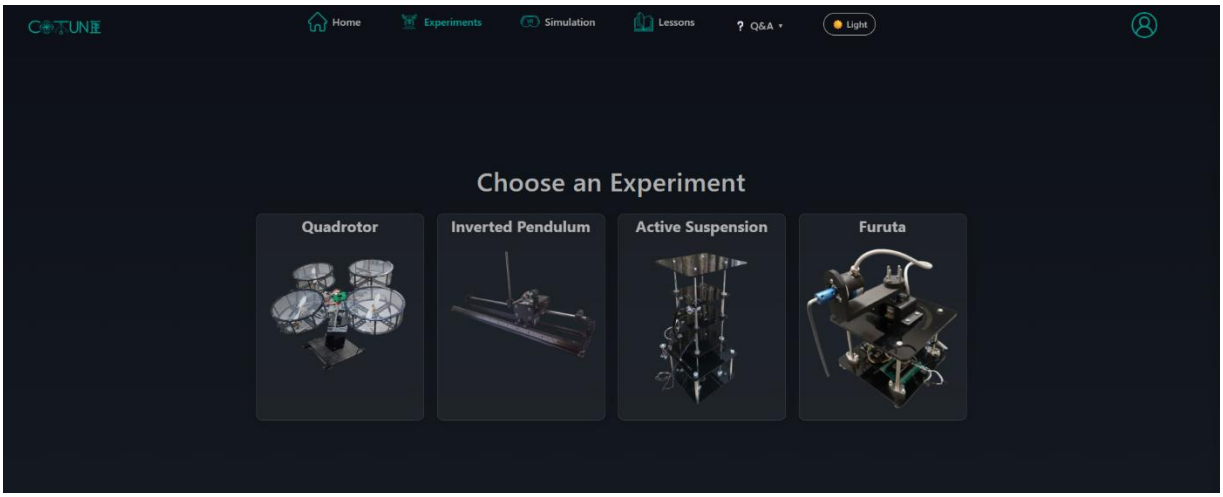

Figure 1-1: CoTune Web Interface

1.4 Existing Experimental Platforms

Prior to the development of the platforms described, the CoTune Laboratory already included experimental platforms designed to cover a range of control applications including:

- A quadrotor system.
- A linear inverted pendulum

Each of these experiments addresses different aspects of control systems, such as stability, multivariable control, motion control, and sensor-based feedback. Together, they form a solid foundation for practical control education within the CoTune framework. However, the existing platforms did not fully cover certain important control problems related to disturbance rejection and classical benchmark systems, motivating the addition of new experiments.

1.5 Motivation For the Newly Added Experiments

The selection of the active suspension and the rotary inverted pendulum was driven by the need to expand the range of control problems accessible through the CoTune Laboratory. The focus was not on adding more experiments for volume, but on introducing systems that offer clear educational value and complement the existing setups.

Relevance to vehicle dynamics and suitability for studying disturbance effects and performance trade-offs in controlled mechanical systems motivated the choice of the **active suspension system**. It allows students to directly observe how control actions influence system response under external excitations, making it well suited for both conceptual understanding and practical testing.

Considered a classical unstable system widely used in control education, the **rotary inverted pendulum** was selected for its ability to clearly demonstrate instability and real-time stabilization. Its dynamics offer an effective platform for testing control strategies under challenging conditions.

CHAPTER 2 LITERATURE REVIEW

2.1 Educational Control Laboratories

Control engineering education is fundamentally rooted in the integration of theoretical analysis with empirical validation. Traditionally, laboratory-based learning has relied on localized, physical testbeds that require student presence, rigid scheduling, and dedicated hardware maintenance. While these environments facilitate essential hands-on experience, they are often characterized by significant limitations, including restricted equipment availability, lack of flexibility for independent parameter exploration, and the inability to conduct experiments outside of formal laboratory hours. Recent trends in mechatronics education have sought to mitigate these constraints through the development of remote-access laboratory frameworks. These systems enable students to interact with real-world physical dynamics via web-based interfaces, bridging the gap between theoretical modeling and physical reality without the logistical overhead of traditional lab settings.

2.2 Active Suspension Systems

The active suspension system serves as a critical application of vertical vehicle dynamics, focusing on the trade-off between ride comfort and road holding.

- **Quarter-Car Modeling:** The classical quarter-car model, comprising two masses (sprung and unsprung), remains the industry standard for investigating suspension disturbances and active control strategies.
- **Prior Implementations:** Existing educational platforms, such as those popularized by Quanser, demonstrate high-fidelity responses but are often prohibitive in cost and lack the modularity required for distributed remote access.
- **Actuation Paradigms:** Current literature highlights various actuation methods, ranging from electromagnetic to pneumatic and hydraulic systems. Recent research into cable-driven and lead-screw mechanisms indicates high potential for laboratory-scale applications due to their ability to achieve precise linear motion with minimal backlash, which is essential for validating LQR (Linear Quadratic Regulator) and MPC (Model Predictive Control) algorithms.

2.3 Rotary Inverted Pendulum (Furuta Pendulum)

The rotary inverted pendulum is recognized as a quintessential benchmark in nonlinear control theory, primarily due to its inherent instability and under-actuated nature.

- **Nonlinear Dynamics:** The system's coupled dynamics, characterized by significant gravitational and inertial forces, provide an ideal platform for studying real-time stabilization and swing-up control strategies.
- **Control Strategies:** Literature frequently evaluates strategies such as PID (Proportional-Integral-Derivative) or LQR (Linear Quadratic Regulator) control for stabilization, and nonlinear energy-based swing-up control.
- **Educational Significance:** Unlike simpler linear systems, the rotary inverted pendulum challenges students to manage complex multivariable control loops, making it an indispensable tool for demonstrating the limits of linear control theory and the necessity of nonlinear compensation.

2.4 Synthesis and Project Motivation

Despite these advancements in remote laboratory technologies and benchmark control testbeds, a significant gap remains in the integration of highly modular, low-cost experimental hardware with a seamless remote-access software framework. Most existing systems prioritize either theoretical high-fidelity or remote accessibility, rarely combining both in a way that is easily scalable for large-enrollment mechatronics courses. This project addresses this gap by developing the active suspension and rotary inverted pendulum systems within the CoTune framework, ensuring that both high-performance physical interaction and flexible remote accessibility are provided as a unified educational resource.

CHAPTER 3 ACTIVE SUSPENSION

3.1 Introduction

3.1.1 Background

The automotive industry is currently undergoing a significant transformation driven by advances in control systems, electronics and embedded software. Modern vehicles are no longer purely mechanical systems; instead, they are increasingly becoming more complex cyber-physical systems. This evolution has led to higher expectations from passengers, who now demand not only reliable transportation, but also enhanced ride comfort, safety, and overall driving experience.

Among the vehicle subsystems, the suspension system plays a crucial role in isolating the vehicle body from road disturbance while ensuring stable vehicle behavior. Traditional passive suspension systems are limited by fixed mechanical properties that prevent adaptation to varying road conditions. These limitations have motivated the development of advanced suspension technologies that integrate control algorithms with mechanical design.

3.1.2 Suspension Fundamentals

The suspension system is a fundamental subsystem in any road vehicle, responsible for supporting the vehicle weight, isolating the vehicle body from road disturbances, and ensuring stable vehicle behavior during motion by maintaining the traction force between the tire and the road surface. A typical suspension system consists of mechanical elements such as springs, dampers, and linkages. The springs support the vehicle load and absorb energy from road irregularities, while the dampers dissipate this energy to limit excessive oscillations. Together, these components transfer forces between the vehicle body and the road surface while reducing unwanted vibrations. Conventional suspension systems require a compromise between ride comfort and road handling; improving one often leads to a reduction in the other, which limits overa ll system performance.

3.1.3 Passive Suspension Vs. Active Suspension

Passive suspension is the traditional configuration used in most conventional vehicles. The dynamic behavior of a passive system is entirely determined by the fixed mechanical properties of its springs and dampers, rendering it unable to adapt to changing road conditions or driving scenarios.

The limitations of passive systems have motivated the development of active suspension technology, which integrates sensors, actuators, and control algorithms into the mechanical design. An active actuator applies controlled forces between the vehicle body and the wheel assembly based on real-time sensor measurements. This allows the system to respond to road disturbances dynamically, rather than relying solely on passive energy dissipation.

3.1.4 Classification of Active Suspension

Active suspension technologies are generally categorized into two primary classes:

- **Semi-Active Suspension:** The active element is embedded within the damper to vary the damping coefficient—often by adjusting an orifice or applying a magnetic field to magnetorheological fluid. These systems are limited because they can only dissipate energy; they cannot generate an independent force to lift the vehicle body or actively push the wheel downward.

- **Full-Active Suspension:** The active element is mounted parallel to the damper to produce independent forces. Such elements typically consist of linear motors or hydraulic cylinders, offering superior dynamic performance at the cost of higher power consumption and system complexity.

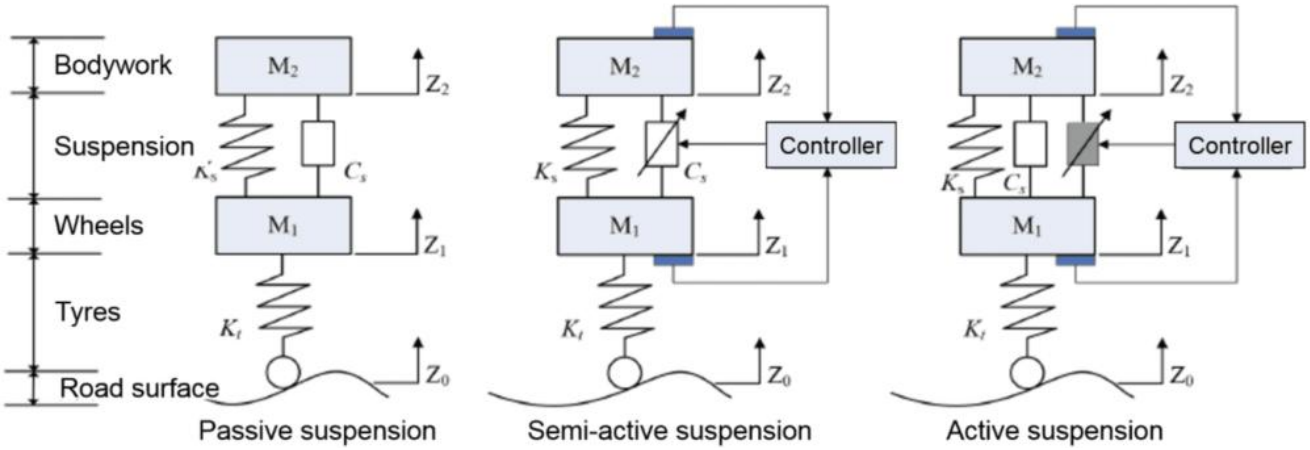

Figure 3-1: Suspension Types

3.1.5 Problem Statement

Although active suspension systems are essential in modern automotive engineering, their practical implementation is rarely emphasized in undergraduate education. Advanced control techniques, such as the Linear Quadratic Regulator (LQR), are frequently studied through simulations that assume ideal conditions, neglecting real-world effects such as friction, actuator saturation, and sensor noise. As a result, students often struggle to translate theoretical control designs into reliable behavior on physical hardware. This challenge is compounded by the fact that existing active suspension experimental platforms are often prohibitively expensive, restricting hands-on experience. This project addresses these limitations by developing an accessible, cost-effective, and remotely operable laboratory-scale active suspension experiment, enabling the practical validation of control strategies while allowing students to directly observe the impact of physical constraints and non-ideal system dynamics.

3.2 Mathematical model & System Identification

3.2.1 Dynamic Model

To design an effective active suspension control system, the physical dynamics of the vehicle must first be translated into a mathematical framework. This study utilizes a 2-Degree-of-Freedom (2-DOF) quarter-car model. This model strikes an optimal balance between mathematical simplicity and practical accuracy, capturing the essential vertical dynamics of both the vehicle body and the wheel assembly while isolating the active suspension control problem. Crucially, this theoretical model serves as the direct mathematical twin for the physical testbed developed in this research.

3.2.2 Model Parameters

The quarter-car model simplifies the vehicle into two lumped masses interacting through spring-damper combinations. Table 1 outlines the theoretical parameters and their direct physical counterparts on the experimental platform.

Parameter	Symbol	Description	Physical Counterpart on Testbed
Sprung Mass	Ms	The mass of the vehicle body.	Top Plate Assembly
Unsprung Mass	Mus	The mass of the wheel and axle.	Middle Plate Assembly
Suspension Stiffness	Ks	Spring rate of the primary suspension.	Coil springs between Top and Middle plates
Suspension Damping	Bs	Viscous friction in the suspension.	Natural friction of the top linear guides
Tire Stiffness	Kus	Spring rate of the pneumatic tire.	Coil springs between Middle and Bottom plates
Tire Damping	Bus	Viscous friction of the tire.	Natural friction of the bottom guides
Control Force	Fc	Active force applied between the masses.	Actuator (Cytron-driven Motor)
Road Disturbance	Zr	Vertical displacement of the road.	Bottom Plate (Sine wave excitation)

Table 3-1: Suspension System Parameters

Assumption:

All vertical displacements (Zs for the sprung mass, Zus for the unsprung mass) are defined with respect to the ground, with the positive direction pointing upwards.

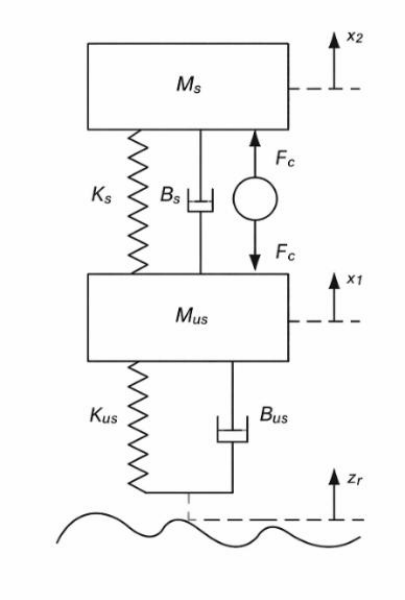

Figure 3-2: Double Mass Spring Damper to Model Active Suspension System

3.2.3 Equation of Motion

Applying Newton's Second Law to the free-body diagram of the two masses yields the fundamental differential equations governing the system.

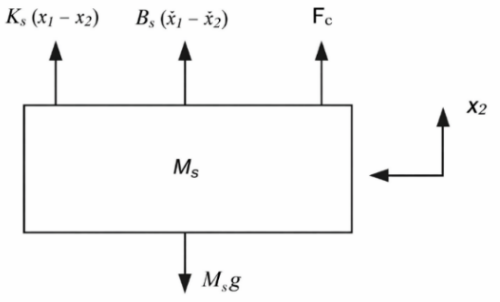

Figure 3-3: Free Body Diagram of M_s

The EOM for M_s will be as follows:

$$\ddot{x}_2 = -g + \frac{F_c}{M_s} + \frac{B_s\dot{x}_1}{M_s} - \frac{B_s\dot{x}_2}{M_s} + \frac{K_s x_1}{M_s} - \frac{K_s x_2}{M_s} \tag{3.1}$$

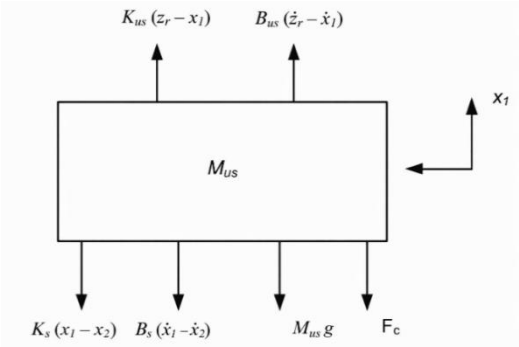

Figure 3-4: Free Body Diagram of Mus

The EOM for M_{us} will be as follows:

$$\ddot{x}_1 = -g - \frac{F_c}{M_{us}} - \frac{(B_s + B_{us})\dot{x}_1}{M_{us}} + \frac{B_s\dot{x}_2}{M_{us}} + \frac{B_{us}\dot{z}_r}{M_{us}} - \frac{(K_{us} + K_s)\,x_1}{M_{us}} + \frac{K_s x_2}{M_{us}} + \frac{K_{us}z_r}{M_{us}} \tag{3.2}$$

Eliminating Gravity force from EOM

The objective of this section is to prove mathematically that the gravity force only changes the equilibrium points in the Active Suspension EOM, and it does not affect the dynamics of the system.

i.e. $x_1 = x_{eq1},\ x_2 = x_{eq2}$

Substituting these changes in equations 2.1 and 2.2 will give the following results

$$M_s g + K_s(x_{eq2} - x_{eq1}) = 0 \tag{3.3}$$

$$M_{us} g + K_s(x_{eq1} - x_{eq2}) + K_{us}x_{eq1} = 0 \tag{3.4}$$

As a result, the equilibrium points due to gravity will be:

$$x_{eq1} = -\frac{g(M_s + M_{us})}{K_{us}} \tag{3.5}$$

$$x_{eq2} = -\frac{g(M_s K_{us} + K_s M_{us} + K_s M_s)}{K_{us} K_s} \tag{3.6}$$

To remove gravity forces from the equations of motion we apply the following change of variables to the equations of motion:

$$x_1 = z_{us} - \frac{g(M_s + M_{us})}{K_{us}} \tag{3.7}$$

$$x_2 = z_s - \frac{g(M_s K_{us} + K_s M_{us} + K_s M_s)}{K_{us} K_s} \tag{3.8}$$

$$\dot{x}_1 = \dot{z}_{us}\,, \dot{x}_2 = \dot{z}_s \tag{3.9}$$

$$\dot{\dot{x}}_1 = \dot{\dot{z}}_{us}, \dot{\dot{x}}_2 = \dot{\dot{z}}_s \tag{3.10}$$

i.e. z_{us} = 0, z_s = 0, will be the equilibrium point of the system.

From Newton's Second Law:

$$\Sigma F_z = m\ddot{z} \tag{3.11}$$

For Sprung Mass:

$$M_s \ddot{z}_s + B_s\,(\dot{z}_s - \dot{z}_{us}) + K_s(z_s - z_{us}) = F_c \tag{3.12}$$

For Unsprung Mass:

$$M_{us}\ddot{z}_{us} + B_s(\dot{z}_{us} - \dot{z}_s) + B_{us}(\dot{z}_{us} - \dot{z}_r) + K_s(z_{us} - z_s) + K_{us}(z_{us} - z_r) = -F_c \tag{3.13}$$

3.2.4 State Space Model

Definition

The state-space representation provides a convenient mathematical framework to model the quarter-car system with multiple inputs and outputs. Representing the system as a set of linear differential equations allows the physical dynamics to be translated into matrix operations, which is strictly required for the subsequent design of the Linear Quadratic Regulator (LQR) and the embedded Kalman Filter observer.

State Space Representation

The quarter-car model contains four energy storage elements (two masses storing kinetic energy, two springs storing potential energy), dictating a system with four states. The state variables are explicitly chosen to reflect the parameters measured and estimated by the experimental platform's sensor suite.

State Vector:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} z_s - z_{us} \\ \dot{z}_s \\ z_{us} - z_r \\ \dot{z}_{us} \end{bmatrix} \tag{3.14}$$

Where,

- x_1 : Suspension travel / Top-to-middle gap.
- x_2 : Vehicle body vertical absolute velocity.
- x_3 : Tire deflection / Middle-to-road gap.
- x_4 : Tire vertical absolute velocity.

Input Vector (U):

Instead of separating the control force and road disturbance, the input vector combines both the actuator command and the external road velocity excitation into a single vector to accommodate the Kalman filter structure.

$$U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} F_c \\ \dot{z}_r \end{bmatrix} \tag{3.15}$$

Where,

- u_1 : Road profile displacement
- u_2 : Road excitation velocity.

System Matrices

Using the Equations of Motion, the continuous dynamics matrices are calculated as:

$$A = \begin{bmatrix} 0 & 1 & 0 & -1 \\ -\frac{K_s}{M_s} & -\frac{B_s}{M_s} & 0 & \frac{B_s}{M_s} \\ 0 & 0 & 0 & 1 \\ \frac{K_s}{M_{us}} & \frac{B_s}{M_{us}} & -\frac{K_{us}}{M_{us}} & -\frac{(B_s + B_{us})}{M_{us}} \end{bmatrix} \tag{3.16}$$

$$B = \begin{bmatrix} 0 & 0 \\ \frac{1}{M_s} & 0 \\ 0 & -1 \\ -\frac{1}{M_{us}} & \frac{B_{us}}{M_{us}} \end{bmatrix} \tag{3.17}$$

Measurement Matrices

Because the system utilizes MPU6050 sensors, the measurement output (Y) includes the physical *accelerations* ($\ddot{z}_s$ and $\ddot{z}_{us}$). Since acceleration reacts instantaneously to the applied actuator force (Fc), the system exhibits direct feedthrough, resulting in a non-zero D matrix.

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{K_s}{M_s} & -\frac{B_s}{M_s} & 0 & \frac{B_s}{M_s} \\ 0 & 0 & 1 & 0 \\ \frac{K_s}{M_{us}} & \frac{B_s}{M_{us}} & -\frac{K_{us}}{M_{us}} & -\frac{(B_s + B_{us})}{M_{us}} \end{bmatrix} \quad D = \begin{bmatrix} 0 & 0 \\ \frac{1}{M_s} & 0 \\ 0 & 0 \\ -\frac{1}{M_{us}} & 0 \end{bmatrix} \tag{3.18}$$

3.3 Mechanical & Electrical Hardware Architecture

3.3.1 Overall Mechanical Architecture

The active suspension system is implemented as a bench-scale mechanical representation of a quarter-car model, designed to capture the essential vertical dynamics of a vehicle suspension while maintaining experimental flexibility. Our Mechanical Architecture is inspired by widely used educational active suspension platforms, particularly the **Quanser active suspension experiment**, which serves as a reference benchmark in control education.

The developed system adopts a vertically stacked, three-mass configuration, where each mass is constrained to move along a single vertical axis. This configuration allows the suspension dynamics to be studied in a simplified yet physically meaningful form, while remaining suitable fo r laboratory-scale implementation and rem ote experimentation.

Figure 3-6: CAD Assembly

Figure 3-5: Real Life Assembly

The mechanical structure consists of three rigid plates arranged vertically:

- Top acrylic plate (sprung mass): represents the vehicle body
- Middle acrylic plate (unsprung mass): represents the wheel-tire assembly
- Bottom plate (road input): represents the road surface excitation

Each plate is guided by parallel vertical stainless-steel shafts and supported by linear bearing blocks, ensuring that motion is strictly translated along the vertical axis with negligible lateral displacement or binding friction. This arrangement allows the physical system to directly emulate the classical quarter-car suspension model defined in 3.2, where the relative displacement between the top and middle plates corresponds to the suspension travel (d_1), and the relative displacement between the middle and bottom plates corresponds to the tire deflection (d_2).

3.3.2 Stroke Limitation and Mechanical Safety

Because the system is designed to be operated via cloud-based remote experimentation (detailed in Chapter 7), physical safety boundaries are critical. To protect the mechanical transmission and sensors from over-travel and collision damage, the overall architecture incorporates hard mechanical constraints:

- **End-of-Travel Detection:** Limit switches are strategically mounted at the absolute upper and lower boundaries of the permissible travel zones. If a plate triggers a limit switch, the control hardware immediately cuts PWM signals to the motor drivers.
- **Structural Clearance:** The vertical shafts and guide blocks are spaced to ensure that even at maximum displacement, there is sufficient physical clearance to prevent the primary suspension and tire springs from reaching full, damaging coil-bind (over-compression).

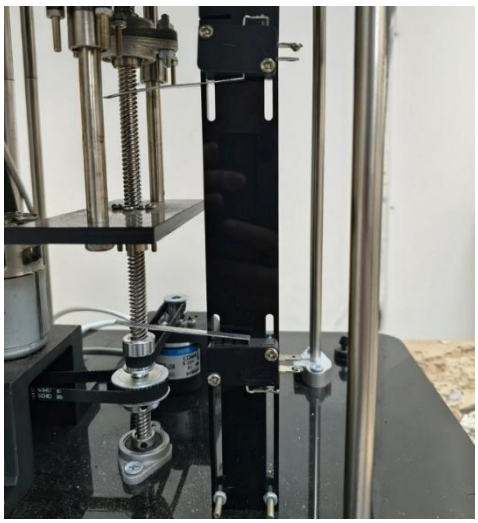

Figure 3-7: Stroke Limitation Safety

3.3.3 Actuation Mechanisms

The actuation system is responsible for actively injecting energy into the mechanical structure, enabling the regulation of vertical mass motion and the precise generation of road-induced disturbances. In contrast to passive suspension elements, the dual-actuator design provides independent, direct control over both the external excitation and the active suspension response.

Primary Actuator Selection (775 DC Motor)

Both the active suspension control and the road excitation mechanisms are driven by 775-series brushed DC motors. This specific motor profile was selected to balance high rotational speed, compact form factor, and sufficient torque for laboratory-scale dynamics. In an active suspension system, the actuator must react within milliseconds to counteract high-frequency road bumps. Operating at a nominal 12 VDC, the 775 motor delivers a no-load speed of 12,000 RPM and a stall torque of 79 N·cm (at 14.4 V), providing the necessary bandwidth and mechanical authority to manipulate the plates rapidly.

Figure 3-8: 775 DC Motor

Motor Type	Operating Voltage	Nominal Voltage	No-Load Speed	Rated Current	Stall Torque	Weight
775 DC Motor	6–20 VDC	12 VDC	12,000 RPM at 12 V	1.2 A at 12 V	79 N.cm at 14.4 V	350 g

Table 3-2: 775 DC Motor Specifications

Active Suspension Actuation (The Capstan Drive)

To exert active control forces between the vehicle body (sprung mass) and the wheel assembly (unsprung mass), the primary 775 motor is rigidly mounted to a bracket on the top plate.

The motor shaft is connected to a primary pulley, which transfers rotational motion through a timing belt transmission system with a 2:1 reduction ratio. This secondary pulley drives a custom capstan (cable-drum) mechanism. A tensioned cable is wrapped around a cylindrical drum with a radius of (r = 6 mm) and routed through guiding elements down to the moving middle plate.

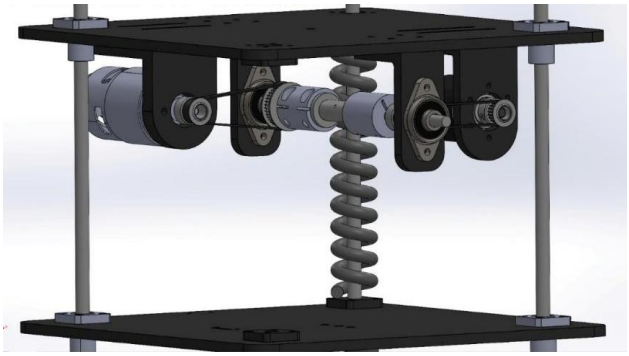

Figure 3-10: CAD Capstan Mechanism Assembly

Figure 3-9: Capstan Mechanism Assembly

The rotation of the drum results in a controlled change in cable length, converting the rotary motion of the DC motor into a bidirectional linear force applied directly along the suspension axis. This mechanism was selected for its ability to deliver smooth, continuous force transmission with near-zero backlash when properly tensioned. The inherently compliant nature of cable transmission also allows the actuator to interact naturally with the suspension's coil springs without introducing rigid binding.

Road Excitation Actuation (The Lead Screw Drive)

To generate deterministic and repeatable road disturbance profiles (such as sine waves, steps, or sweeping frequencies), a separate 775 motor is mounted on a rigid bracket fixed to the stationary base frame.

Similar to the suspension drive, the motor utilizes a belt transmission with a 2:1 reduction ratio. However, instead of a capstan, the driven pulley is mounted directly onto a vertical lead screw mechanism.

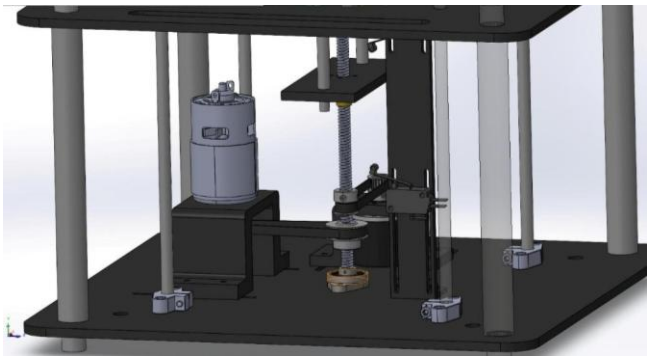

Figure 3-12: CAD Road Excitation Assembly

Figure 3-11: Lead Screw Assembly

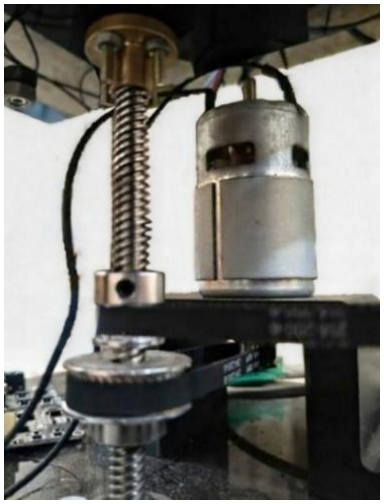

Figure 3-13: Road Motor Fixation

The lead screw and nut assembly feature a pitch of 8 mm per revolution, converting the motor's rotary motion into a highly precise linear vertical displacement of the bottom plate. The lead screw architecture was prioritized for the road input due to its high positional accuracy, mechanical stiffness, and inability to be easily back-driven. This ensures that the imposed road disturbance is strictly a function of the motor command and remains completely independent of the dynamic forces reacting above it in the suspension stages.

3.3.4 Sensor Suite Selection

The experimental platforms rely on a multi-modal sensor suite to provide real-time state feedback, enabling closed-loop control and system monitoring. The selection of sensors was driven by the requirements for high sampling rates, non-contact measurement, and integration with the ESP32-based control architecture.

Time of Flight (ToF) Displacement Sensors

To measure the vertical displacements (d_1 and d_2) required for the suspension model, two **VL53L0X** Time-of-Flight (ToF) sensors are utilized. These sensors offer millimeter-level precision and high sampling rates suitable for capturing rapid suspension oscillations.

- **Placement:** The sensors are mounted on fixed brackets aligned vertically with the moving plates.
- **Addressing Strategy:** Since these sensors share the same I^2C address, a shutdown-based (XSHUT) startup sequence is employed during system initialization. This ensures each device is enabled sequentially and assigned a unique address, allowing for reliable multi-sensor data acquisition.

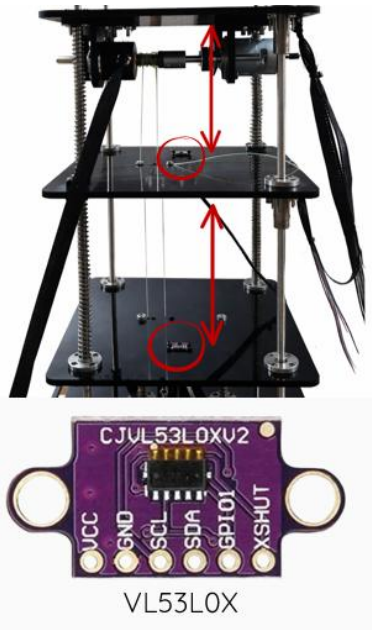

Figure 3-14: Integration of VL53L0X ToF Sensors Indicating the measured relative distances corresponding to suspension travel and tire deflection.

Inertial Measurement Unit (IMU)

To capture the vertical acceleration of the sprung mass, an MPU6050 six-axis IMU is integrated into the top plate. This sensor provides acceleration data that is essential for validating the dynamic response of the system under external road disturbances. By fusing ToF displacement data with IMU acceleration measurements, the system achieves a robust observation of the suspension's state, facilitating the implementation of more advanced control algorithms such as State-Feedback and Kalman filtering.

Figure 3-15: Mounting of the MPU6050 IMU on the sprung mass for vertical acceleration monitoring and state-feedback control.

Incremental Encoders

Two YT06-OP incremental encoders with 600 PPR are utilized to provide precise rotational velocity and position feedback from the 775 DC motors. These sensors allow for the calculation of the motor shaft position, which is essential for determining the linear translation of the capstan and lead-screw stages through the 2:1 reduction ratio.

- **Feedback Mechanism:** The encoders provide quadrature output signals (Phase A and Phase B), which allow for the detection of both the magnitude and direction of rotation.
- **Signal Conditioning:** To ensure compatibility with the ESP32's 3.3 V logic levels and to guarantee signal integrity, the quadrature outputs are interfaced using external 1.2 kΩ pull-up resistors.
- **Integration:** The quadrature signals are decoded directly within the Simulink environment using a specialized decoder block, which translates the raw pulses into accurate angular position data. This data is then processed to close the feedback loops

for the suspension and road-input stages, providing the necessary resolution for high-performance motion control.

Figure 3-16: YT06-OP Incremental Encoder

3.3.5 Electrical System Overview

The electrical system provides the interface between the mechanical plant and the control environment, emphasizing robustness, real-time compatibility, and seamless integration with the Simulink framework. All sensing, actuation, and signal processing tasks are implemented exclusively within the Simulink environment, ensuring consistency between simulation and experimental implementation.

Power Supply and Distribution

The system is powered by a controllable DC power supply (12–24 V), which serves as the main energy source for the experiment. Power distribution is separated into high-power and low-power domains:

- **High-Power Path:** Supplies the Cytron motor drivers, which are optimized for high-current demands during dynamic motor operation.

Figure 3-17:12-24 V Power Supply

Figure 3-18: DC-DC Step Down Converter

- **Low-Power Path:** A dedicated DC–DC converter steps down the main supply voltage to a regulated 3.3 V rail for the ESP32 and logic-level devices. A common ground reference is maintained across all components to ensure signal integrity and reduce electrical noise coupling.

Actuator Drive Electronics

Actuation is achieved using Cytron motor drivers. These act as power amplification stages, accepting low-power PWM and direction signals from the ESP32 and converting them into high-current drive signals. This configuration isolates the control hardware from the electrical stresses associated with motor operation and ensures safe current handling under varying mechanical loads.

Figure 3-19: Cytron MDD10A Motor Driver

3.3.6 Wiring and Interconnection Architecture

This section outlines the distribution of power and signal lines across the platform, maintaining a unified control structure centered around the ESP32.

Motors and Encoders Connections

- **Motor Drivers:** Two DC motors are connected to dedicated Cytron driver channels. The high-current motor connections remain electrically isolated from the low-power control circuitry.
- **Incremental Encoders:** Quadrature output signals (Phase A and B) are interfaced with the ESP32 using external 1.2 kΩ pull-up resistors to ensure compatibility with 3.3 V logic levels while maintaining signal reliability.

Sensor Interconnection via ESP32 Shield

Inertial and ToF sensors are interfaced through a custom ESP32 PCB shield, which provides organized routing for power and communication lines. Communication is achieved via the I²C bus, with the SDA and SCL lines connected directly to the ESP32. This shared-bus architecture reduces wiring complexity and supports synchronized data acquisition for all sensors.

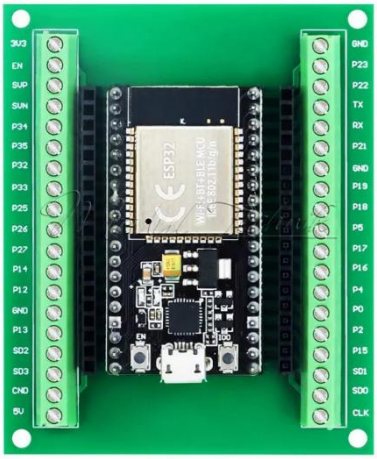

Figure 3-20: ESP32 PCB Shield

Wiring Overview

Figure 22 illustrates the overall wiring architecture. The diagram emphasizes the clear separation between high-current actuation circuits and low-power sensing logic, confirming the stability and safety of the integrated platform.

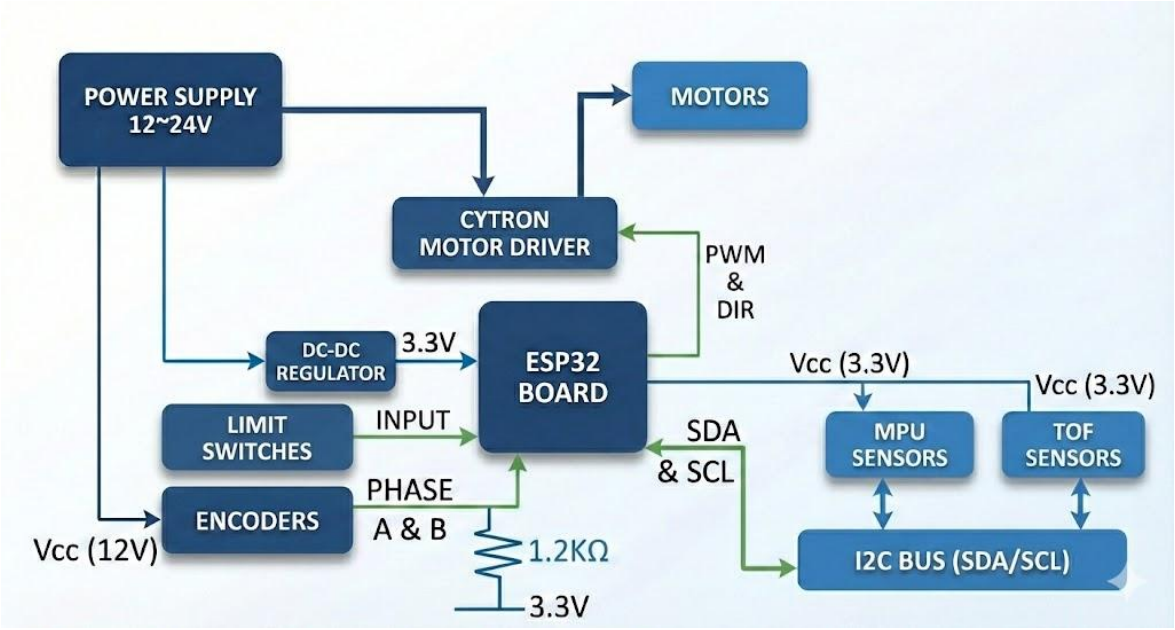

Figure 3-21: Wiring Architecture

3.4 System Identification

3.4.1 Introduction

To design an effective model-based controller—such as a Linear Quadratic Regulator (LQR) for the active suspension platform, an accurate mathematical representation of the physical system is required. While nominal parameter values can provide a baseline, real-world physical systems exhibit non-ideal characteristics such as friction, unmodeled damping, and manufacturing tolerances. This chapter details the system identification process utilized to accurately estimate the electrical and mechanical parameters of the CoTune platform using empirical data acquisition and MATLAB's Parameter Estimator.

3.4.2 Physical Parameter Measurement

Before performing dynamic estimation, the static and geometric parameters of the system were directly measured. The mass of the sprung and unsprung plates, as well as the mechanical transmission constants, were physically quantified to reduce the number of unknown variables in the estimation algorithm. The directly measured parameters are:

- **Sprung Mass (M_s):** 2.45 kg
- **Unsprung Mass (M_{us}):** 1.00 kg
- **Capstan Pulley Radius (r):** 6 mm
- **Transmission Gear Ratio (K_g):** 2 (representing a 2:1 reduction)

3.4.3 DC Motor Parameter Estimation

The actuation authority of the active suspension relies entirely on the 775 DC motor. To model its dynamics, a grey-box estimation approach was employed using isolated empirical data.

Experimental Procedure

Data acquisition was performed on the DC motor in isolation. The motor was subjected to varying voltage step inputs, and the resulting rotational velocity was recorded using the incremental rotary encoder.

Estimation Model and Results

The acquired dataset was fed into MATLAB's Parameter Estimator alongside a standard DC motor Simulink block diagram.

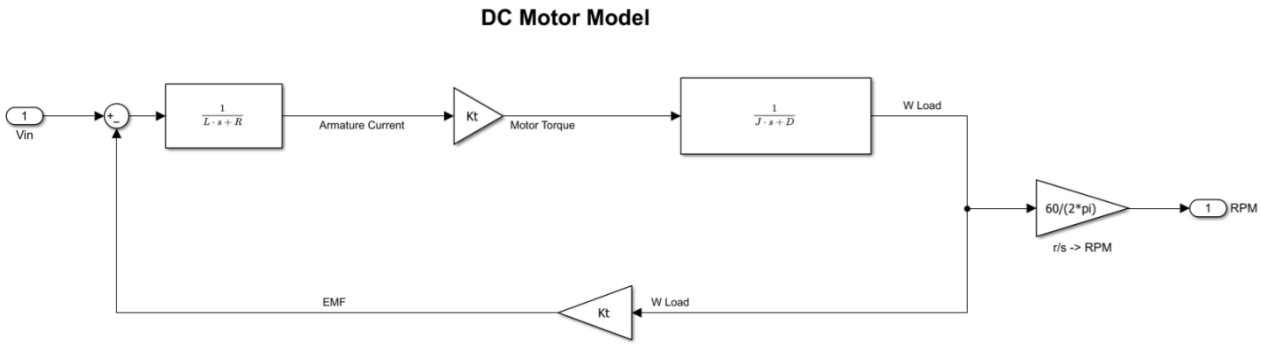

Figure 3-22: Simulink block diagram of the DC motor utilized for grey-box parameter estimation

The model defines the relationship between the electrical input voltage (V_{in}) and the mechanical output speed in RPM. The estimation algorithm iteratively minimized the error between the simulated model output and the empirical encoder data to identify the core motor constants. The estimated electrical and electromechanical parameters are:

- **Torque Constant (K_t):** 0.05667 N · m/A
- **Armature Resistance (R_m):** 3 Ω

3.4.4 Active Suspension System Parameter Estimation

Following the isolation of the actuator dynamics, the complete active suspension assembly was evaluated. The objective of this phase was to identify the dynamic stiffness and damping coefficients of the mechanical structure.

Experimental Procedure and Data Acquisition

The full system was fully assembled and excited using the road-input lead-screw mechanism. To capture the full dynamic envelope, the four primary state sensors were utilized simultaneously:

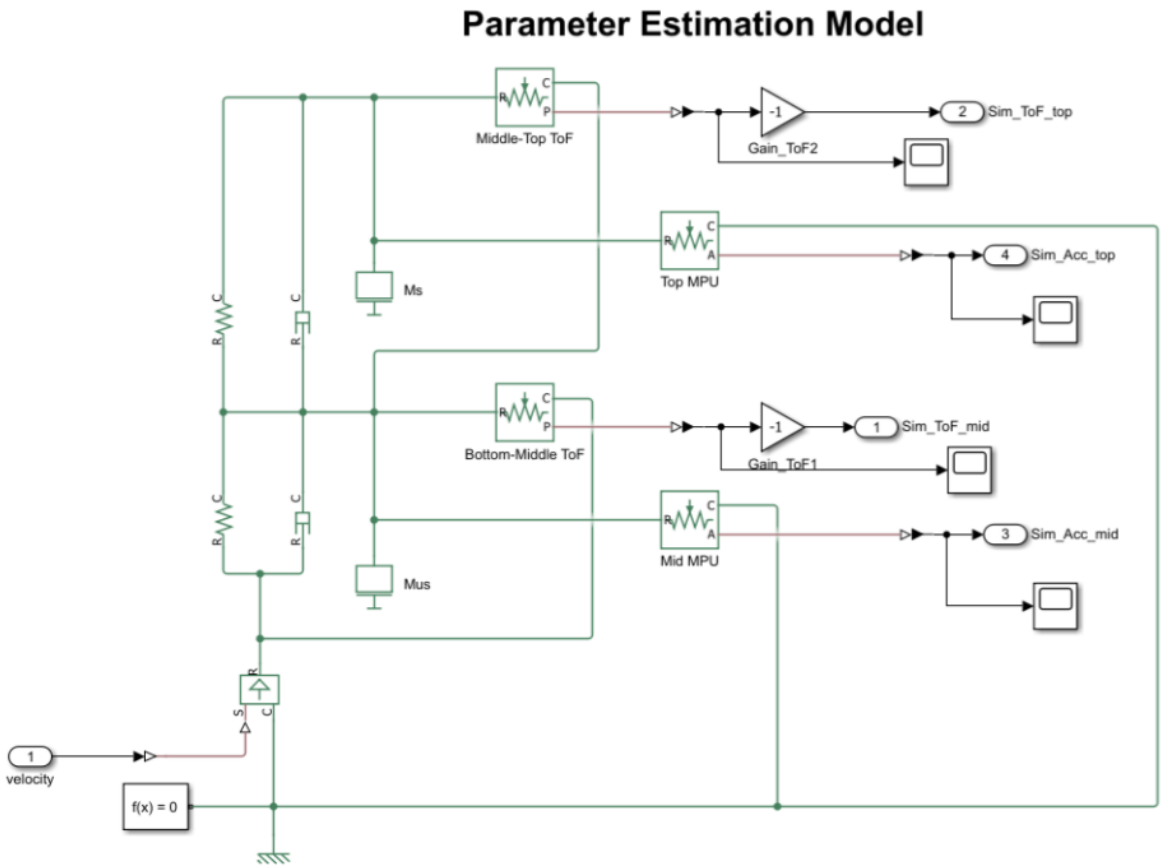

Figure 3-23: Simscape multi-body model representing the active suspension architecture for stiffness and damping estimation

1. Top Time-of-Flight (ToF) displacement sensor.
2. Middle Time-of-Flight (ToF) displacement sensor.
3. Top MPU6050 Accelerometer (M_s acceleration).
4. Middle MPU6050 Accelerometer (M_{us} acceleration).

The system excitation input—the velocity of the bottom road plate—was not directly measured by a dedicated velocity sensor. Instead, it was derived by differentiating the displacement readings from the road-motor encoder. To ensure high signal fidelity and prevent high-frequency noise amplification during differentiation, an offline low-pass filter was applied to the velocity data before utilizing it in the estimation model.

Estimation Model and Results

The filtered velocity input and the four sensor output arrays were loaded into the Parameter Estimator. The reference architecture was a comprehensive Simscape multi-body mathematical model [Insert Figure: Parameter Estimation Model], which simulates the physical interactions between the masses, ideal translational springs, and dampers. By constraining the known masses (M_s and M_{us}), the algorithm accurately converged on the structural constants of the quarter-car model:

- **Suspension Stiffness (K_s):** 900 N/m
- **Tire Stiffness (K_{us}):** 1250 N/m
- **Suspension Damping (B_s):** 58.08 N · s/m
- **Tire Damping (B_{us}):** 14.726 N · s/m

These experimentally validated parameters form the foundational A and B state-space matrices required for the subsequent control design phase.

3.4.5 Parameter Validation and Conclusion

Standard parameter estimation workflows typically involve a direct graphical comparison between empirical datasets and simulated model responses. However, for this active suspension architecture, the validation of the identified parameters (K_s, K_{us}, B_s, B_{us}) was conducted through **indirect dynamic validation**.

Rather than relying solely on open-loop curve fitting, the accuracy of the estimated parameters was validated in closed-loop operation. The estimated values were utilized to calculate the State-Space A and B matrices, which subsequently drove the design of the Linear Quadratic Regulator (LQR). The successful stabilization of the physical hardware and the rejection of road disturbances—detailed in the subsequent Control Design and Results chapters—serves as the definitive empirical proof that the identified parameters accurately represent the physical plant's dynamics within the required operational envelope.

3.5 Control Strategy and Offline Simulation

Before deploying control algorithms to the physical ESP32 microcontroller, the mathematically identified system parameters from Chapter 4 (K_s, K_{us}, B_s, B_{us}) must be validated in an ideal, noise-free environment. This chapter presents the offline simulation framework and the formulation of the Linear Quadratic Regulator (LQR) control strategy.

The simulation environment was developed using a combined Simscape and Simulink architecture. Simscape was employed to model the physical multi-body dynamics of the mechanical suspension, while Simulink was utilized for controller implementation and signal routing. This offline model serves a dual purpose: it proves that the identified system parameters accurately represent the physical quarter-car dynamics, and it provides the computational engine required to solve the Algebraic Riccati Equation for the optimal LQR gain matrix (K).

3.5.1 Simulation Framework Overview

The simulation framework was developed using an integrated SolidWorks-Simulink-Simscape workflow to ensure strict geometric and dynamic consistency between the mechanical CAD design and the mathematical model.

First, the complete mechanical assembly was designed in SolidWorks. The mass, inertia, and dimensional properties of the sprung and unsprung masses were exported and imported into the MATLAB/Simulink environment. Simscape Multibody was then employed to translate these physical bodies into dynamic blocks, linking them with idealized 1D translational spring and damper elements representing the physical coils and friction identified during system identification.

This modular, physics-based approach allows the mechanical structure, the external road excitation, and the LQR control loop to be evaluated interactively. By utilizing an offline simulation first, the control logic could be rigorously tested and tuned for aggressive disturbance rejection without risking mechanical damage to the physical laboratory hardware.

Simscape-Based Plant Modeling

The Simscape-Simulink model represents the physical realization of the active suspension system and forms the core of the simulation framework. Rather than relying strictly on abstract transfer functions, the mechanical plant is implemented using Simscape multibody components derived directly from the imported CAD model. This ensures that the simulated dynamics are rigorously consistent with the physical geometry, kinematic constraints, and mass distributions of the laboratory system.

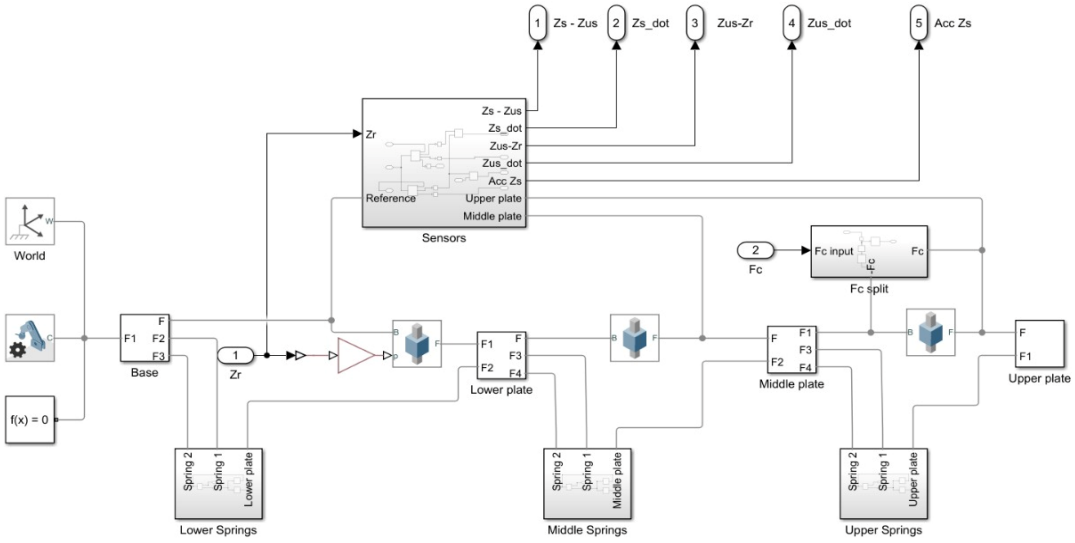

Figure 3-24: Overall Simscape representation of the active suspension system, illustrating the interconnected mass and spring elements.

As shown in Figure 3-24, the model consists of the base, lower plate, middle plate, and upper plate. These rigid bodies are interconnected through 1D translational spring and damper elements. To validate the system identification performed in Chapter 4, the stiffness and damping coefficients of these Simscape blocks were explicitly set to the experimentally derived values ($K_s = 900$ N/m, $K_{us} = 1250$ N/m, $B_s = 58.08$ N · s/m, and $B_{us} = 14.72$ N · s/m).

Road excitation is applied at the base through the displacement input Z_r, while the active control force F_c is injected into the system and distributed via a force-splitting block. Sensor outputs such as suspension deflection, relative velocities, and sprung-mass acceleration are extracted from the plant using idealized Simscape sensors and routed to output ports for feedback evaluation.

Plant Subsystem Architecture

To maintain modularity and debugging clarity, the plant subsystem is structured to perfectly mirror the physical assembly, ensuring a clear 1-to-1 correspondence between simulated blocks and real-world hardware components.

Sprung and Unsprung Mass Subsystems

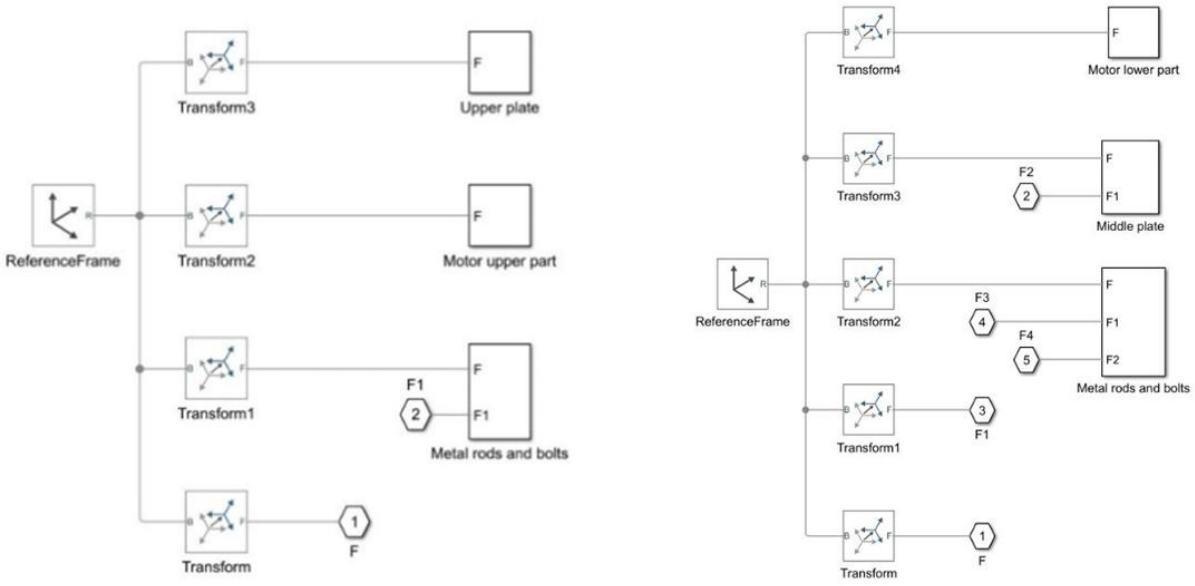

Figure 3-25: Simscape internal architectures for the Upper Plate (Sprung Mass) and Middle Plate (Unsprung Mass) subsystems.

The upper plate (representing the vehicle body) and the middle plate (representing the wheel-tire assembly) are encapsulated in dedicated subsystems. Within these blocks, the solid properties are defined using the masses established in Chapter 4 ($M_s = 2.45$ kg and $M_{us} = 1.00$ kg). Rigid transform blocks are utilized to map the physical connection points of the metal guide rods, bolts, and motor mounts to the central center-of-mass reference frame, ensuring that the simulated inertia perfectly matches the physical rig.

Base and Excitation Subsystem

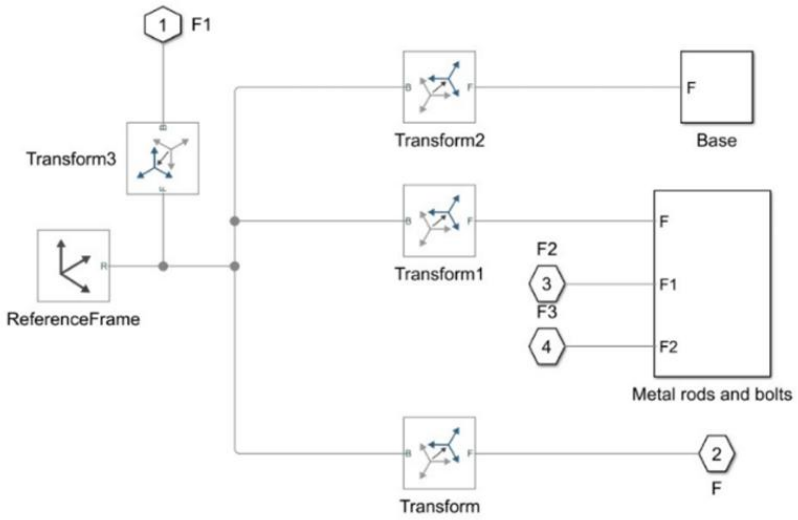

Figure 3-26: Simscape architecture of the base subsystem and road-input reference frame.

The base subsystem anchors the entire simulation to the global reference frame. It includes the structural elements of the bottom plate and the lead-screw actuation mechanism. In the simulation, this base is not entirely fixed; it is subjected to the Z_r reference signal, allowing the entire structure to experience vertical road profiles (such as square waves or swept sine waves) that propagate upward through the simulated tire springs.

Sensor and Feedback Subsystem

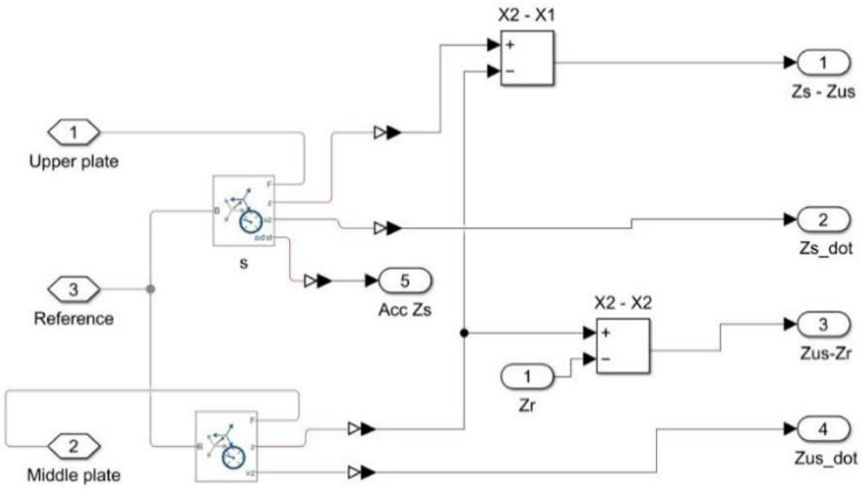

Figure 3-27: Sensor and Feedback Subsystem

To close the control loop, an ideal sensing architecture was embedded within the plant. Unlike the physical system (which requires complex Kalman filtering to derive velocity from noisy distance sensors), the Simscape environment allows for the direct measurement of perfect state derivatives. The sensor subsystem outputs the exact relative displacements ($Z_s - Z_{us}$, $Z_{us} - Z_r$) and their corresponding velocities ($\dot{Z}_s$, $\dot{Z}_{us}$), providing the LQR controller with an uncorrupted state vector to calculate the optimal active control force.

Simscape Full Model Graphical Representation

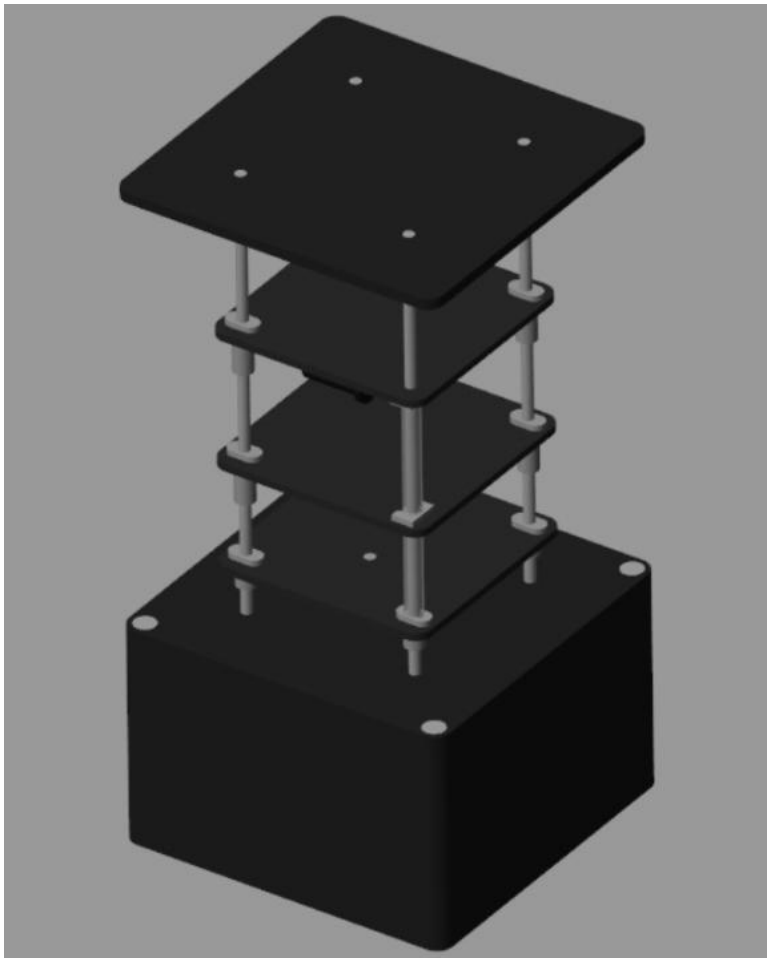

Figure 3-28: Simscape Graphical Representation

3.5.2 Linear Quadratic Regulator (LQR) Theory

Introduction to Pole Placement and Limitations

Full State Controllability implies that the closed-loop poles of a system can be arbitrarily placed using a suitable feedback gain matrix (K). Intuition may suggest placing these poles far into the left-half of the complex plane to achieve an exceptionally fast system response. However, driving the poles excessively far into the left-half plane results in a significantly larger gain matrix. This, in turn, demands unreasonably high control signals that will quickly saturate the physical actuators. Therefore, standard pole placement requires a trial-and-error approach to balance the system's fast response against the physical limits of the actuation hardware.

LQR Problem Formulation

To provide a more systematic approach, the Linear Quadratic Regulator (LQR) defines a cost function (J) that penalizes both the state deviation (how far the car is bouncing) and the control signal (how much power the motor is using). By defining Q and R as diagonal weighting matrices, the optimization problem is formulated to find the feedback gains that result in the minimum overall cost. The true power of LQR lies in its tuning simplicity; the Q and R matrices have meaningful physical interpretations, making the tuning process highly intuitive.

Derivation of the Optimal Gain

The solution to this optimization problem yields the linear control law:

$$u = -Kx \tag{3.20}$$

Where the optimal gain matrix K is defined as:

$$K = R^{-1}B^{T}S \tag{3.21}$$

Here, S is the solution to the continuous-time Algebraic Riccati Equation (ARE):

$$A^{T}S + SA - SBR^{-1}B^{T}S + Q = 0 \tag{3.22}$$

LQR Design Process

In the original pole placement formulation, the design process starts by guessing a closed-loop pole location and calculating the feedback matrix using Ackermann's formula. Using the LQR formulation, the design process becomes highly systematic:

1. Select the relative importance of each state variable versus the actuator effort by tuning the Q and R matrices.

2. Solve the Algebraic Riccati Equation to automatically generate the optimal feedback matrix K.

LQR and State Estimation (The Separation Principle)

Both traditional pole placement and LQR require the full state vector x to be measured and fed back. This condition is rarely realized in actual mechanical applications, either because it is uneconomical to use a dedicated sensor for every state, or because certain states (like pure velocity) cannot be measured directly without noise.

In practical applications, the system only provides partial state measurements. If the system is Observable, a State Estimator (Observer) can be built to take in these incomplete, noisy measurements and reconstruct the full state vector. One highly effective state observer for noisy digital systems is the Kalman Filter, which is detailed in Section 3.6.

The Separation Principle of control theory guarantees that an optimal state observer (which minimizes estimation error) and an optimal controller (which minimizes the cost function) can be designed independently. When combined, the overall system remains optimal. The powerful combination of a Kalman filter and an LQR controller is formally known as Linear Quadratic Gaussian (LQG) control.

3.5.3 Controller Subsystem and LQR Formulation

The controller subsystem is responsible for computing the active control force applied to the plant based on the full-state feedback provided by the sensor subsystem. It operates natively

within the Simulink environment, allowing the control law to be tuned and evaluated without altering the underlying physical Simscape model.

Reference Input and Excitation Profile

To rigorously evaluate the transient and steady-state response of the system, a deterministic road disturbance is applied at the base of the model (Z_r). For this simulation, a harsh excitation profile was selected to represent severe road anomalies:

- **Signal Type:** Square wave displacement
- **Amplitude:** 2 cm (0.02 m)
- **Frequency:** 1 Hz

This specific profile was chosen because square waves contain virtually infinite high-frequency harmonics, making them an excellent benchmark for evaluating suspension travel limits, road handling recovery, and controller bandwidth.

Linear Quadratic Regulator (LQR) Architecture

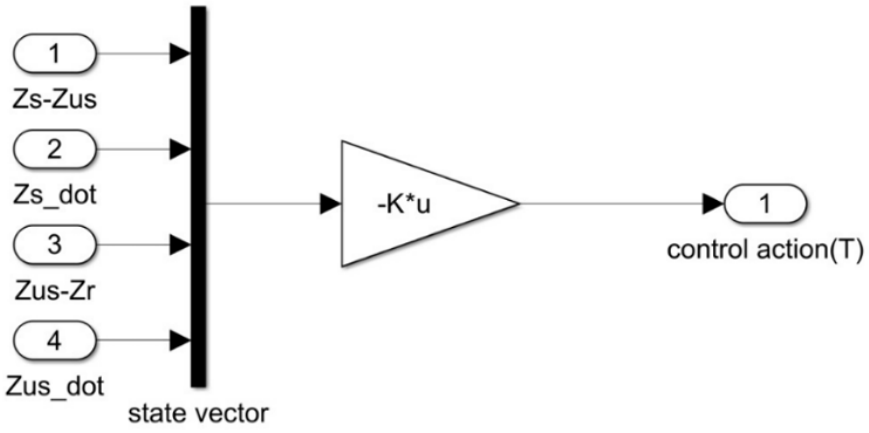

Figure 3-29: The Simulink controller subsystem routing the state vector through the LQR gain matrix.

The control strategy implemented is a Linear Quadratic Regulator (LQR), which provides an optimal control sequence by minimizing a cost function that mathematically balances state deviations (ride comfort and road handling) against the required control effort (actuator saturation limits).

Before calculating the optimal gain matrix (K), the continuous-time state-space system (A, B) derived from the parameters in Chapter 4 was evaluated for controllability. The controllability

matrix was confirmed to have a full rank of $4/4$, proving that the actuator has the theoretical authority to drive all states to the origin. The Algebraic Riccati Equation (ARE) was then solved using appropriately tuned state and input weighting matrices (Q and R), yielding the following robust control parameters:

LQR Design Parameter	Computed Value
System Controllability Rank	$4/4$ (Fully Controllable)
LQR Gain Matrix ($\boldsymbol{K}$)	$[11.3848, \quad 61.4922, \quad -206.1923, \quad -4.4806]$
Closed-Loop Poles	$-14.3835 \pm 11.8910i, \quad -20.3057 \pm 49.0206i$

Table 3-3: LQR formulation summary, indicating a stable closed-loop pole placement.

The controller block (Figure 3-28) continuously processes the measured state vector x and applies the linear control law $F_c = -Kx$ to generate the dynamic actuation force.

3.5.4 Simulation Results and Performance Evaluation

To directly quantify the performance improvements provided by the LQR controller, a comparative simulation was executed. During the initial phase ($t = 0$ to $t = 5$ seconds), the active control effort was intentionally disabled ($F_c = 0$), forcing the system to behave as a purely passive suspension. At exactly $t = 5$ seconds, the LQR controller was engaged. This

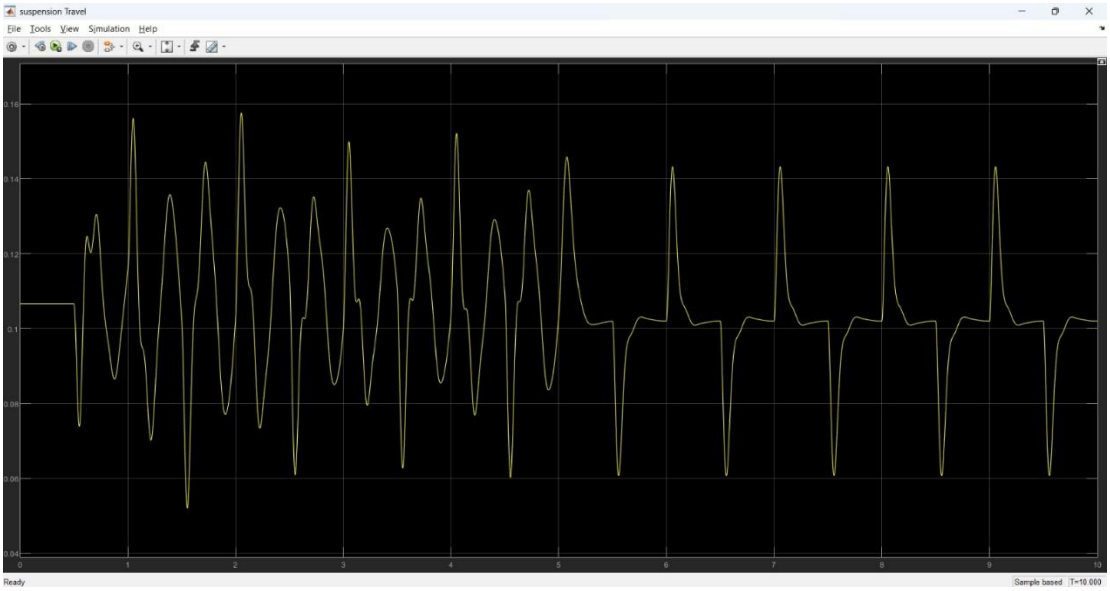

Figure 3-30: Relative suspension travel (Zs - Zus). Controller activation substantially reduces the deflection magnitude, minimizing the risk of mechanical bottoming-out.

mid-simulation transition provides a direct, localized comparison of the passive vs. active dynamic responses under identical road excitation conditions.

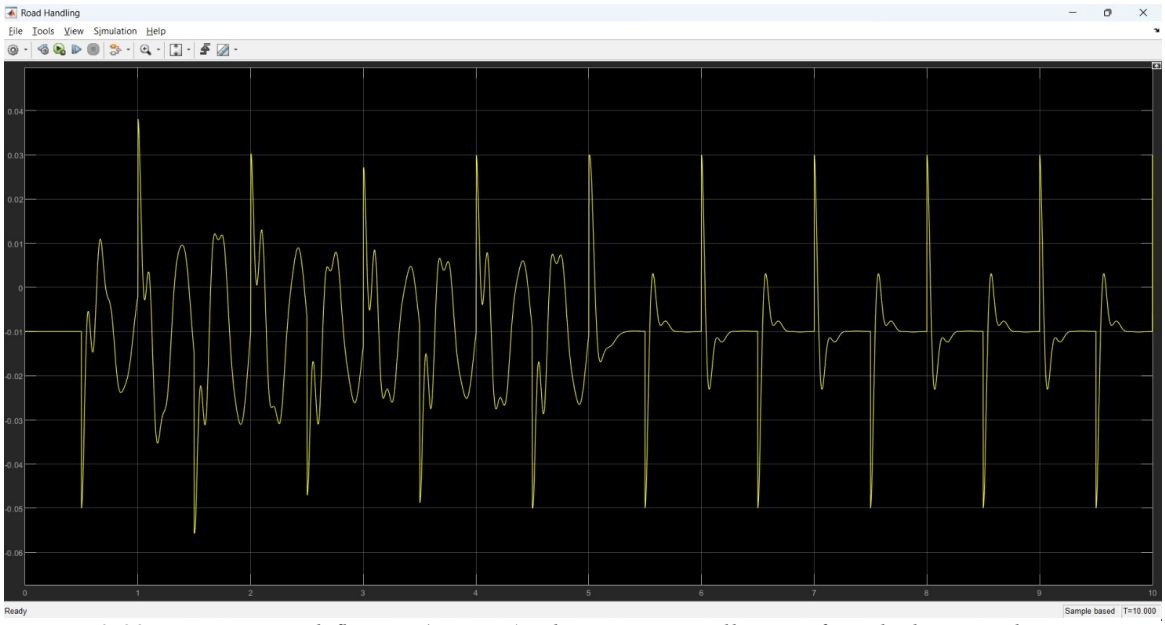

Figure 3-32: Dynamic tire deflection (Zus - Zr). The active controller significantly dampens the unsprung mass oscillations, improving tire-to-road contact forces.

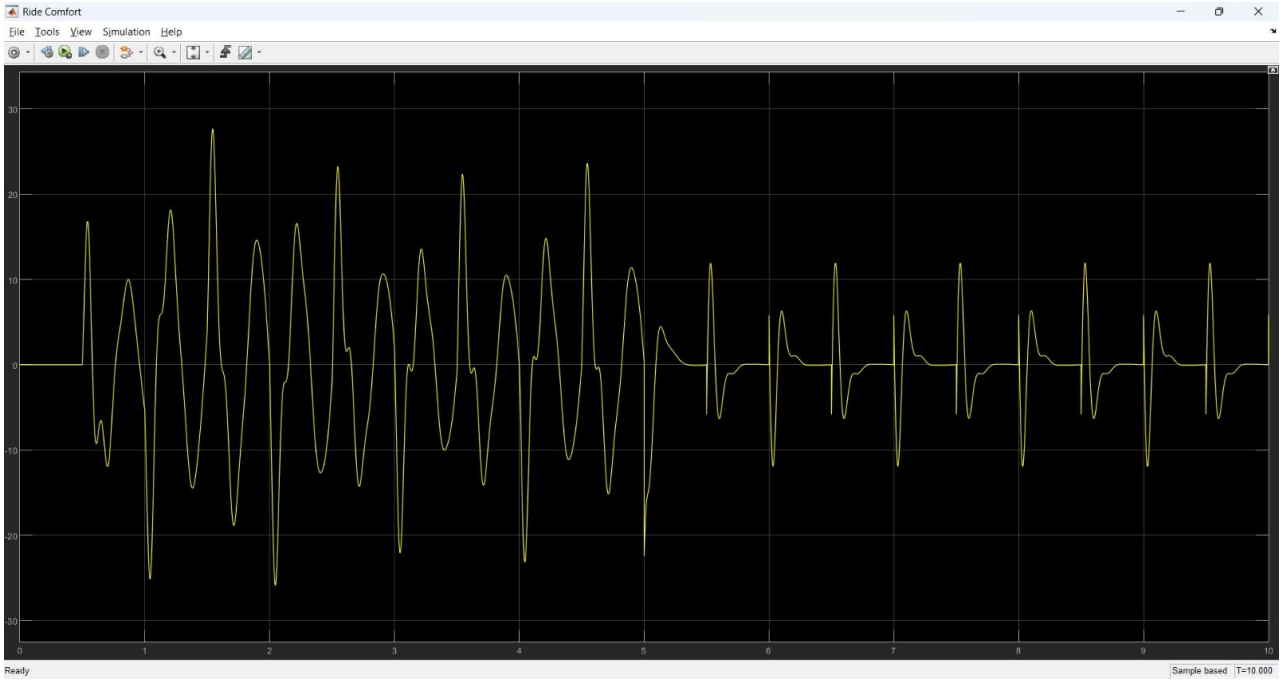

Figure 3-31: Vertical acceleration of the sprung mass (ddotZ_s). The active suspension aggressively attenuates high-frequency acceleration spikes, demonstrating a vast improvement in simulated ride comfort compared to the passive configuration.

3.6 Embedded Software and State Estimation

3.6.1 The Simulink S-Function Interface & Sensor Initialization

Justification for Custom Embedded C++

While standard Simulink support packages offer generic I²C read/write blocks, they proved insufficient for the specific hardware architecture of the active suspension platform. The primary limitation arises from the VL53L0X Time-of-Flight sensors. Unlike the MPU6050 IMUs, which feature a dedicated hardware pin (AD0) to toggle their I²C address between 0x68 and 0x69, the VL53L0X sensors share a single, hardcoded default address (0x29).

To operate multiple ToF sensors on the same I²C bus without data collision, a strict, timing-critical boot sequence is required. This involves pulling the hardware XSHUT pins LOW to disable all sensors, then sequentially bringing them HIGH and assigning unique software addresses (0x30 and 0x31) via I²C commands before the next sensor is awakened. Because native Simulink blocks cannot natively manage this hardware-software initialization handshake, a custom C++ S-Function was developed to serve as the unified embedded interface.

The Sensors Block Architecture

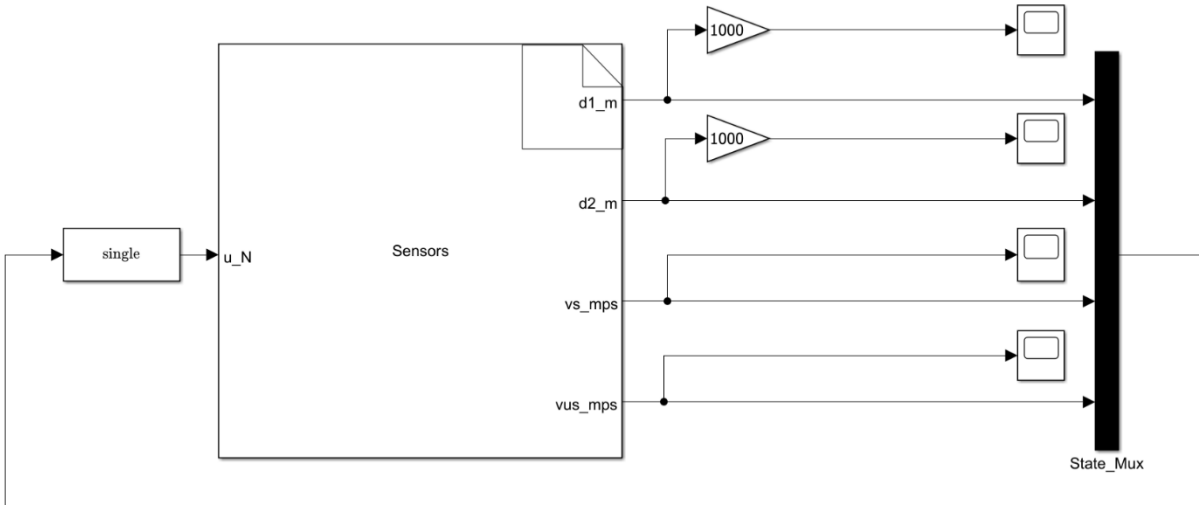

Figure 3-33: The custom Simulink S-Function block acting as the bridge between the ESP32 hardware and the control environment

The custom S-Function block, shown in Figure 3-24, encapsulates the entirety of the FreeRTOS task management, I²C bus polling, and Kalman filtering.

- **Input (u_N):** The block requires a single input—the current actuator control effort (force in Newtons). This is a critical requirement for the Kalman Filter's prediction step, allowing the algorithm to mathematically anticipate the system's next physical state before the sensors register the movement.

- **Outputs (d_1, d_2, v_s, v_{us}):** The block outputs the fully filtered and fused state variables. To ensure the displacements are centered around zero for the LQR controller, the raw ToF measurements are offset by hardcoded baseline values (0.198 m and 0.182 m), which represent the physical resting gaps of the plates at mechanical equilibrium.

3.6.2 Real-Time Operating System (FreeRTOS) Architecture

The Need for Deterministic Execution

Active suspension control requires strictly deterministic execution to maintain system stability. The embedded microcontroller must simultaneously acquire data from four separate I²C sensors, execute complex matrix multiplications for the Kalman Filter, and output a control signal without missing a single computational deadline. A standard "super-loop" (polling) software architecture is inherently insufficient for this level of concurrency, as a delay in reading a slow sensor would physically block the execution of the high-speed control mathematics.

To overcome this, the embedded software was structured using FreeRTOS (Real-Time Operating System). This preemptive multitasking framework allows the software to be divided into isolated, periodic tasks with assigned priorities, ensuring that critical control calculations always preempt slower background processes. The absolute periodicity of these tasks is maintained using the **vTaskDelayUntil()** API, which guarantees precise execution frequencies regardless of the computational load within the task itself.

Task Allocation and Priorities

The data acquisition and filtering workload was strategically divided into two distinct, multi-rate FreeRTOS tasks to optimize CPU utilization and respect the physical limitations of the hardware sensors.

Task Name	Execution Rate	Priority	Primary Function
KfMpuTask	500 Hz (2 ms)	2 (High)	IMU Acquisition & Primary Kalman Update
KfTofTask	50 Hz (20 ms)	1 (Standard)	ToF Acquisition & Secondary Kalman Update

Table 3-4: FreeRTOS task allocation, highlighting execution rates and priority assignments

High Priority Task (KfMpuTask)

Operating at 500 Hz (a 2 ms period), this task serves as the primary heartbeat of the state estimation system. It is assigned the highest priority (Level 2) to ensure the controller can react to high-frequency road disturbances instantly. During its execution window, the task wakes up the two MPU6050 sensors and reads the vertical accelerations of both the sprung and unsprung masses. Immediately following data acquisition, it executes the primary Kalman Filter prediction step ($x_{k+1|k}$) and the IMU measurement update.

Standard-Priority Task (KfTofTask)

Unlike inertial sensors, Time-of-Flight (ToF) sensors physically require longer integration times to emit photons, receive the reflection, and calculate the distance. To accommodate the physical VL53L0X timing budget of 20,000 μs (20 ms), this task operates at a slower rate of 50 Hz. It is assigned a lower priority (Level 1) so that it can run concurrently in the background. When the distance measurements are ready, this task performs the secondary Kalman Filter update step for positional data. This multi-rate architecture ensures that the slow optical measurements never block the high-speed inertial tracking.

3.6.3 I²C Bus Management & Concurrency Control

The Shared Bus Bottleneck

A critical hardware constraint in this embedded architecture is that all four primary sensors—the two VL53L0X Time-of-Flight sensors and the two MPU6050 IMUs—share a single physical I²C bus (SDA: Pin 21, SCL: Pin 22). Because FreeRTOS operates as a preemptive operating system, a severe risk of data collision exists. If the low-priority 50 Hz ToF task is in the middle of a multi-byte I²C read, and the high-priority 500 Hz IMU task wakes up and demands bus access, the physical communication lines would become corrupted, leading to an immediate system lockup.

Mutex Implementation and Bus Recovery

To resolve this concurrent resource limitation, a Mutual Exclusion (Mutex) semaphore (**g_i2cMutex**) was implemented.

- **Resource Locking:** Before any task can initiate an I²C transaction, it must attempt to acquire the **g_i2cMutex** via the custom **I2C_Lock()** wrapper. If the bus is currently in use by another task, the requesting task enters a blocked state until the bus is freed. This guarantees atomic data transfers for every sensor reading.

- **Timeout and Hardware Recovery:** In physical testing environments, electrical noise or loose jumper wires can occasionally cause an I²C slave device to hold the data line LOW, permanently freezing the bus. To prevent this, the Mutex acquisition utilizes a strict timeout mechanism (**pdMS_TO_TICKS(50)).** If a timeout occurs, a dedicated recovery protocol (**recoverI2CBus()**) is triggered. This function physically toggles the SCL pin nine times to clear any stuck slave devices, flushes the buffer, and reinitializes the I²C peripheral at 400 kHz, ensuring continuous, fault-tolerant operation.

3.6.4 Sensor Initialization & Auto-Calibration

Before the control loop engages, the raw physical sensors must be initialized and mathematically centered to provide accurate state feedback to the Kalman Filter.

IMU Bias Removal (Auto-Calibration)

Low-cost MEMS accelerometers naturally exhibit a DC offset (bias) due to manufacturing tolerances and slight deviations in mechanical mounting angles. If left uncorrected, this bias would integrate into massive velocity drift within the state estimator. During the **Sensors_Start** boot sequence, an auto-calibration routine executes. The microcontroller polls both MPU6050 sensors 500 times, accumulating the static Z-axis readings while the suspension is at rest. These values are averaged, converted to standard acceleration units (m/s^2), and stored as **g_mpu1_z_offset** and **g_mpu2_z_offset**. These offsets are dynamically subtracted from all future readings during the 500 Hz control loop, virtually eliminating static gravity and mounting bias.

Time-of-Flight XSHUT Boot Sequence and Centering

Unlike the MPU6050, the VL53L0X sensors do not feature a hardware pin to toggle their I²C addresses. To enable multi-sensor operation, a software-driven initialization sequence (**setVL53IDs()**) was engineered.

1. The ESP32 pulls the XSHUT pins (Pins 25 and 26) LOW to force both sensors into a hardware reset.
2. The first sensor is brought HIGH, initialized, and assigned a unique software address (0x30).
3. The second sensor is then brought HIGH and assigned address 0x31.

Once initialized, the continuous range readings must be normalized for the LQR controller, which operates on the assumption that the system's equilibrium state is exactly zero. To achieve this, the ToF measurements are offset by hardcoded baseline values (0.198 m for the top plate and 0.182 m for the middle plate). These values represent the physical resting gaps

of the plates, allowing the firmware to output pure relative displacements (d_1 and d_2) around the suspension's mechanical equilibrium.

3.6.5 The Multi-Rate Discrete Kalman Filter Formulation

To provide the LQR controller with clean, full-state feedback, a discrete linear Kalman Filter was formulated and embedded in C++. The filter fuses the high-speed, noisy inertial data from the MPU6050s with the slower, high-precision distance measurements from the VL53L0X sensors.

Discretization of the State Space Plant

The matrices embedded within the ESP32 are not arbitrary; they are the direct mathematical discretization of the physical quarter-car model identified in Section 3.4. The system state vector x consists of four variables: $[d_1, d_2, v_s, v_{us}]^T$.

First, the continuous-time state-space matrices (A and B) were constructed using the empirically validated parameters ($M_s = 2.45$, $M_{us} = 1.00$, $K_s = 900$, $K_{us} = 1250$, $B_s = 58.08$, $B_{us} = 14.72$). Because the microcontroller operates in the digital domain, these continuous matrices were transformed into the discrete-time domain using a Zero-Order Hold (ZOH) discretization method at the strict execution sample time of $T_s = 0.002$ s (500 Hz).

This process generated the precise discrete state transition matrix (A_d) and input matrix (B_d) utilized in the prediction step:

$$A_d = \begin{bmatrix} 0.9976 & 0.0023 & 0.0018 & -0.0018 \\ 0.0016 & 0.9976 & 0.0001 & 0.0018 \\ -0.6775 & -0.0561 & 0.9555 & 0.0437 \\ 1.6337 & -2.3253 & 0.1071 & 0.8631 \end{bmatrix}, \quad B_d = \begin{bmatrix} 0.00000265 \\ -0.00000188 \\ 0.00075287 \\ -0.00181533 \end{bmatrix}$$

Note that while the theoretical continuous-time quarter-car model features a 4×2 input matrix (accounting for both the control force u and the road disturbance Z_r), the embedded discrete B_d matrix is reduced to 4×1. This is because the future road disturbance is an unknown, unmeasurable input to the microcontroller. Consequently, the prediction step relies solely on the known control effort, while the unmodeled road excitations are statistically accounted for by the process noise covariance matrix (Q) and corrected during the sensor update steps.

The Measurement and Feedthrough Matrices

The output equations are defined by the measurement matrix (C) and the feedthrough matrix (D). The C matrix maps the theoretical system states directly to the physical sensors.

- **Rows 1 and 2** correspond to the direct positional ToF measurements, essentially acting as an identity mapping (1-to-1).

- **Rows 3 and 4** mathematically relate the system's states to the physical acceleration measured by the IMUs. The coefficients in these rows are derived directly from Newton's Second Law for the sprung and unsprung masses. For example, Row 4 contains the exact physical stiffness and damping constants (900 and -1250) identified in Section 3.4:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -367.34 & 0 & -23.70 & 23.70 \\ 900.00 & -1250.00 & 58.08 & -72.80 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \\ 0.4081 \\ -1 \end{bmatrix}$$

Covariance Tuning

The performance of the Kalman Filter relies on the process noise covariance matrix (Q) and the measurement noise covariance matrices (R). Because the system operates at multiple rates, the measurement noise was decoupled into two distinct matrices: R_{tof} for the optical sensors and R_{mpu} for the inertial sensors. These matrices were tuned experimentally to balance filter responsiveness with noise rejection:

$$Q = diag(10^{-6}, 10^{-4}, 10^{-4}, 10^{-3}) \tag{3.23}$$

$$R_{tof} = diag(10^{-3}, 10^{-3}), \quad R_{mpu} = diag(0.01, 0.01) \tag{3.24}$$

Multi-Rate Execution (Predict and Update)

The execution of the Kalman Filter respects the FreeRTOS architecture outlined in Section 3..2.

Prediction (500 Hz): At every 2 ms interval, the high-priority task calculates the *a priori* state estimate based on the previous state and the current control effort (u_N) from the Simulink controller:

$$x_{k|k-1} = A_d x_{k-1|k-1} + B_d u_{k-1} \tag{3.25}$$

IMU Update (500 Hz): Immediately following the prediction, the filter incorporates the MPU6050 vertical accelerations (a_s, a_{us}) to correct the velocity estimations.

ToF Update (50 Hz): Asynchronously, every 20 ms, the standard-priority task injects the precise gap measurements (d_1, d_2) into the filter. This multi-rate update corrects any long-term integration drift generated by the IMUs, ensuring the absolute positional states remain anchored to physical reality.

3.6.6 Cascaded Signal Filtering

While the Kalman Filter optimally minimizes stochastic noise, real-world structural vibrations and high-frequency harmonics can still propagate into the state estimates, potentially causing aggressive, chattering control outputs from the LQR. To provide a completely smooth control signal, a secondary cascaded filtering architecture was implemented in the **Sensors_Outputs_wrapper** function.

Positional Low-Pass (Alpha) Filtering

The raw position estimates (x_0, x_1) from the Kalman Filter are passed through a first-order exponential smoothing filter (Alpha Filter) with a smoothing factor of $\alpha_p = 0.6$. This acts as a digital low-pass filter, mitigating micro-vibrations in the ToF laser reflections while maintaining rapid tracking of the macroscopic suspension travel.

High-Pass to Low-Pass Velocity Cascade

Extracting pure velocity derivatives from digital data is highly susceptible to noise amplification. To condition the velocity states (x_2, x_3), a cascaded approach was utilized:

- **High-Pass Filter ($\alpha_{hp} = 0.95$):** The raw velocities are first passed through an aggressive high-pass filter to strip away any residual low-frequency integration drift that managed to bypass the Kalman ToF update.

- **Low-Pass Filter ($\alpha_{lp} = 0.15$):** The drift-free signal is then heavily low-pass filtered. This severely attenuates high-frequency acceleration spikes (such as the impact of a harsh road bump), resulting in a smooth, continuous velocity curve (v_s, v_{us}) that is safe to feed into the derivative gains of the LQR matrix.

3.7 Hardware-in-the-Loop (HIL) Control Implementation

While the preceding sections established the theoretical control laws and the embedded state estimation algorithms, calculating a control force in software is insufficient unless it can be physically realized by the actuation hardware. This section details the Hardware-in-the-Loop (HIL) implementation used to bridge the digital control environment with the physical CoTune testbed. Utilizing the Simulink Support Package for Arduino Hardware, the complete control architecture—including the LQR matrix, mathematical conversions, and safety logic—was compiled and deployed directly onto the ESP32 microcontroller. This allows the ESP32 to operate autonomously in real-time, reading sensor buses, evaluating the control law, and generating the high-frequency Pulse Width Modulation (PWM) signals required by the Cytron motor drivers.

3.7.1 Active Suspension Controller Execution

The primary control loop is executed at 500 Hz. The Simulink architecture for this loop was designed to sequentially transform the theoretical LQR control effort (in Newtons) into a discrete electrical signal (an 8-bit PWM duty cycle) that the motor driver can interpret.

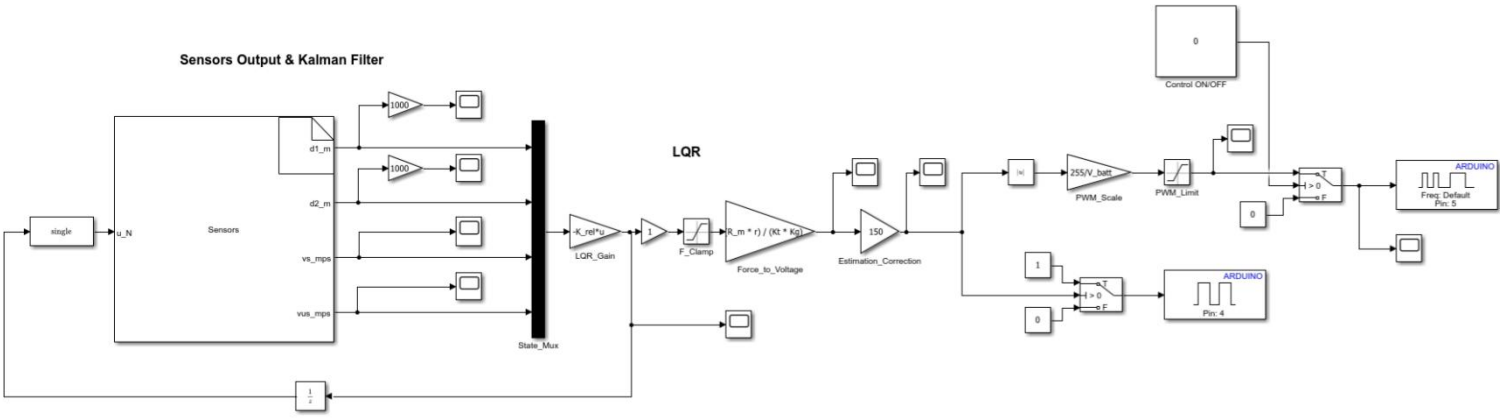

Figure 3-34: The active suspension HIL control loop, detailing the transformation from state-feedback to PWM generation.

As illustrated in Figure 3-34, the control execution follows a strict, sequential signal flow:

State Feedback and Force Calculation: The fully filtered state vector $[d_1, d_2, v_s, v_{us}]^T$ is output from the custom Sensors S-Function. This vector is multiplied by the pre-calculated

LQR gain matrix (-K_rel*u) to determine the optimal active control force (F_c) required to stabilize the system.

Force-to-Voltage Mapping: The Cytron MD30C motor driver does not natively understand force (Newtons); it regulates voltage. Therefore, the requested LQR force must be mathematically converted into a target motor voltage. This is achieved using the electro-mechanical constants identified in Section 3.4. The conversion block calculates the required voltage using the motor armature resistance (R_m), torque constant (K_t), capstan pulley radius (r), and the timing belt gear ratio (K_g):

$$V_{target} = \frac{R_m \cdot r}{K_t \cdot K_g} \cdot F_c \tag{3.26}$$

PWM Scaling and Saturation: The ESP32 generates an 8-bit PWM signal, meaning the hardware timer expects an integer value between 0 (0% duty cycle) and 255 (100% duty cycle). The target voltage is scaled by the available system battery voltage (255 / V_batt) to determine the exact PWM integer. A saturation block (PWM_Limit) is applied immediately after this step to clamp the maximum allowable duty cycle, protecting the 775 DC motor from drawing excessive stall currents.

Directional Logic and Hardware Output: A standard DC motor requires both a magnitude (PWM) and a polarity (Direction) to operate bidirectionally. The scaled signal is split:

- The absolute magnitude ($|u|$) is routed to the ESP32 hardware PWM pin (Pin 5).

- A comparative switch evaluates the sign of the target voltage. If the voltage is positive, the Direction pin (Pin 4) is pulled HIGH (1). If negative, it is pulled LOW (0), allowing the capstan mechanism to actively push or pull the suspension plates seamlessly.

3.7.2 Road Disturbance Mechanism & Limit Switches

To rigorously evaluate the active suspension's performance, the physical road plate must generate deterministic, repeatable disturbance profiles. Rather than relying on open-loop voltage commands, a secondary, parallel Hardware-in-the-Loop (HIL) control structure was implemented on the ESP32 to provide closed-loop position tracking for the road excitation motor.

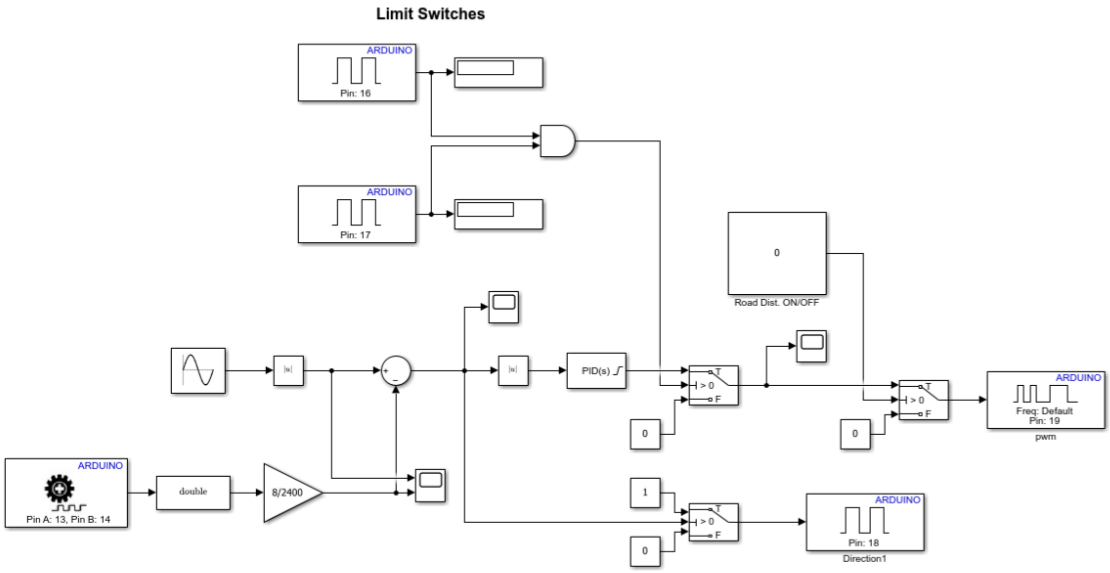

Figure 3-35: Simulink architecture for the road disturbance PID tracking and hardware safety override logic.

Closed-Loop Position Tracking

As depicted in Figure 3-35, the desired road profile is generated mathematically using a discrete Sine Wave block. To ensure the physical road plate precisely tracks this reference, closed-loop position feedback is utilized.

Encoder Feedback: The ESP32 reads the YT06-OP incremental encoder via a hardware interrupt block configured for Pins 13 and 14.

Pulse-to-Displacement Conversion: The raw quadrature pulse count is converted into a linear physical displacement (in meters) by applying a scaling gain of 8/2400. This ratio mathematically maps the 2400 quadrature pulses per revolution (600 PPR × 4) to the exact 8 mm linear pitch of the lead-screw transmission.

PID Execution: The measured physical displacement is subtracted from the reference sine wave to compute the instantaneous tracking error. This error is fed into a discrete PID controller. The controller computes the necessary corrective effort, which is then split into an absolute magnitude (sent to the Cytron PWM port on Pin 19) and a directional boolean (sent to the Direction port on Pin 18).

Mechanical Safety Overrides (Limit Switches)

The lead-screw mechanism driving the road plate provides massive mechanical advantage and cannot be easily back-driven. Consequently, if a software anomaly or aggressive reference signal forces the plate beyond its maximum allowable stroke, the motor could instantly destroy the acrylic frame. To prevent catastrophic failure, a software-enforced hardware kill-switch was integrated directly into the HIL execution loop.

Two digital read blocks continuously monitor physical limit switches mounted at the absolute top and bottom of the road stroke (Pins 16 and 17). These switches are wired in a Normally Closed (NC) configuration, meaning they continuously send a logic HIGH (1) during safe operation.

The signals from both switches are routed through a logical AND gate.

- **Safe Operation:** As long as the plate is within the safe operational zone, the AND gate outputs 1 (True). This triggers the primary signal switch block to pass the live PID control effort directly to the motor driver.

- **Emergency Halt:** If the plate over-travels and strikes either limit switch, that specific pin drops to LOW (0). The AND gate instantly evaluates to 0 (False). This forces the digital switch block to reject the PID signal and instead route a constant 0 to the PWM output, instantly cutting power to the road motor before mechanical binding or structural damage can occur.

3.8 Chapter Summary

This chapter detailed the complete end-to-end development of the CoTune Active Suspension experimental platform. The process began with the mechanical and electrical hardware design, utilizing a dual 775 DC motor architecture to provide independent control over both the suspension actuation and the road disturbance generation.

Following the physical assembly, grey-box system identification was conducted to empirically derive the mechanical stiffness, damping, and electromechanical constants. These parameters successfully grounded the theoretical 2-DOF quarter-car model in physical reality, enabling the formulation of an optimal Linear Quadratic Regulator (LQR) control strategy. Offline simulations in the Simscape-Simulink environment verified that the LQR mathematically stabilized the system and effectively attenuated aggressive road disturbances.

Finally, the transition from theoretical simulation to physical reality was achieved through a comprehensive embedded software architecture. By utilizing a multi-rate FreeRTOS framework on an ESP32 microcontroller, the system successfully managed concurrent I^2C sensor acquisition and executed a discrete Kalman Filter for full-state estimation. The Hardware-in-the-Loop (HIL) deployment bridged the digital control logic with the physical Cytron motor drivers, incorporating strict mechanical safety overrides to ensure robust, autonomous operation. With the local active suspension testbed fully operational, the subsequent chapters will detail the development of the secondary rotary inverted pendulum platform and the overarching cloud-telemetry infrastructure required for remote experimentation.

CHAPTER 4 ROTARY INVERTED PENDULUM

4.1 Introduction

The rotary inverted pendulum, commonly known as the Furuta pendulum, is a benchmark electromechanical system utilized extensively in advanced control theory education and research. Unlike a linear inverted pendulum, which balances a rod on a cart restricted to a finite linear track, the Furuta pendulum features a base arm that rotates in the horizontal plane to balance a free-swinging vertical pendulum.

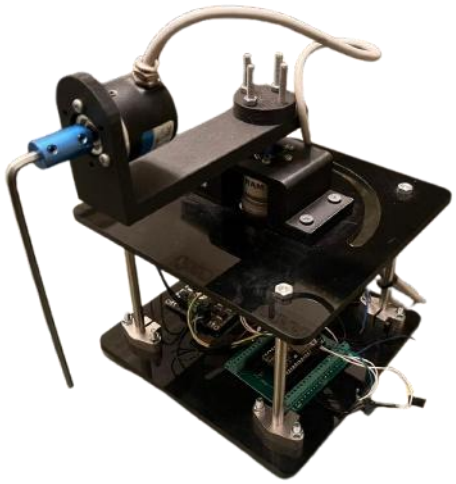

Figure 4-1: The physical assembly of the rotary inverted pendulum testbed

This platform presents a highly complex control challenge: it is an underactuated, highly non-linear system possessing two degrees of freedom (DOF) but only a single control input (the motor torque applied to the rotary arm). The control objective is inherently dual-phased. First, the system must autonomously execute a "swing-up" maneuver to drive the unactuated pendulum from its stable, downward resting position to the unstable upright equilibrium. Second, it must immediately transition to a stabilization controller to maintain the upright position against gravitational forces and external disturbances. This chapter details the comprehensive mechanical design, dynamic mathematical modeling, system identification, and Hardware-in-the-Loop (HIL) control implementation required to achieve this objective.

4.2 Mechanical and Electrical Hardware Architecture

The physical architecture of the rotary inverted pendulum must facilitate rapid, low-friction rotation while ensuring robust structural rigidity to withstand aggressive dynamic maneuvers. The system is composed of two primary mechanical links: the actuated rotary arm (operating in the horizontal plane) and the unactuated pendulum link (operating in the vertical plane).

4.2.1 Actuation System and Wire Management Constraints

Actuation is provided by a permanent magnet DC (PMDC) motor mounted vertically at the base, with its shaft directly coupled to the central pivot hub of the horizontal rotary arm.

Figure 4-2: Rotary Inverted Pendulum DC Motor

Motor Type	Operating Voltage	No-Load Speed	Rated Current	Stall Torque	Weight
SG555123000. Motor	12 VDC	300 RPM at 12 V	800 mA at 12 V	4 N.cm at 12 V	350 g

Table 4-1: Rotary Inverted Pendulum DC Motor Specifications

Unlike fully continuous rotational systems that utilize slip rings, this testbed employs a direct-wired architecture. The signal wires for the distal pendulum encoder are routed directly along the top of the rotary arm and down through the central rotational axis. This introduces a critical physical constraint to the system: the rotary arm cannot rotate infinitely in a single direction without entangling and potentially severing the sensor wires. Consequently, the energy-based swing-up control algorithm (detailed in Section 4.6) must be carefully tuned to achieve an upright pendulum state within a limited number of arm revolutions.

4.2.2 Sensor Suite and State Measurement

Full-state feedback is critical for both the swing-up and balancing phases. The system relies on a dual-encoder architecture to accurately measure the angular positions of both mechanical links:

- **Motor Encoder (Rotary Arm Angle, θ):** A high-resolution quadrature encoder is mounted directly to the rear shaft of the base DC motor. This sensor tracks the absolute angular displacement of the horizontal arm relative to the stationary base.

- **Pendulum Encoder (Pendulum Angle, α):** A secondary incremental encoder is mounted at the distal end of the rotary arm, serving as the pivot point for the vertical pendulum link. This sensor measures the relative angle of the pendulum with respect to the gravitational vector.

Because standard optical encoders only provide discrete positional data, the angular velocities of the arm ($\dot{\theta}$) and the pendulum ($\dot{\alpha}$) cannot be measured directly. Instead, these critical velocity states are derived computationally within the HIL software via numerical differentiation and high-frequency low-pass filtering.

4.3 Mathematical Modeling

The rotary inverted pendulum consists of an actuated rotary arm and a passive pendulum link mounted at the arm tip. The generalized coordinates are chosen as $q = [\theta, \alpha]^T$, where θ is the rotary arm angle and α is the pendulum angle measured from the upright vertical position. Counter-clockwise rotation is defined as positive for both angles.

4.3.1 Dynamic Model and Parameter Mapping

The system is modeled as two rigid bodies interacting dynamically. The primary generalized coordinates are the rotary arm angle (θ) and the pendulum angle (α). The physical parameters governing the system include the mass of the arm (m_r) and pendulum (m_p), the arm radius (r), the pendulum's center of mass length (l_p), and their respective moments of inertia (J_r, J_p). Viscous friction coefficients for the motor shaft (B_r) and the pendulum pivot (B_p) are also incorporated to capture non-ideal mechanical losses.

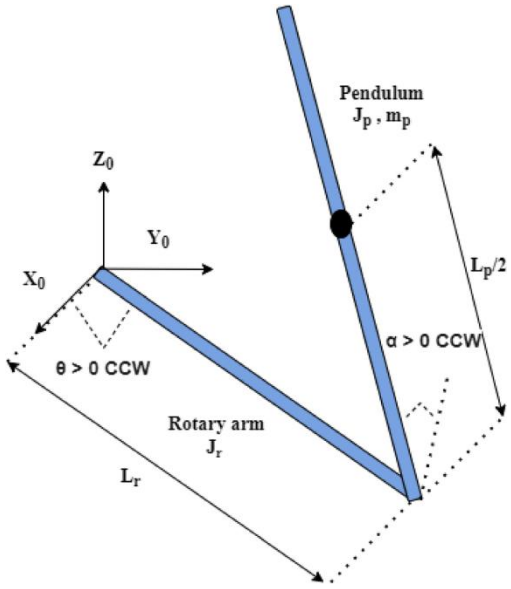

Figure 4-3: Rotary Inverted Pendulum FBD

4.3.2 Non-Linear Equations of Motion

The nonlinear equations of motion describe the coupled dynamics between the actuated rotary arm and the passive pendulum link. They are derived using the Euler–Lagrange formulation, where the system's kinetic and potential energies are expressed in terms of the generalized coordinates θ and α. The resulting equations capture inertial coupling, gravitational effects, and viscous damping, while the rotary arm is driven by a motor-generated torque. These nonlinear dynamics represent the full physical behavior of the system and form the basis for subsequent linearization about the upright equilibrium used for control design.

$$\left(m_p L_r^2 + \frac{1}{4} m_p L_p^2 - \frac{1}{4} m_p L_p^2 \cos^2(\alpha) + J_r\right)\ddot{\theta} - \left(\frac{1}{2} m_p L_p L_r \cos(\alpha)\right)\ddot{\alpha} + \left(\frac{1}{2} m_p L_p^2 \sin(\alpha)\cos(\alpha)\right)\dot{\theta}\dot{\alpha} + \left(\frac{1}{2} m_p L_p L_r \sin(\alpha)\right)\dot{\alpha}^2 = \tau - B_r\dot{\theta} \tag{4.1}$$

$$-\frac{1}{2} m_p L_p L_r \cos(\alpha)\,\ddot{\theta} + \left(J_p + \frac{1}{4} m_p L_p^2\right)\ddot{\alpha} - \frac{1}{4} m_p L_p^2 \cos(\alpha)\sin(\alpha)\,\dot{\theta}^2 - \frac{1}{2} m_p L_p g \sin(\alpha) = -B_p\dot{\alpha} \tag{4.2}$$

4.3.3 Motor Torque-Voltage Relationship

The motor torque–voltage relationship is incorporated into the rotary arm equation of motion, allowing the system dynamics to be expressed directly in terms of the motor input voltage. This formulation captures the effect of motor back-emf as a velocity-dependent damping term and provides a suitable model for control design. The derived equations follow the standard form of equations of motion, where inertial, damping, and gravitational effects are balanced by the applied torque.

$$\tau = \frac{\eta_g K_g \eta_m k_t \left(V_m - K_g k_m \dot{\theta}\right)}{R_m} \tag{4.3}$$

4.3.4 Linearized Model and State Space Representation

The nonlinear equations of motion are linearized about the upright equilibrium configuration of the pendulum. This operating point corresponds to zero angular velocities and small deviations of the pendulum angle from the vertical position ($\theta = 0, \alpha = 0, \dot{\theta} = 0, \dot{\alpha} = 0$), allowing the system to be approximated by a linear time-invariant model suitable for control design.

Under the small-angle assumption, the trigonometric terms are approximated as $\sin(\alpha) \approx \alpha$ and $\cos(\alpha) \approx 1$, while higher-order nonlinear terms are neglected. Applying these approximations yields the following linearized equations, which capture the dominant coupled dynamics near the upright position.

The system is formulated into the standard continuous-time linear state-space representation ($\dot{x} = Ax + Bu$ and $y = Cx + Du$). The system matrix (A) and input matrix (B) describe the coupling between the rotary arm and pendulum dynamics, as well as the influence of the motor input voltage on the system states. Because only the positions of the motor and link angles are physically measured by the encoders, the output matrices (C and D) are defined to extract only these observable states.

Linear State Space Model

The Linear State Space equations are

$$x = Ax + Bu \tag{4.4}$$

And

$$y = Cx + Du \tag{4.5}$$

For the linearized rotary inverted pendulum system, the system matrix $A$and input matrix $B$are obtained directly from the linearized equations of motion. These matrices describe the coupling between the rotary arm and pendulum dynamics, as well as the influence of the motor input voltage on the system states.

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & a_{32} & a_{33} & a_{34} \\ 0 & a_{42} & a_{43} & a_{44} \end{bmatrix} \tag{4.6}$$

$$B = \begin{bmatrix} 0 \\ 0 \\ b_3 \\ b_4 \end{bmatrix} \tag{4.7}$$

Where:

$$a_{32} = \frac{\left(\frac{1}{2} {m_p}^2 {L_p}^2 g L_r\right)}{\Delta} \tag{4.8}$$

$$a_{33} = -\frac{\left(J_p + \frac{m_p {L_p}^2}{4}\right) B_r}{\Delta} \tag{4.9}$$

$$a_{34} = \frac{\frac{1}{2} m_p L_p L_r B_p}{\Delta} \tag{4.10}$$

$$a_{42} = \frac{\left(J_r + \frac{m_p {L_r}^2}{4}\right) \frac{1}{2} m_p L_p g}{\Delta} \tag{4.11}$$

$$a_{43} = \frac{\frac{1}{2} m_p L_p L_r B_r}{\Delta} \tag{4.12}$$

$$a_{44} = \frac{\left(J_r + \frac{m_p {L_r}^2}{4}\right) B_p}{\Delta} \tag{4.13}$$

$$b_3 = \frac{J_p + \frac{m_p {L_p}^2}{4}}{\Delta} \tag{4.14}$$

$$b_4 = \frac{\frac{1}{2} m_p L_p L_r}{\Delta} \tag{4.15}$$

Where x is the state, u is the control input, A, B, C, and D are state space matrices. For the rotary pendulum system, the state and output are defined

$$x^T = [\theta \ \ \alpha \ \ \dot{\theta} \ \ \dot{\alpha}] \tag{4.16}$$

And

$$y^T = [x_1 \ \ x_2] \tag{4.17}$$

In the output equation, only the position of the motor and link angles are measured. Based on this, the C and D matrices in the output equation are

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{4.18}$$

$$D = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{4.20}$$

4.4 System Parameter Identification

Since the pendulum dynamics are directly influenced by the actuator behavior, the parameter estimation process was structured into two sequential stages. First, the electrical and mechanical parameters of the DC motor were identified independently. These identified motor parameters were then fixed and used in the second stage to estimate the remaining mechanical parameters of the rotary arm–pendulum system. This staged approach improves identifiability, reduces coupling between parameters, and increases confidence in the resulting model.

4.4.1 Stage 1: Motor Parameter Estimation

In the first stage, the DC motor driving the rotary arm was modeled independently from the pendulum dynamics. The objective of this phase was to identify the motor's electrical and mechanical parameters without the influence of the pendulum load. The motor model includes the armature electrical dynamics and the rotational mechanical dynamics, accounting for back-EMF, torque generation, inertia, and viscous damping. The applied input to the model was the motor voltage, while the measured output was the motor angular speed.

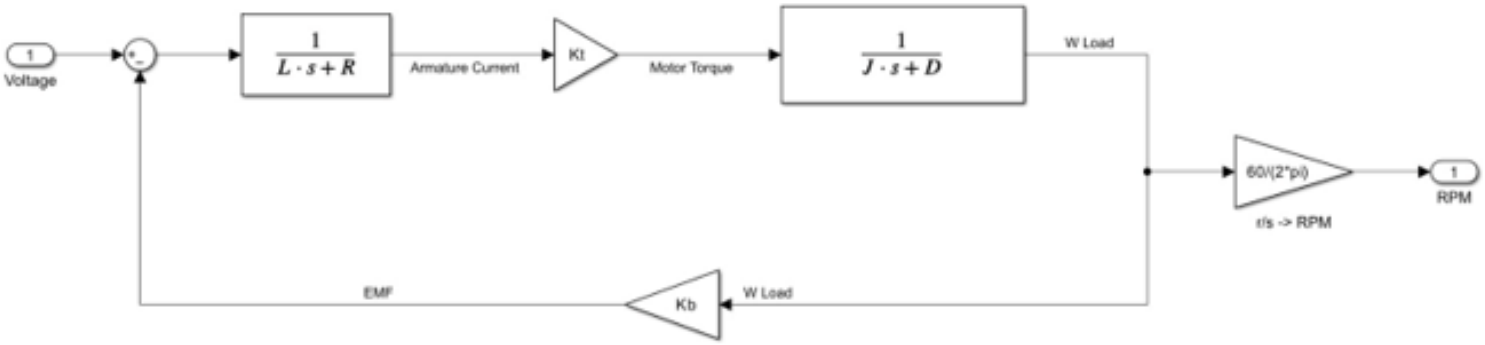

Figure 4-4:DC Motor Block Diagram Model

Parameter	Symbol	Value	Unit
Armature Resistance	R	3.9336	Ω
Armature Inductance	L	0.24721	H
Motor Torque Constant	K_t	0.39595	$N \cdot m/A$
Motor Inertia	J	0.0025202	$kg \cdot m^2$
Viscous Damping Coefficient	D	4.329×10^{-6}	$N \cdot m \cdot s/rad$

Table 4-2: Identified DC Motor Parameters

4.4.2 Stage 2: Coupled System Estimation

In the second phase, the estimated motor parameters were incorporated into the full rotary inverted pendulum model. The remaining mechanical parameters associated with the rotary arm and pendulum were then identified using the coupled arm–pendulum dynamic equations.

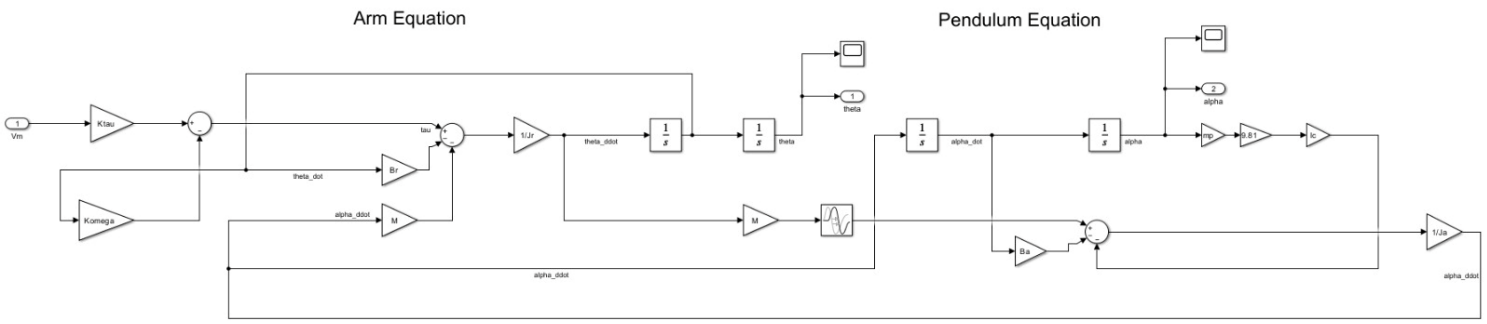

Table 4-3: Linearized Rotary Inverted Pendulum Block Diagram

Parameter	Symbol	Value	Unit
Rotary Arm Inertia	J_r	0.00061423	$kg \cdot m^2$
Pendulum Link Inertia	J_p	0.00061677	$kg \cdot m^2$
Rotary Arm Viscous Damping	B_r	2.2119×10^{-6}	$N \cdot m \cdot s/rad$

Table 4-4: Identified Mechanical System Parameters

4.5 Simulation Framework and Control Design

This section presents the simulation framework utilized to evaluate the rotary inverted pendulum. The primary objective of the simulation is to validate the derived dynamic model using the empirically estimated parameters, assess the effectiveness of the proposed control strategy, and establish consistency between the simulated behavior and the physical experimental results.

4.5.1 Full Model Architecture

The top-level simulation environment integrates the physical plant, the control logic, and the state routing into a unified closed-loop system.

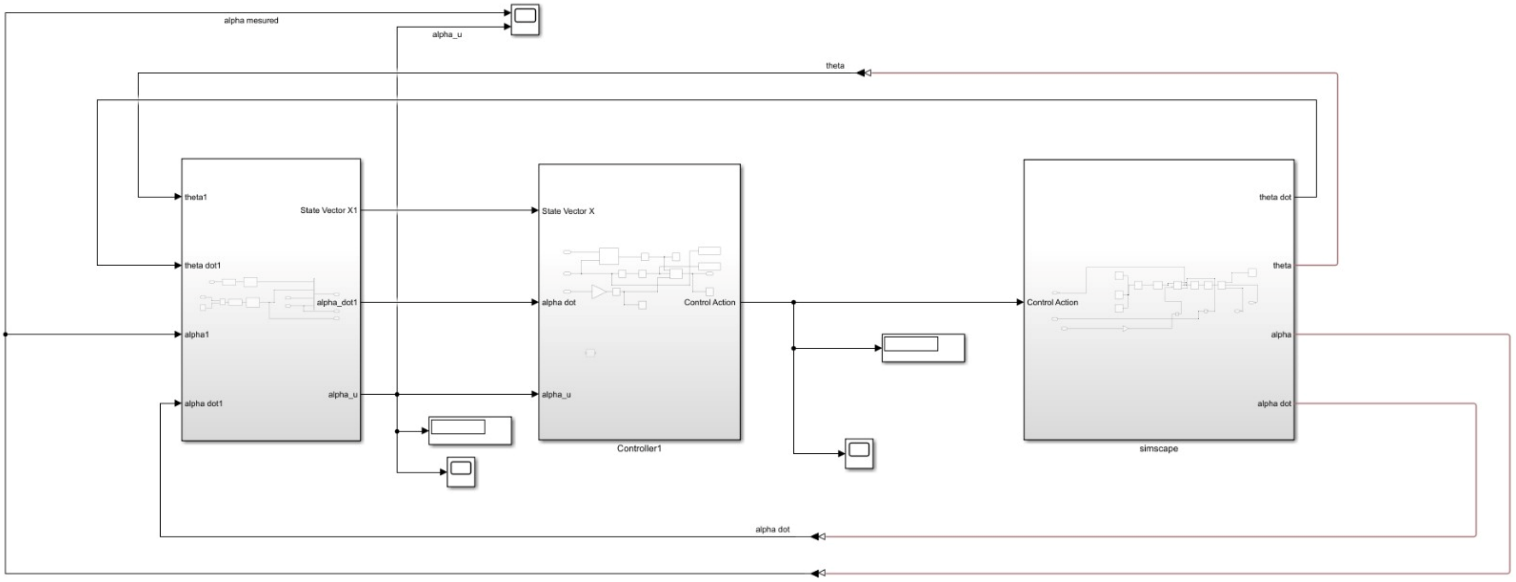

Figure 4-5: The top-level Simulink architecture integrating the Simscape Multibody plant, the controller subsystem, and the signal routing.

At this macroscopic level, the architecture is designed for modularity. The continuous-time physical outputs generated by the plant are routed back to the discrete-time controller subsystem. The controller subsequently evaluates the physical states and outputs a commanded control effort, which is fed directly back into the actuation joint of the plant.

4.5.2 The Plant Subsystem and Physical Multibody

To accurately capture the non-linear, 3D spatial dynamics of the system, the mechanical testbed was modeled using Simscape Multibody rather than relying exclusively on mathematical transfer functions.The internal architecture of the plant is divided into four highly specific sub-assemblies:

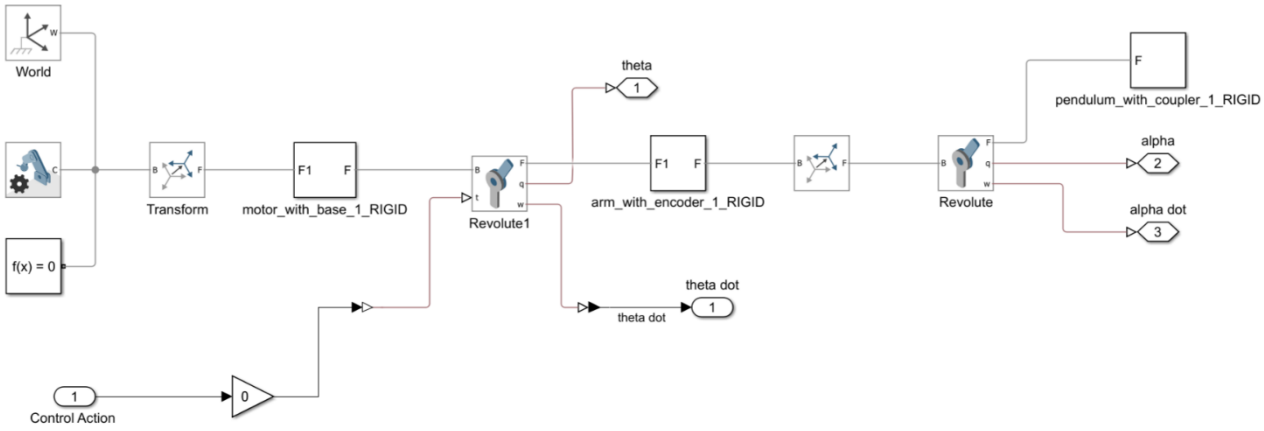

Figure 4-7: The complete plant subsystem, containing the physical modeling blocks and actuation inputs.

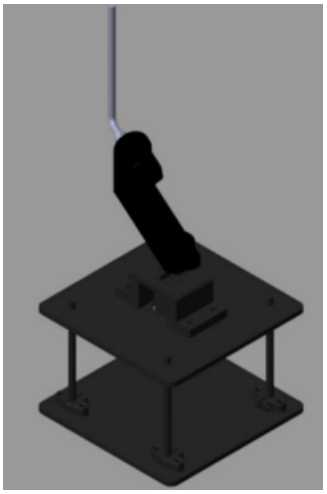

Figure 4-6: The internal Simscape Multibody architecture, detailing the rigid transforms and physical links.

The plant subsystem encapsulates the entire physical reality of the Furuta pendulum. It utilizes Solid blocks to represent the mass and inertia of the rotary arm and the pendulum link, populated with the exact empirical values (J_r, J_p) identified in Section 4.4. Revolute Joint blocks are used to define the specific axes of rotation. The base revolute joint acts as the actuated motor, accepting the incoming control torque, while the distal revolute joint acts as the passive, free-swinging pendulum pivot.

4.5.3 State Vector Subsystem

To execute the state-feedback control laws, the raw physical data from the Simscape plant must be conditioned into a clean, mathematical array.

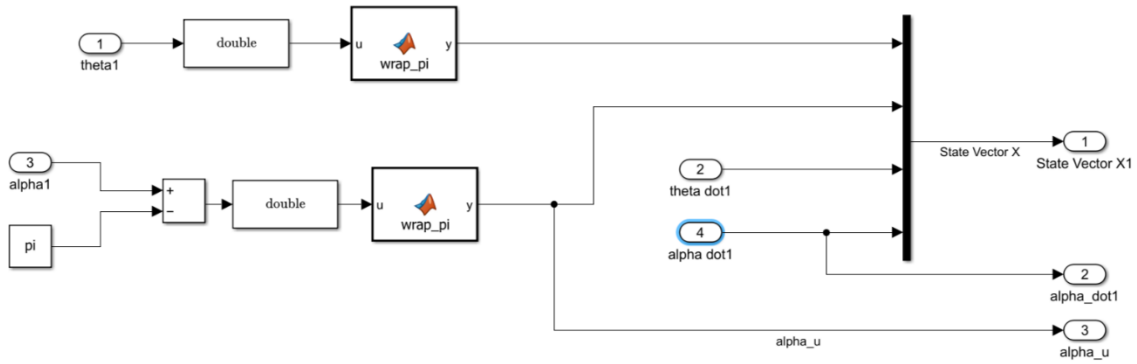

Figure 4-8: The State Vector subsystem, responsible for extracting and formatting the feedback signals.

This subsystem acts as the data-processing bridge. It extracts the raw angular positions (θ, α) from the Simscape joint sensors. Because full-state feedback requires velocities, the subsystem processes these position signals to derive the angular velocities ($\dot{\theta}, \dot{\alpha}$). The four variables are then multiplexed into a single, cohesive state vector $x = [\theta, \alpha, \dot{\theta}, \dot{\alpha}]^T$, perfectly aligning with the mathematical framework established during the LQR formulation.

4.5.4 Controller Subsystem

The "brain" of the simulation is the hybrid controller, tasked with solving the dual-phase challenge of an underactuated pendulum.

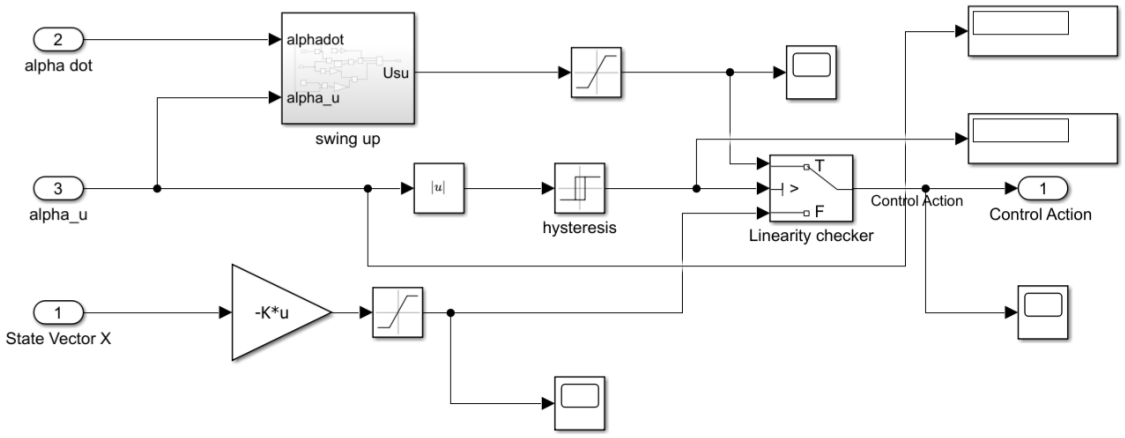

Figure 4-9: he Full Controller subsystem, detailing the switching logic between the Swing-Up and LQR algorithms.

The Full Controller subsystem manages the transition between large-angle, non-linear maneuvering and small-angle linear stabilization. It continuously monitors the pendulum angle (α). If the pendulum is pointing downwards or outside the linear stabilizable region, the system activates the Swing-Up subsystem. Once the pendulum enters a narrow, predefined threshold near the upright vertical equilibrium (e.g., $\pm 15°$), a switching condition triggers, instantly disabling the swing-up logic and engaging the LQR matrix to catch and balance the pendulum.

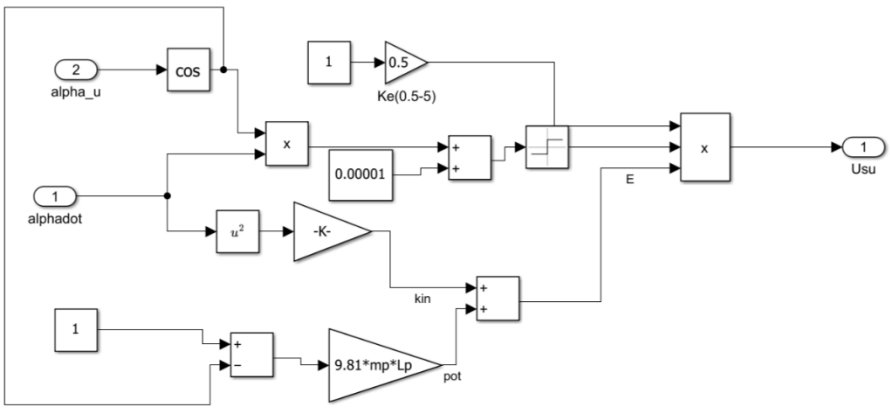

Figure 4-10: The internal logic of the Swing-Up subsystem.

The dedicated Swing-Up subsystem executes an energy-based control law. Rather than trying to force the pendulum directly to a specific angle, this subsystem calculates the total mechanical energy of the pendulum. It generates an oscillatory control command that rhythmically pumps kinetic energy into the rotary arm, gradually increasing the amplitude of the pendulum's swing until it reaches the critical threshold required for the LQR to take over.

4.6 Hardware-in-the-Loop (HIL) Implementation

While Section 4.5 validated the control architecture in a purely simulated environment, physical deployment requires managing real-world hardware constraints, including sensor noise, integer conversions, and electrical actuation mapping. This section details the Hardware-in-the-Loop (HIL) implementation used to deploy the hybrid controller directly onto the ESP32 microcontroller using the Simulink Support Package for Arduino Hardware.

4.6.1 Embedded HIL Architecture

The top-level HIL model completely replaces the Simscape physical plant with actual hardware Input/Output (I/O) blocks.

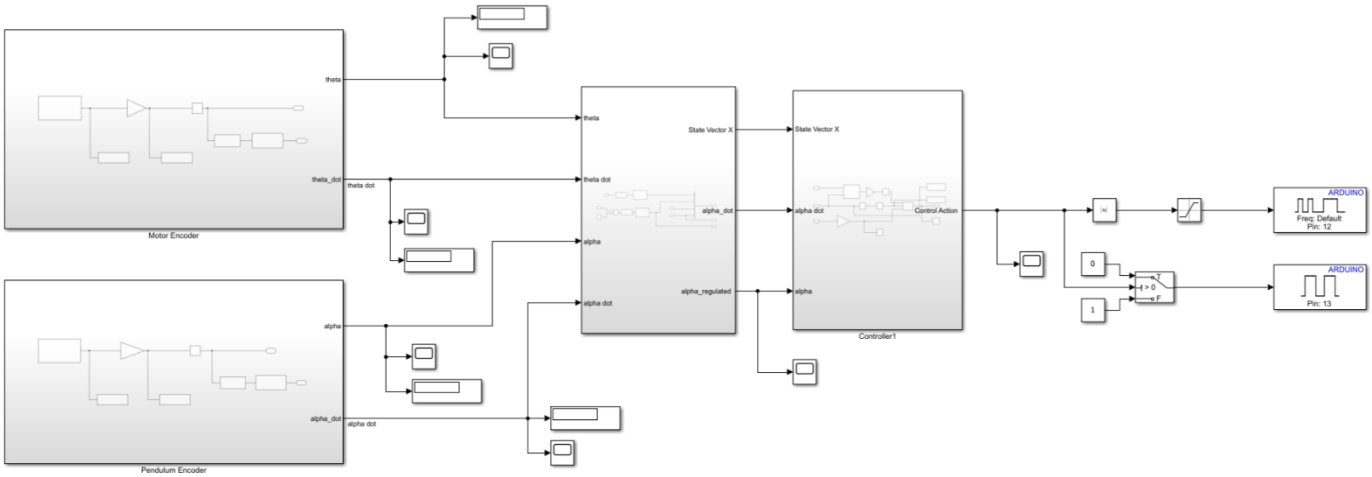

Figure 4-11: The top-level HIL architecture, bridging the digital control subsystem with the physical ESP32 hardware

Instead of routing continuous mathematical torque signals to a virtual 3D joint, the controller now interfaces with the physical environment. The model is compiled into C++ and flashed onto the ESP32, allowing it to autonomously sample the physical encoders, process the hybrid control loop at a high discrete frequency, and command the Cytron motor driver in real-time.

4.6.2 Sensor Processing and Velocity Estimation

Physical quadrature encoders do not natively output radians or velocities; they output discrete integer pulses. A dedicated subsystem was built to condition these raw electrical signals into the continuous mathematical state vector required by the controller.

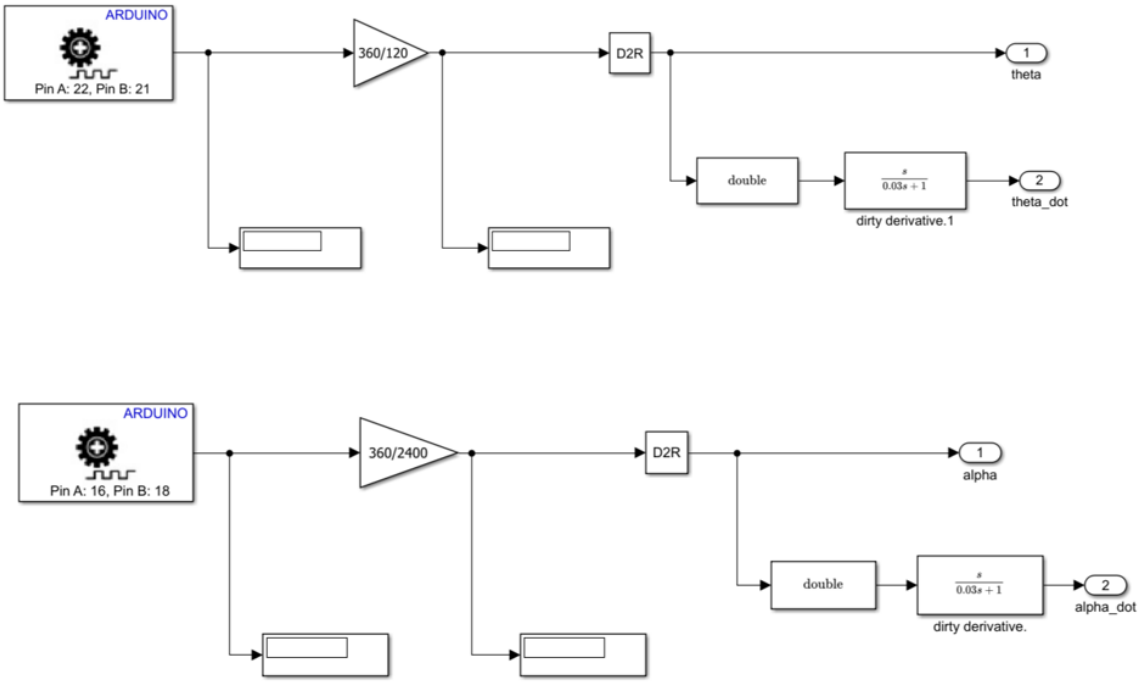

Figure 4-12: The hardware sensor processing subsystem, detailing the encoder-to-radian conversion and discrete velocity derivatives.

As shown in Figure 4-11, the ESP32 reads the high-speed quadrature pulses from the base motor and the distal pendulum pivot.

Pulse-to-Radian Conversion: The raw ticks are converted into physical angular displacements. The base encoder count is scaled by a specific gain of $\pm 2\pi/2400$ (representing a 600 Pulse-Per-Revolution encoder operating in 4X quadrature mode). A similar gain of $\pm 2\pi/2000$ is applied to the pendulum encoder.

Velocity Derivation: Because hardware encoders only provide position (θ, α), the angular velocities $(\dot{\theta}, \dot{\alpha})$ must be calculated dynamically. The radian signals are passed through Discrete Derivative blocks. To prevent the amplification of high-frequency sensor quantization noise, these derivatives are heavily filtered before being passed to the state vector array.

4.6.3 Embedded Hybrid Control Execution

The core intelligence of the ESP32 lies in the seamless execution of the dual-phase control strategy.

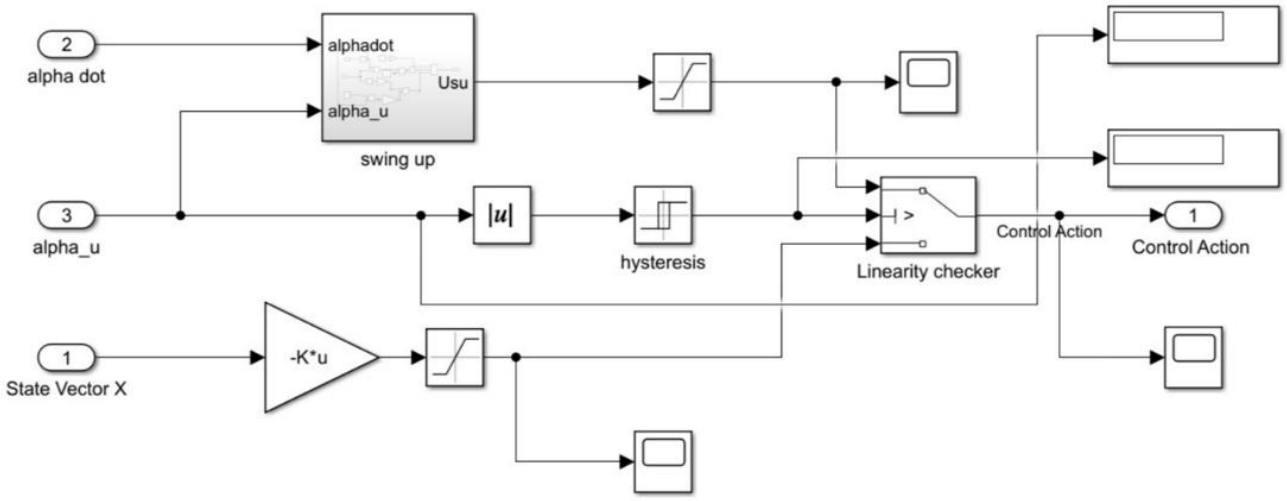

Figure 4-13: The embedded Full Controller subsystem, highlighting the autonomous switching logic between the Swing-Up and LQR phases.

The controller continuously monitors the physical pendulum angle (α). An absolute value comparative threshold governs the switching logic. While the pendulum is hanging downward or outside the linear stabilizable zone, the switch forces the system into Swing-Up mode. The instant the physical pendulum crosses into the predefined upright threshold, the switch flips, instantly routing the LQR control effort to the motors to "catch" and balance the system.

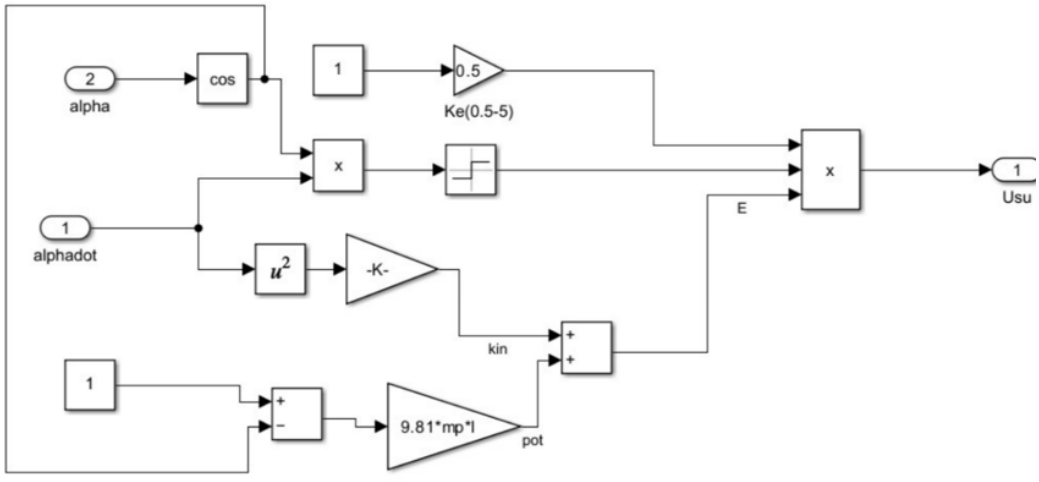

Figure 4-14: The internal mathematical execution of the Energy-Based Swing-Up controller.

During the swing-up phase, the dedicated subsystem evaluates the instantaneous mechanical energy of the pendulum. By utilizing a network of trigonometric math blocks and signal multipliers, the ESP32 calculates the error between the pendulum's current resting energy and the potential energy required to balance it upright. The algorithm outputs a highly dynamic, oscillatory control effort, pumping the rotary arm back and forth to build kinetic energy in the pendulum link without wrapping the direct-wired cables around the central hub.

4.7 Physical Actuation Mapping

The final stage of the HIL loop translates the theoretical control effort into an electrical format the Cytron motor driver can interpret.

Pulse Width Modulation (PWM): The absolute magnitude of the required effort is routed to an Arduino PWM block configured for ESP32 Pin 19. A saturation limit is applied prior to this block to ensure the commanded duty cycle does not exceed the maximum allowable operating voltage of the motor, protecting the hardware from overcurrent damage.

Directional Logic: A conditional switch evaluates the sign of the required effort. The resulting boolean logic (1 for positive torque, 0 for negative torque) is routed to a standard Digital Output block on Pin 18, commanding the Cytron driver to seamlessly alternate the motor's rotational direction.

4.8 Simulation Results and Performance Evaluation

The performance of the rotary inverted pendulum system is evaluated through both simulation and real-time implementation on the physical plant. The system response is assessed using the pendulum angle α, pendulum angular velocity $\dot{\alpha}$, rotary arm angle θ, and rotary arm angular velocity $\dot{\theta}$. The results show successful swing-up and stabilization of the pendulum around the upright equilibrium, with close qualitative agreement between simulated and experimental responses, confirming the validity of the dynamic model and the effectiveness of the implemented control strategy.

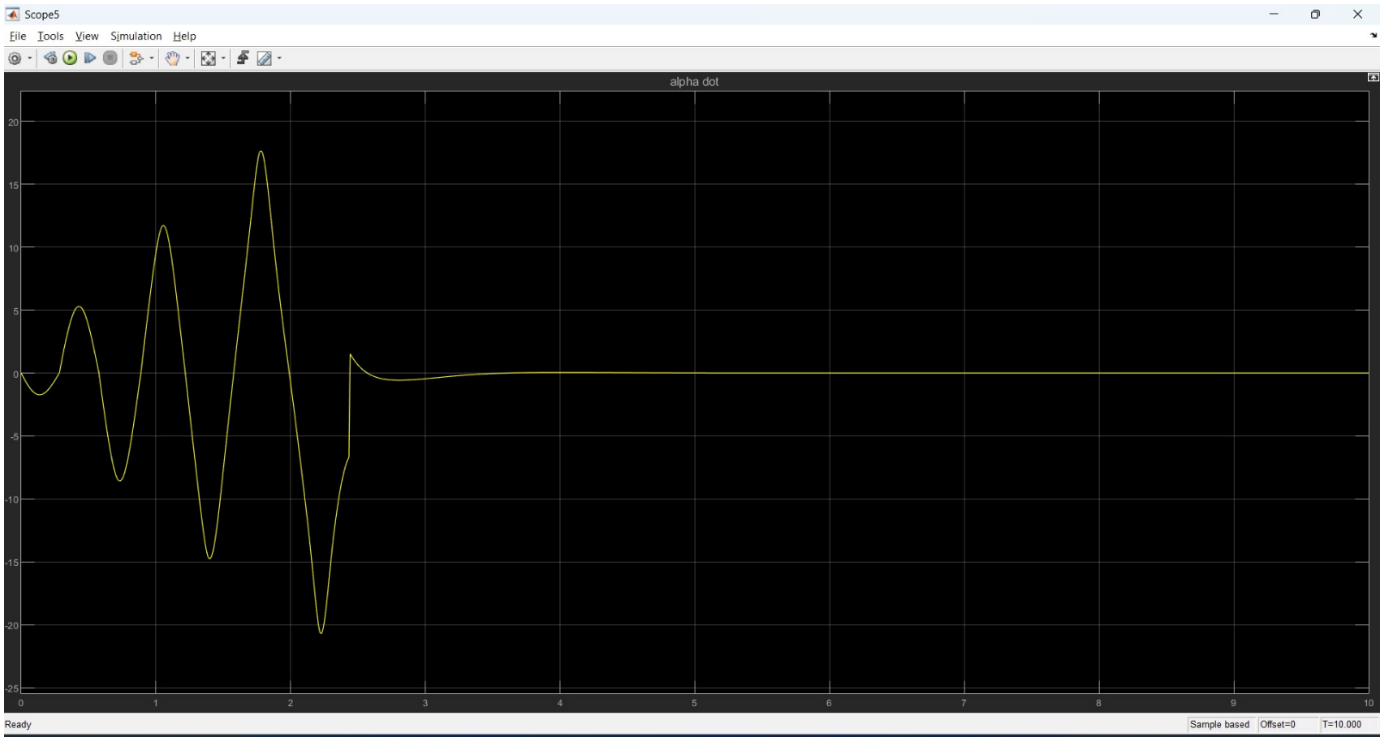

Figure 4-16: Real-time telemetry of the rotary arm angular position (θ). The trajectory illustrates the large-angle sweeping maneuvers generated by the swing-up controller, followed by the high-frequency positional corrections executed by the LQR to maintain

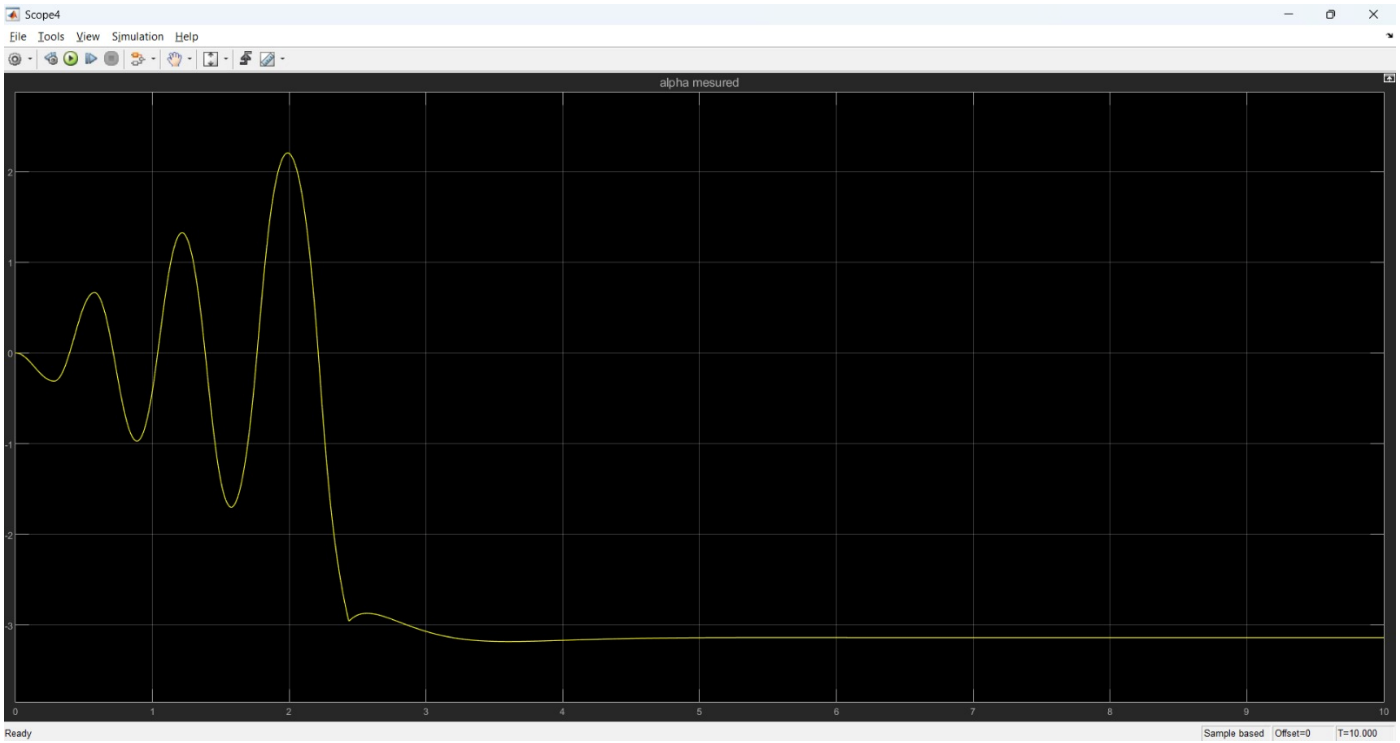

Figure 4-15: Real-time telemetry of the pendulum angular position (α). The graph captures the complete state transition from the downward resting equilibrium (−π radians) to the successful capture and stabilization at the unstable upright equilibrium.

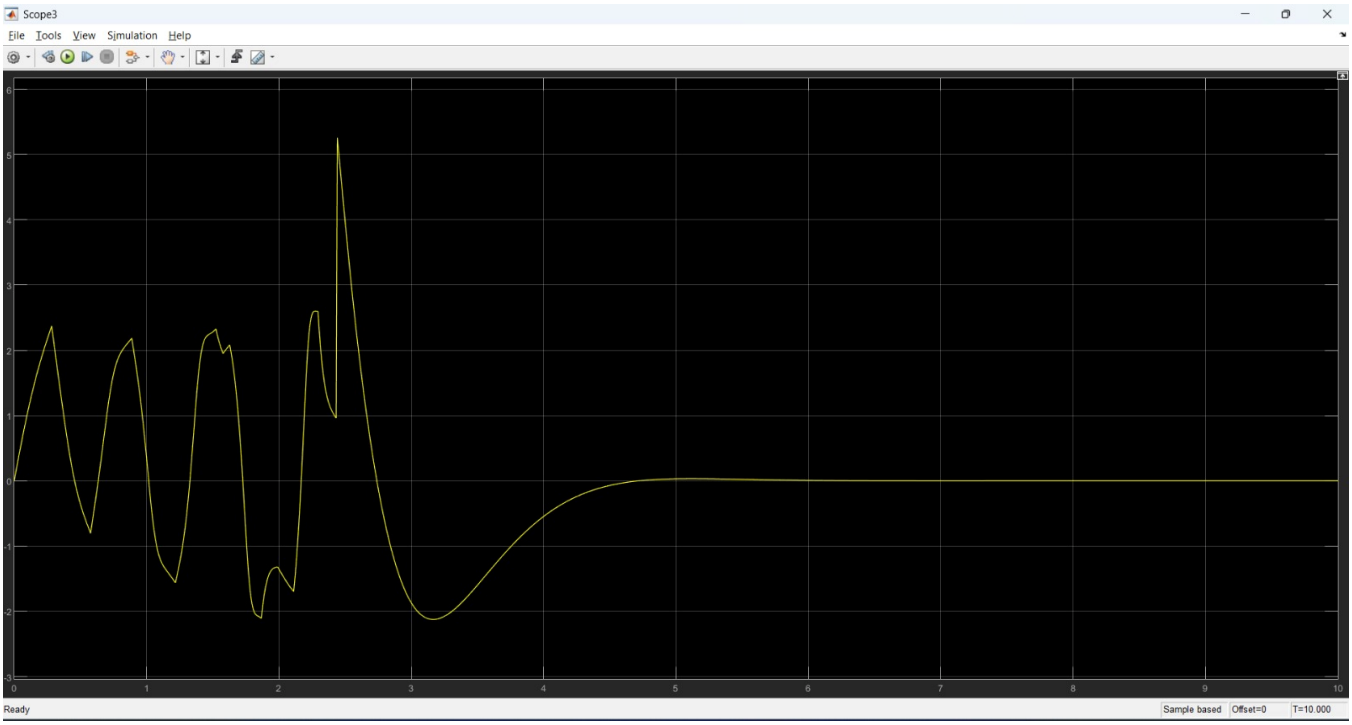

Figure 4-18: Rotary arm angular velocity ($\dot{\theta}$). The data demonstrates the aggressive, high-speed motor reversals required to rhythmically pump kinetic energy into the system during the initial phase, settling into near-zero velocity during steady-state b

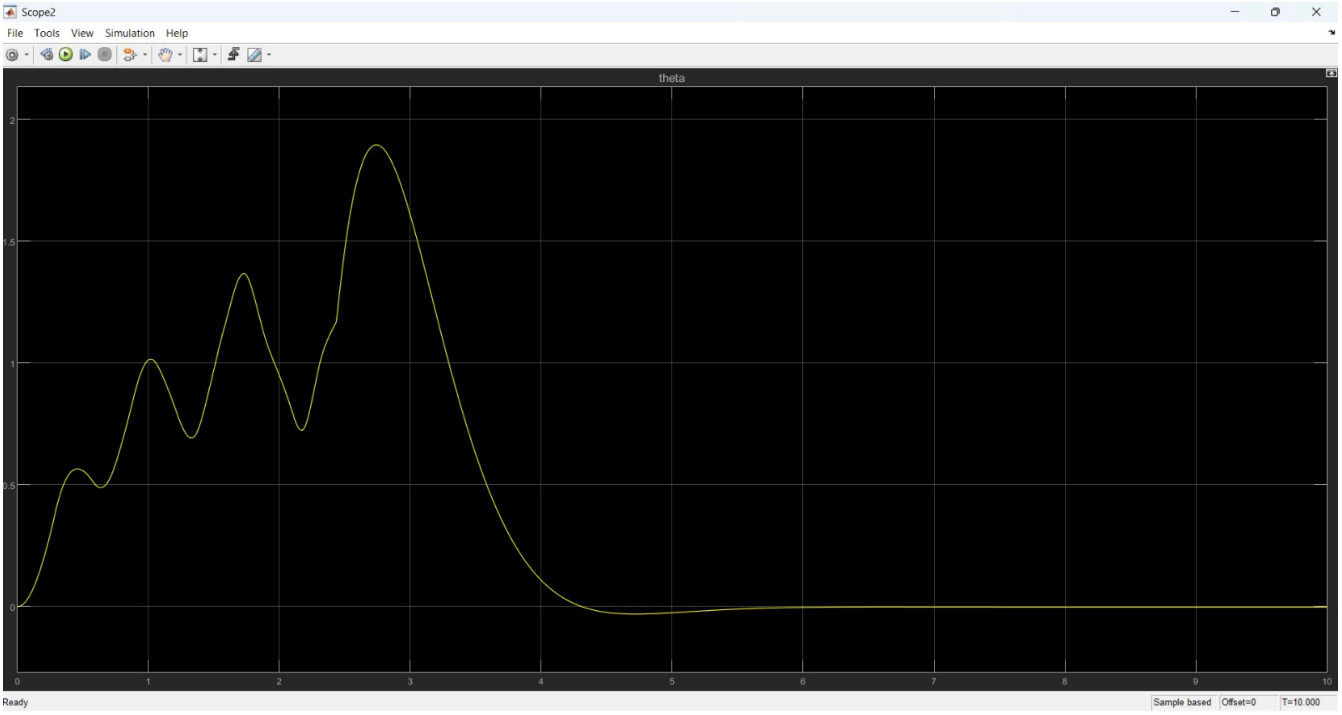

Figure 4-17: Pendulum angular velocity($\dot{\alpha}$). The velocity profile highlights the cyclical momentum of the pendulum link during the swing-up maneuvers, effectively dampening to zero immediately following LQR activation.

4.9 Chapter Summary

This chapter detailed the complete mechatronic development of the Rotary Inverted Pendulum testbed. The process began with establishing the physical hardware constraints—specifically managing the direct-wired architecture—followed by the rigorous derivation of the non-linear Euler-Lagrange equations of motion. A two-stage system identification process successfully bridged theory and reality, yielding highly accurate motor and pendulum inertial parameters.

These parameters were heavily validated using an integrated SolidWorks-to-Simscape Multibody environment, proving the viability of the hybrid Swing-Up and LQR control strategy. Finally, the simulation architecture was successfully transitioned into a real-time Hardware-in-the-Loop deployment on an ESP32 microcontroller. By directly sampling high-speed encoders and dynamically calculating instantaneous mechanical energy, the physical testbed autonomously executes the complex maneuvers required to stabilize the underactuated system.

With both the Active Suspension (Chapter 3) and Rotary Inverted Pendulum (Chapter 4) fully operational at the local hardware level, the final engineering challenge is bridging these physical systems to the internet. The subsequent chapter will detail the overarching CoTune cloud-telemetry infrastructure required to transform these testbeds into a globally accessible remote laboratory.

CHAPTER 5 CLOUD INTEGRATION & REMOTE EXPERIMENTATION

5.1 Introduction

The experimental platforms developed in Chapters 2 and 3 demonstrated the feasibility of real-time control of the Active Suspension System and the Rotary Inverted Pendulum through Hardware-in-the-Loop (HIL) deployment on the ESP32 microcontroller. Both systems achieved their respective control objectives: the active suspension attenuated road-induced disturbances under LQR optimal feedback control, and the rotary inverted pendulum successfully executed a swing-up maneuver followed by stabilization at the upright equilibrium. However, these capabilities remained physically bounded — interaction with either experiment required direct laboratory access, and system observability was limited to locally attached instruments.

This chapter documents the second stage of the CoTune project: the transformation of those physically bounded experimental rigs into fully remotely accessible learning platforms, integrated into the **CoTune Cloud website**. The objective of this integration is to enable any enrolled student, from any geographic location, to log into the platform, join a managed experiment queue, configure LQR control parameters from first principles using the embedded in-browser LQR solver, issue commands to the physical hardware over the internet, observe the hardware response in real time through **live telemetry graphs** and a **live video stream**, and review session data afterward — all without requiring physical presence in the laboratory.

To achieve this, a comprehensive cloud infrastructure was designed and deployed. The architecture is centered on **AWS IoT Core** as the managed **MQTT** message broker, **Amazon DynamoDB** for session and parameter persistence, **AWS Amplify** for frontend hosting and continuous integration, and a modular **React-based single-page application** as the student-facing interface. The physical link between the cloud and the hardware is provided by the ESP32 microcontroller, which connects to the AWS IoT Core endpoint over Wi-Fi using the MQTT protocol, receiving control commands from the web application and publishing sensor telemetry data back to all subscribed clients in real time.

5.2 Overall Cloud System Architecture

The CoTune remote experimentation platform follows a three-tier client-server-device architecture. The three tiers are: (1) the **physical hardware tier**, consisting of the ESP32 microcontroller interfaced with the experiment; (2) the **cloud services tier**, provided entirely by Amazon Web Services (AWS); and (3) the **client tier**, consisting of the React-based web application accessed by students through a standard web browser. The data flow across all three tiers is mediated exclusively by the MQTT publish-subscribe protocol, brokered by AWS IoT Core.

5.2.1 Hardware Tier

At the hardware tier, the ESP32 microcontroller serves a dual role: it is the real-time controller executing the LQR algorithm on sensor data at the loop rates established in Chapters 2 and 3, and it is simultaneously the internet gateway for the physical experiment. The ESP32 connects to the local Wi-Fi network and establishes a persistent authenticated MQTT session with the AWS IoT Core broker. It subscribes to command topics to receive control parameters issued by the web application, and it publishes sensor state data at each control cycle to feedback topics consumed by the browser in real time. The implementation of Wi-Fi connectivity and MQTT communication within the Simulink S-Function framework is described in Section 5.6.

5.2.2 Cloud Tier

The cloud tier is built exclusively on AWS managed services, all deployed in the **eu-west-3 (Paris)** region:

- **AWS IoT Core** acts as the managed MQTT broker. It handles all publish/subscribe message routing between the ESP32 and connected browser clients, enforcing identity-based access through AWS IoT policies attached to authenticated Amazon Cognito identities.
- **Amazon Cognito** provides user identity management and authentication. Students register and log in through a Cognito User Pool configured with email-based verification. Upon authentication, they obtain short-lived AWS credentials from a

Cognito Identity Pool. These credentials carry the **IoTPolicy** that authorizes the browser to publish to command topics and subscribe to telemetry topics.

- **Amazon DynamoDB** serves as the NoSQL persistence layer. Three tables underpin the platform: **QueueTable** manages per-experiment access queues and session state; **SuspensionLQRParameters** stores the Q and R matrices, computed LQR gain vectors, and road disturbance parameters submitted by each student during active suspension sessions; and **RotaryLQRParameters** stores the equivalent data for rotary inverted pendulum sessions.

- **AWS Amplify** handles frontend application hosting, continuous integration with the GitHub repository (automatic deployment on push), and provides the JavaScript SDK used by the React application to interact with all AWS services in the browser.

5.2.3 Client Tier

The client tier is a React single-page application (SPA). Students access the platform through any modern web browser with no software installation required. The application uses the AWS Amplify SDK and the AWS SDK for JavaScript v3 directly in the browser to: establish WebSocket-based MQTT connections (wss://) to the IoT Core broker; read and write DynamoDB records for queue management and parameter logging; and authenticate with Cognito. The browser therefore operates as a first-class MQTT client — subscribing to live hardware telemetry topics and publishing control commands that travel through the cloud broker to the physical ESP32 in real time.

5.3 Web Application Architecture and Modularization

5.3.1 Background and Motivation for Refactoring

The CoTune web application was initially developed as a **flat file** React project, in which all JavaScript components, CSS stylesheets, page logic, and utility modules were placed together in a single source directory. As the platform grew to support multiple experiments, authentication flows, URDF-based 3D viewers, AI assistant features, and live telemetry pages, this flat layout exceeded 90 source files in a single folder. The absence of structural organization made it increasingly difficult to isolate functionality, manage shared

dependencies, or onboard new contributors. A full architectural refactoring was carried out to reorganize the codebase into a modular, directory-based structure before implementing the cloud integration features.

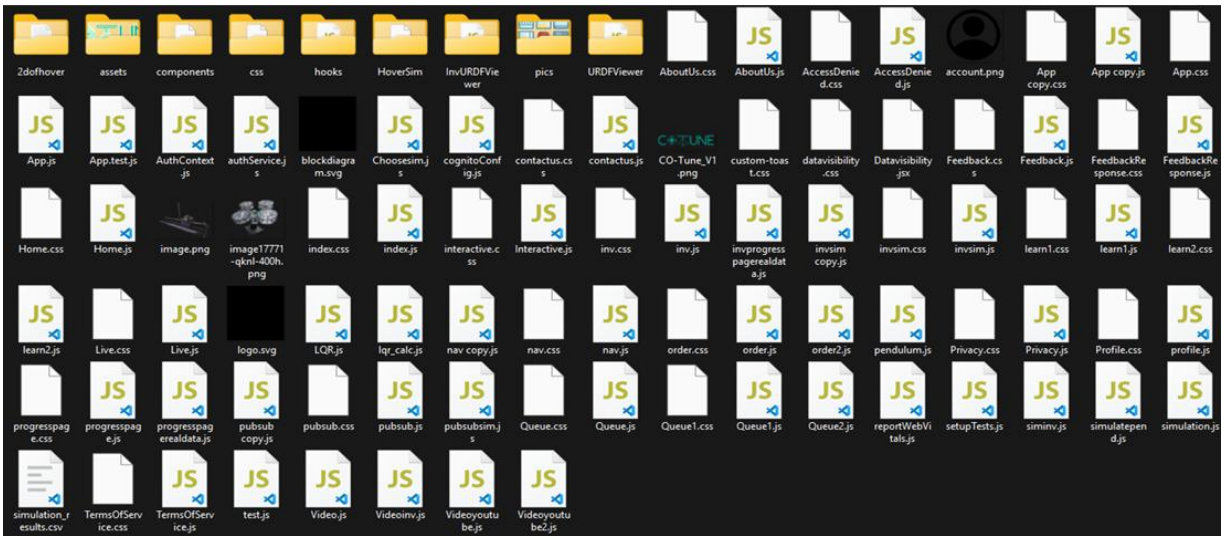

Figure 5-1: System Files Before Architecture Refactoring

5.3.2 Directory Structure

Following the refactoring, the **src/** directory was organized into seven purpose-driven subdirectories, as shown in Table 5.1. This structure enforces clear separation of concerns, ensures that each directory has a well-defined responsibility, and enables the addition of new experiments or features without disturbing existing modules.

Directory	Purpose	Representative Files
assets/	Static media: images, icons, animations, SVGs	CO-Tune_V2.png, logo.svg, inverted.gif
auth/	AWS Cognito authentication logic and context	AuthContext.js, authService.js, cognitoConfig.js
components/	Reusable UI components shared across pages	ChatSidebar.js, Modal.js, loading.jsx, FlipCard.jsx
controllers/	Control algorithm implementations	LQR.js, lqr_calc.js

experiments/	URDF model viewers and physics simulation modules	SuspensionURDFViewer.jsx, FurutaURDFViewer.jsx, suspensionPhysics.js
layout/	Global layout shell: navigation bar and footer	nav.js, nav.css, footer.jsx, footer.css
pages/	All routed page components	SuspensionLive.js, RotaryLive.js, Queue1.js, Home.js

Table 5-1: CoTune Source Directory Structure

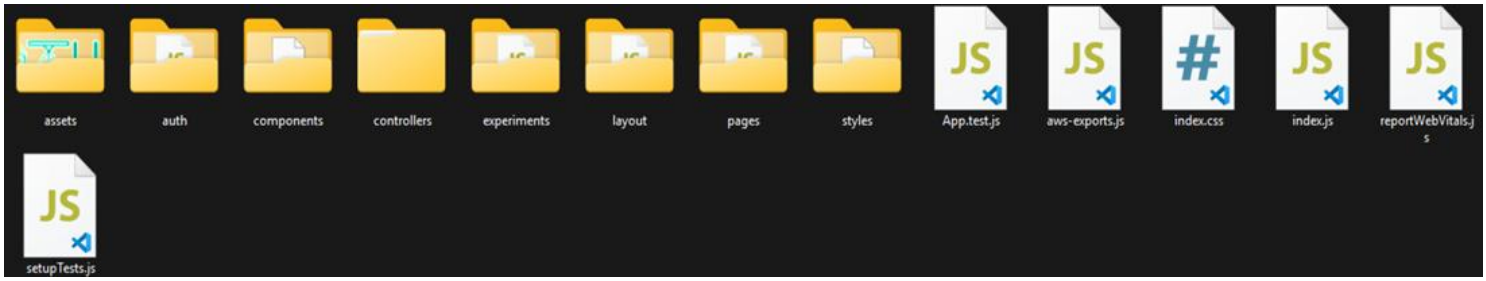

Figure 5-2: System Files After Architecture Refactoring

5.3.3 Key Architectural Decisions

Centralized Authentication Context. The auth/ directory implements a React Context (AuthContext.js) that wraps the entire application, making the authenticated user's session, Cognito credentials, and identity ID available to every component through the useAuthenticator hook provided by the AWS Amplify UI React library. This eliminates the need for every page component to independently manage authentication state.

Decoupled LQR Controller. The LQR computation logic matrix algebra, the algebraic Riccati equation solver, and gain calculation is isolated in controllers/LQR/lqr_calc.js. This module is imported by both the Active Suspension live page and the Rotary Inverted Pendulum live page, ensuring that the numerical LQR implementation is a single shared source of truth, independent of the UI layer.

URDF Viewers as Self-Contained Modules. The experiments/ directory packages each 3D URDF viewer and its associated physics simulator as a self-contained module. The viewers use three.js via the @react-three/fiber and @react-three/drei bindings, and load robot description files using the urdf-loader library. Encapsulating each viewer in its own subdirectory allows the simulation subsystem to be developed and updated independently of the live experiment pages.

Modular Page Routing. All routed page views reside in pages/, and routing is defined centrally. The application uses react-router-dom v7 for client-side navigation, which is consistent with the SPA model and allows the Amplify-hosted application to serve all routes from a single index.html entry point.

5.3.4 Key Dependencies

The platform relies on the following principal external libraries, listed with their major versions:

- **React 18** — UI rendering and state management
- **react-router-dom v7** — Client-side routing

- **AWS Amplify v6** — Authentication, hosting integration, and IoT PubSub client
- **@aws-sdk v3** — Browser-compatible SDK for DynamoDB, IoT Core, and Cognito
- **Chart.js v4 + react-chartjs-2 v5** — Real-time telemetry graph rendering
- **Three.js + @react-three/fiber + @react-three/drei** — WebGL-based 3D URDF visualization
- **urdf-loader v0.12** — Parsing and rendering URDF robot description files in the browser
- **numeric v1.2** — Numerical linear algebra for the in-browser LQR solver
- **groq-sdk v1.1** — API client for the CoTune AI Assistant (Section 5.13)
- **framer-motion v12** — Declarative UI animations

5.4 Cloud Deployment and Database Integration

5.4.1 AWS Amplify Hosting and CI/CD Pipeline

The CoTune web application is deployed and hosted through **AWS Amplify**. Amplify is connected to the project's GitHub repository and monitors the **main** branch. Every push to that branch automatically triggers a full build-and-deploy pipeline without manual intervention, ensuring that all cloud updates are immediately reflected in the live platform.

The build process is defined in **amplify.yml** at the project root. The build phase invokes the Create React App build tool, which compiles the React application into optimized static assets and outputs them to the **build/** directory. These static files are then served globally by the Amplify CDN. The build is configured with **GENERATE_SOURCEMAP=false** to reduce bundle size for production, and **NODE_OPTIONS=--max_old_space_size=7168** allocates 7 GB of heap memory to the Node.js build process to accommodate the large dependency tree introduced by the Three.js URDF viewers and the AWS SDK v3.

Once deployed, the application is accessible through a public URL assigned by Amplify, and users interact with the platform entirely through a web browser with no software installation required.

5.4.2 Amazon Cognito: Authentication and Identity

User identity and access control are managed through a two-layer Amazon Cognito configuration:

User Pool: Handles user registration, login, and session management. Accounts are created with email address as the primary identifier and require email verification. The User Pool enforces a minimum password length of eight characters.

Identity Pool: Upon successful authentication through the User Pool, the Identity Pool issues short-lived AWS credentials scoped to the authenticated user's unique **identityId**. These temporary credentials grant the browser client the specific IAM and IoT permissions it needs to interact with DynamoDB and AWS IoT Core, without exposing any long-lived secrets in the frontend code.

When a student opens a live experiment page, the browser first calls **fetchAuthSession()** to retrieve these credentials, then attaches the **IoTPolicy** to the authenticated Cognito identity using the **AttachPolicyCommand**. This policy grants the identity permission to publish to command topics and subscribe to telemetry topics for the duration of the session.

5.4.3 Amazon DynamoDB: Data Persistance

Amazon DynamoDB serves as the serverless NoSQL database for the platform. The following tables are used, all hosted in the eu-west-3 region:

- **ProfileTable:** Stores each user's display name and profile information, keyed on IdentityID. The main page fetches the student's name from this table upon login to display a personalized greeting.

- **QueueTable:** The core table for the experiment access queue management system (described in detail in Section 5.9). Each entry records the user's userId, timestamp (used as sort key for ordering), entryTime, and sessionStartTime. The queue logic is implemented entirely in the browser through direct DynamoDB operations without a backend server.

- **SuspensionLQRParameters**: Records each Active Suspension session's submitted parameters. Each item stores the student's identityId, session timestamp, the Q matrix diagonal values (Q00, Q11, Q22, Q33), the R penalty value, road disturbance amplitude and frequency, and the computed LQR gain vector (K0 through K3).

- **RotaryLQRParameters**: The equivalent table for Rotary Inverted Pendulum sessions, storing the same Q and R values alongside the four computed LQR gains.

This data persistence layer allows students to track their parameter tuning history through the platform's Progress pages and provides the instructors with a persistent audit trail of all experimental sessions.

5.5 MQTT Based Communication

5.5.1 Protocol Overview

The Message Queuing Telemetry Transport (MQTT) protocol was selected as the communication backbone of the CoTune remote experimentation platform. MQTT is a lightweight publish-subscribe messaging protocol designed for constrained devices and unreliable networks. Its topic-based routing model is a natural fit for the asymmetric data flows in this system: the web application publishes low-frequency command messages to the hardware, while the ESP32 publishes high-frequency telemetry data back to all subscribed browser clients simultaneously.

AWS IoT Core acts as the managed MQTT broker, eliminating the need to operate a dedicated message broker server. All message routing, connection management, and access policy enforcement is handled by the AWS service.

5.5.2 Connection Security and Authentication

Two client types connect to the MQTT broker, each using a different authentication mechanism appropriate to its environment:

ESP32 Hardware Client — Mutual TLS (mTLS) over TCP: The ESP32 connects to the AWS IoT Core broker on port **8883** using TLS 1.2. Authentication is performed through mutual X.509 certificate exchange. The device presents three credential artifacts, stored locally in a **secrets.h** header file:

- AWS_CERT_CA — the Amazon Root CA certificate
- AWS_CERT_CRT — the unique device certificate issued by AWS IoT for this ESP32
- AWS_CERT_PRIVATE — the corresponding private key

The **WiFiClientSecure** library configures the TLS context with these credentials before the **PubSubClient** establishes the MQTT session.

Browser Client — WebSocket over TLS with Cognito Credentials: The React application connects to the MQTT broker through a WebSocket upgrade over TLS (WSS) on port 443, using the AWS Amplify **PubSub** library. Authentication uses the short-lived AWS credentials issued to the user's Cognito identity, as described in Section 5.4.2

5.5.3 MQTT Topic Architecture

The platform defines six MQTT topics that are organized by direction and function. Table 5.2 lists all topics.

Topic	Direction	Experiment	Purpose
SUSP/Parameters	Web → ESP32	Active Suspension	LQR gains, disturbance parameters, START/STOP command
SUSP/Stream	Web → ESP32	Active Suspension	Video stream start/stop notification with session identity
suspension/telemetry	ESP32 → Web	Active Suspension	Real-time sensor state feedback
ROTARY/Parameters	Web → ESP32	Rotary Pendulum	LQR gains and START/STOP command
ROTARY/Stream	Web → ESP32	Rotary Pendulum	Video stream start/stop notification with session identity
rotary/telemetry	ESP32 → Web	Rotary Pendulum	Real-time sensor state feedback

Table 5-2: MQTT Topic Definitions

5.5.4 Command Message Formats (Web to ESP32)

All command messages are serialized as JSON objects. The ESP32 parses incoming payloads using the **ArduinoJson** library.

- **Active Suspension — START Command** (published to SUSP/Parameters):

```
{
 "command": "START",
```

"K": [k0, k1, k2, k3],

"amp": roadAmplitude,

"freq": roadFrequency

}

The four K values are the LQR gain vector computed by the in-browser solver from the student's Q and R matrices. amp and freq define the sinusoidal road disturbance profile (amplitude in millimetres and angular frequency in rad/s).

- **Rotary Pendulum — START Command** (published to ROTARY/Parameters):

{

"command": "START",

"K": [k0, k1, k2, k3]

}

The four K values are the LQR gain vector for the pendulum state vector [θ, α, θ̇, α̇]. No disturbance parameters are sent for the rotary experiment.

- **STOP Command** (both experiments, same format):

{

"command": "STOP"

}

This message is published alongside the START/STOP command to notify the streaming infrastructure of the active session identity and timestamp, enabling stream start/stop correlation with the queue system.

5.5.5 Telemetry Message Formats (ESP32 to Web)

Active Suspension Telemetry (published to suspension/telemetry at **1 Hz**):

{

"d1": <suspension travel, m>,

"d2": <tire deflection, m>,

"vs": <sprung mass velocity, m/s>,

"vus": <unsprung mass velocity, m/s>,

"u": <control force, N>

}

The five fields correspond directly to the four state variables of the quarter-car model and the LQR control output, as defined in Chapter 2.

Rotary Pendulum Telemetry (published to rotary/telemetry at **10 Hz**):

{

"a1": <arm angle, rad>,

"a2": <pendulum angle, rad>,

"v1": <arm angular velocity, rad/s>,

"v2": <pendulum angular velocity, rad/s>,

"u": <control torque, N>

}

The five fields correspond to the full state vector of the Furuta pendulum and the control torque. The higher telemetry rate for the rotary experiment (10 Hz vs. 1 Hz for suspension) reflects the faster dynamics of the pendulum balancing task, where transient behavior during swing-up and stabilization must be captured at sufficient resolution to be meaningful for the student.

5.5.6 QoS and Message Reliability

All MQTT messages in this platform use **QoS 0** (at-most-once delivery). This choice was made deliberately: telemetry messages are time-stamped state snapshots that lose their utility if delivered late, and the high repetition rate means that occasional dropped packets have no significant effect on the displayed graphs. For command messages, the 5-second button debounce enforced in the browser (Section 5.11) prevents accidental re-sends. The total

message payload for any single topic message remains below 256 bytes in all cases, well within the **PubSubClient** and IoT Core per-message limits.

5.6 Simulink ESP32 WiFi and MQTT Integration

5.6.1 Overview

Chapters 2 and 3 documented the Hardware-in-the-Loop Simulink models for the Active Suspension and the Rotary Inverted Pendulum, deployed to the ESP32 through the Simulink Support Package for Arduino-compatible hardware. In their original form, these models operated in a fully local, self-contained manner: control parameters were compiled into the model at build time, and there was no runtime mechanism for a remote user to modify gains, issue commands, or receive state data over a network.

To enable remote operation, a new S-Function block (**Cloud_MQTT)** was designed, implemented in C++, and integrated into each HIL Simulink model. This block runs a dedicated FreeRTOS task on the ESP32 that manages Wi-Fi connectivity, TLS-authenticated MQTT communication with AWS IoT Core, and the runtime exchange of control parameters and telemetry between the cloud and the Simulink model. The integration was designed as a non-intrusive addition: the existing sensor, Kalman filter, LQR, and actuator drive subsystems documented in the previous chapters remain structurally unchanged. The Cloud_MQTT block is wired in parallel with the existing signal paths, extending the model with cloud capability without modifying its control logic.

5.6.2 Cloud_MQTT S-Function Architecture

The Cloud_MQTT S-Function is implemented using the Simulink S-Function Builder and follows the standard three-wrapper structure: **Start_wrapper**, **Outputs_wrapper**, and **Terminate_wrapper**.

- **Start_wrapper — FreeRTOS Task Initialization:** The Start_wrapper executes once at model initialization. Its sole responsibility is to spawn the CloudTask FreeRTOS task using xTaskCreatePinnedToCore(), pinning it to **Core 1** of the ESP32 with a stack size of **8192 bytes** and priority **1**. The Simulink control loop itself runs on Core 0. This core separation ensures that all network I/O operations — which are inherently blocking

and variable in latency — are completely isolated from the real-time control execution on Core 0, preserving the deterministic loop timing of the Kalman filter and LQR controller.

- **Outputs_wrapper — Simulink Step Interface:** The Outputs_wrapper executes at every Simulink sample step. It performs a single atomic exchange of data through the shared memory region protected by a FreeRTOS critical section (portENTER_CRITICAL / portEXIT_CRITICAL):

  - **Inputs from Simulink** (written into the shared region): the current sensor state vector (d1, d2, vs, vus, u for suspension; a1, a2, v1, v2, u for rotary). These are made available to the CloudTask for telemetry publishing.

  - **Outputs to Simulink** (read from the shared region): the received LQR gain vector K[4], and the state flags enable_lqr / enable_control and exp_running. These signals flow directly into the control subsystem to activate or deactivate the LQR controller and road excitation at runtime.

  The use of portMUX_TYPE critical sections, rather than FreeRTOS mutexes, was deliberate: the Outputs_wrapper is called from the Simulink ISR context on Core 0, and FreeRTOS mutexes cannot be acquired from interrupt context. Critical sections disable interrupts on both cores for the duration of the data exchange, guaranteeing atomicity without deadlock risk.

- **Terminate_wrapper:** The terminate wrapper is intentionally left empty. The FreeRTOS task is designed to run for the full duration of the Simulink model execution and does not require explicit cleanup on termination.

5.6.3 FreeRTOS Cloud Task Implementation

The CloudTask function runs continuously in an infinite loop on Core 1. Its lifecycle proceeds through three phases:

Phase 1: Network Initialization (one-time): Upon startup, the task initializes the TLS context by loading the AWS X.509 credentials from secrets.h, configures the PubSubClient

with the AWS IoT Core endpoint on port 8883, and begins the Wi-Fi connection sequence using the credentials in SECRET_SSID and SECRET_PASS. The task attempts up to 20 connection retries with 1-second intervals. If the Wi-Fi connection fails after all retries, the ESP32 calls ESP.restart() to perform a hardware reset and retry from the beginning.

Phase 2: MQTT Connection and Subscription: Once Wi-Fi is established, the task enters its main loop. At each iteration, it checks whether the MQTT client is still connected to the broker. If disconnected, it attempts reconnection and re-subscribes to the command topic (SUSP/Parameters for the suspension experiment; ROTARY/Parameters for the rotary experiment). The client.loop() call processes incoming messages and maintains the broker heartbeat.

Phase 3: Callback Execution and Telemetry Publishing: Incoming MQTT messages are processed by the mqttCallback function, which executes on the CloudTask stack when client.loop() dispatches a received message. The callback parses the JSON payload using StaticJsonDocument<256>, extracts the command field, and updates the shared state variables inside a critical section:

- On **START**: the K gain vector and disturbance parameters are stored in g_cloud_K[], g_cloud_amp, and g_cloud_freq. The experiment is marked as running (g_exp_running = true), but the LQR and telemetry are not yet activated (g_lqr_active = false). The current time is recorded via millis() to begin the warmup countdown.

- On **STOP**: all flags are cleared immediately, disabling LQR output and telemetry in the next Simulink step.

Telemetry publication is handled within the main loop. At the configured publishing interval, the task reads the current state variables from the shared region, serializes them into a JSON string using serializeJson(), and publishes the result to the telemetry topic via client.publish().

5.6.4 Simulink Model Integration

Figure 5-3 shows the top-level HIL Simulink model for the Active Suspension after the Cloud_MQTT block was added. The figure illustrates the signal routing between the existing subsystems and the new Cloud_MQTT block.

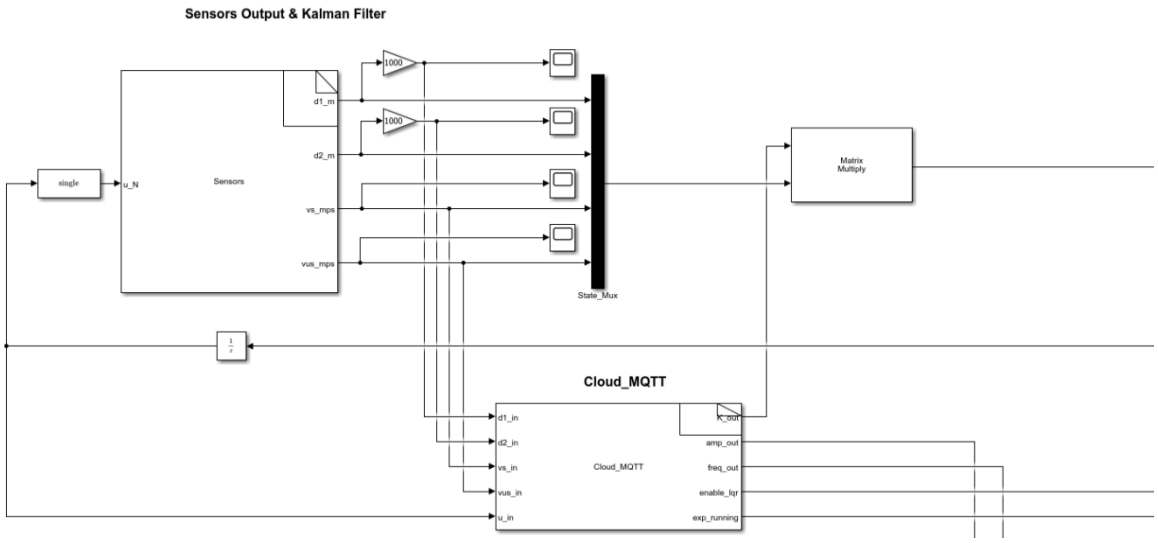

Figure 5-3: Top-level HIL model with Cloud_MQTT block

The Cloud_MQTT block receives **five input** signals from the existing model: the two **displacement measurements** d1_in and d2_in (in metres), the two **velocity signals** vs_in and vus_in, and the **current control output** u_in. These are the same signals already present in the model for the Simulink scope displays and the Kalman filter. No new sensor wiring or signal conditioning is required — the Cloud_MQTT block taps into existing signal lines.

The block produces **five output** signals. The **K-matrix output** (K_out, a 4×1 vector) is fed into a Matrix Multiply block that computes the LQR control law $u = -K \cdot x$ using the state vector assembled by the State_Mux. This replaces the previously hard-coded gain values: the K values are now set at runtime by the student through the web interface. The **enable_lqr boolean** output gates the LQR force through a Switch block when enable_lqr is false (during warmup or when no experiment is running), the Switch outputs zero, preventing any unintended actuation. The **amp_out** and **freq_out** outputs parameterize the sinusoidal road excitation profile, and exp_running enables the road motor drive subsystem.

Figure 5-4 shows the LQR control subsystem and the downstream signal processing chain.

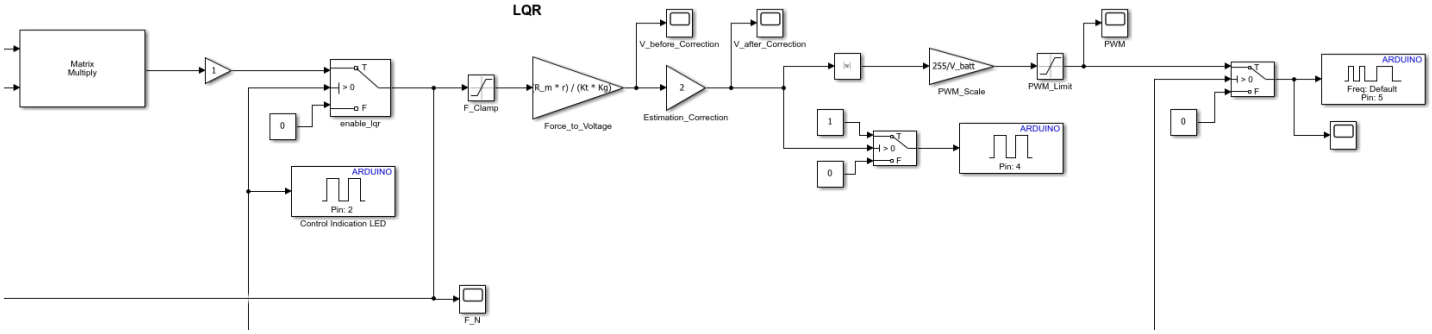

Figure 5-4: LQR with Cloud_MQTT output routing, PWM chain

The computed LQR force passes through a force clamp (F_Clamp) to enforce actuator saturation limits, then through a Force-to-Voltage conversion block that applies the motor constant relationship V = (R_m × r) / (Kt × Kg) to map the desired Newton force to a motor voltage. An estimation correction factor of ×2 accounts for the capstan mechanical advantage asymmetry identified during commissioning. The voltage is then normalized to an 8-bit PWM range through PWM_Scale = 255 / V_batt and limited through PWM_Limit. The PWM value is output to the Cytron MDD10A motor driver on Arduino **Pin 5**, with the direction signal on **Pin 4**. The road excitation signal is synthesized by multiplying a clock signal by freq_out, passing it through a sine function, and multiplying the result by amp_out to produce the commanded road position, which drives the road motor subsystem.

5.6.5 Startup Sequence and Warmup Phase

A deliberate 10-second warmup phase was implemented in both S-Functions following receipt of a START command. During this window, the enable_lqr / enable_control output to Simulink remains false, holding the LQR force to zero. For the suspension experiment, this period allows the road excitation motor to begin its sinusoidal motion and reach steady-state amplitude before the controller engages, avoiding a transient force spike on a stationary plant. For the rotary experiment, this period allows the operator to manually position the pendulum near the downward equilibrium before the swing-up algorithm activates, and telemetry is also suppressed for this window to avoid publishing meaningless pre-swing data.

After 10 seconds, the respective g_lqr_active (suspension) or g_control_active (rotary) flag is set to true within the CloudTask, which causes the Outputs_wrapper to assert the enable_lqr / enable_control output at the next Simulink sample step, activating the controller. Telemetry publication also begins at this point for both experiments.

5.6.6 Comparison of Active Suspension System and Rotary Inverted Pendulum Implementations

Table 5.3 summarizes the key implementation differences between the two Cloud_MQTT S-Functions.

Parameter	Active Suspension	Rotary Pendulum
Command topic	SUSP/Parameters	ROTARY/Parameters
Telemetry topic	suspension/telemetry	rotary/telemetry
S-Function inputs	d1_in, d2_in, vs_in, vus_in, u_in	a1_in, a2_in, v1_in, v2_in, u_in
S-Function outputs	K_out[4], amp_out, freq_out, enable_lqr, exp_running	K_out[4], enable_control
Telemetry rate	1 Hz	10 Hz
CloudTask loop period	100 ms	10 ms
Warmup delay	10 s (road motor stabilization)	10 s (swing-up preparation)
Disturbance parameters received	Yes (amp, freq)	No
MQTT client ID	ESP32_Suspension	rotary-policy

Table 5-3: Cloud_MQTT S-Function Comparison

The key architectural difference is the telemetry rate: the rotary pendulum publishes at 10 Hz to capture the fast angular dynamics during swing-up and stabilization, while the suspension experiment publishes at 1 Hz, sufficient for the slower vertical dynamics of the quarter-car model. Accordingly, the CloudTask for the rotary experiment runs a tighter main loop at 10 ms, versus 100 ms for the suspension.

5.7 Real Time Sensor Pipeline and Graph Rendering

5.7.1 How Data Gets from Hardware to Screen

The path from physical sensor reading to visible chart update in the student's browser follows a four-stage pipeline:

1. **Acquisition**: The ESP32 reads sensor values (ToF displacements, IMU accelerations, encoder positions) through the FreeRTOS tasks documented in Chapter 2 and Chapter 3, and computes the LQR control output at each Simulink step.

2. **Publication**: The CloudTask reads the current state variables from the shared memory region (via the Outputs_wrapper critical section) and serializes them into a JSON payload, which is published to the appropriate MQTT telemetry topic over TLS to AWS IoT Core.

3. **Subscription and State Update**: The browser, connected to the same broker via WebSocket, receives the incoming message through the AWS Amplify PubSub subscription. The next callback of the subscription updates the corresponding React state arrays by appending the new value.

4. **Rendering**: The React state update triggers a re-render of the Chart.js line charts via the react-chartjs-2 binding, which redraws the updated data without a page reload.

5.7.2 Chart Configuration

All charts on both live experiment pages are configured with **animation: false** to eliminate the per-frame animation overhead that is incompatible with continuous real-time data ingestion. Charts use **CategoryScale** for the x-axis with a virtual time label, and **LinearScale** for the y-

axis. A shared CSS variable (var(--text-primary)) is applied to all axis labels and tick colors, enabling seamless light/dark mode switching without chart reconfiguration.

The x-axis timestamps are not drawn from the system clock but are computed as a monotonically increasing counter multiplied by the expected sample interval (0.1 s per step for the rotary experiment, producing a virtual time axis that is accurate when the hardware publishes at the nominal **10 Hz rate**).

5.7.3 Data Decimation

Over a full five-minute session, the rotary experiment accumulates up to 3,000 data points per signal at 10 Hz, and the suspension experiment accumulates up to 300 points per signal at 1 Hz. To prevent browser performance degradation as the session progresses, a decimation step is computed before each chart render:

const step = Math.max(1, Math.floor(timestamps.length / 500));

This limits the maximum number of points rendered per chart to 500, regardless of session length. Both the data arrays and the timestamp array are filtered by the same step index, preserving the shape of the signal while reducing the DOM load on the WebGL chart canvas.

5.8 Live Video Streaming

5.8.1 Streaming Infrastructure

Live visual observation of the physical experiment is provided through a Twitch live stream integrated directly into both live experiment pages. A dedicated Twitch channel, **cotune26**, was created for the CoTune laboratory. A camera pointed at the physical experiment rig transmits the video feed to this channel, and the stream is embedded in the experiment page so the student can observe the hardware's physical motion alongside the telemetry graphs in real time.

5.8.2 Implementation

The video stream is encapsulated in a reusable LiveVideoPlayer React component located in pages/Video.js. The component renders an HTML <iframe> using the Twitch embedded player API: https://player.twitch.tv/?channel=cotune26&parent={window.location.hostname}

The parent parameter is resolved dynamically at runtime using window.location.hostname, making the embed compatible with both the local development server and the deployed Amplify domain without any configuration change. The allowFullScreen attribute is enabled, allowing students to expand the stream to full screen during their session.

The component is fully responsive. The iframe height adapts across four breakpoints: 600 px on desktop, 428 px on tablets (≤ 1024 px), 304 px on mobile (≤ 768 px), and 165 px on small screens (≤ 480 px). The same LiveVideoPlayer component instance is imported and rendered identically on both SuspensionLive.js and RotaryLive.js, ensuring a consistent streaming experience across both experiments.

When the CoTune stream is offline, the Twitch embed displays Twitch's standard offline message with a link to the channel, notifying the student that the hardware session is not currently available.

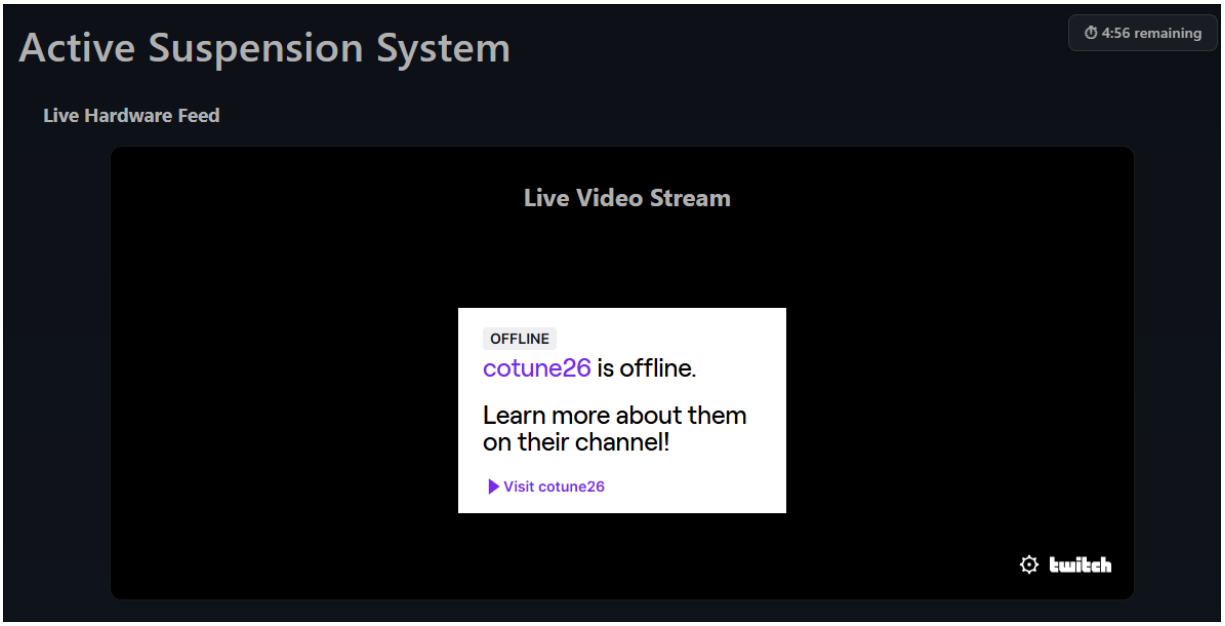

Figure 5-5: Screenshot of Live Video Stream component embedded in the live experiment page

5.9 Queue Management System

5.9.1 Design Rationale

Since both physical experiment rigs are single-instance shared resources, simultaneous access by multiple students would result in conflicting commands being sent to the same ESP32. A Queue Management System was implemented to serialize access, granting each student an exclusive, time-bounded session while allowing all other users to monitor their position in the waiting list in real time.

5.9.2 Independent Per Experiment Queues

Each experiment has a fully independent queue backed by its own DynamoDB table:

- **Active Suspension**: managed by the useQueueStatus hook in Queue4.js, backed by QueueTable
- **Rotary Inverted Pendulum**: managed by the useQueueStatus hook in Queue3.js, backed by RotaryQueueTable

This separation means that a student using the Rotary Pendulum does not affect the queue state of the Active Suspension, and both experiments can be simultaneously occupied by different students without interference.

5.9.3 Queue State Model

Each entry in the queue table contains the following fields:

Field	Type	Description
userId	String	Cognito identity ID (primary key)
timestamp	Number	Entry timestamp in Unix ms (sort key for ordering)
entryTime	Number	Same as timestamp — when the user joined
sessionStartTime	Number	Set when session begins; 0 if waiting

Table 5-4: Queue State Model

The queue is ordered by timestamp ascending — the student who joined earliest is always at the front.

5.9.4 Session Lifecyle

Joining: When a student navigates to a live experiment page and their Cognito credentials are ready, joinQueue() is called. It first scans the table to check if the user is already present (preventing duplicate entries on page refresh), then inserts a new item with the current Unix timestamp.

Polling: The client polls the queue table every **1 second** using DynamoDB ScanCommand. The scan result is sorted by timestamp, giving the current queue order. The student's position is their index + 1 in this sorted list.

Session Start: When the first user in the sorted list has sessionStartTime = 0, the system calls startSession(), which writes the current timestamp into that user's sessionStartTime field. This marks the official start of their five-minute window.

Session Timer: The active student's remaining time is computed as SESSION_DURATION − (now − sessionStartTime). This countdown is displayed as a fixed timer widget in the top-right corner of the experiment page throughout the session.

Auto-Expiry and Handoff: When the remaining time reaches zero, two things happen simultaneously: the browser calls sendCommand("STOP") to halt the ESP32 (preventing unattended actuation), and endSession() deletes the user's entry from the queue table and navigates to the Progress page. Waiting students, whose 1-second polling detects the first user's removal, automatically advance in position. The new first student's sessionStartTime field is then set to zero, triggering startSession() on their next poll cycle and granting them hardware access.

Waiting State: Students with position > 1 are shown a loading screen displaying their live queue position. The experiment controls are not rendered until position === 1 and isAllowed === true.

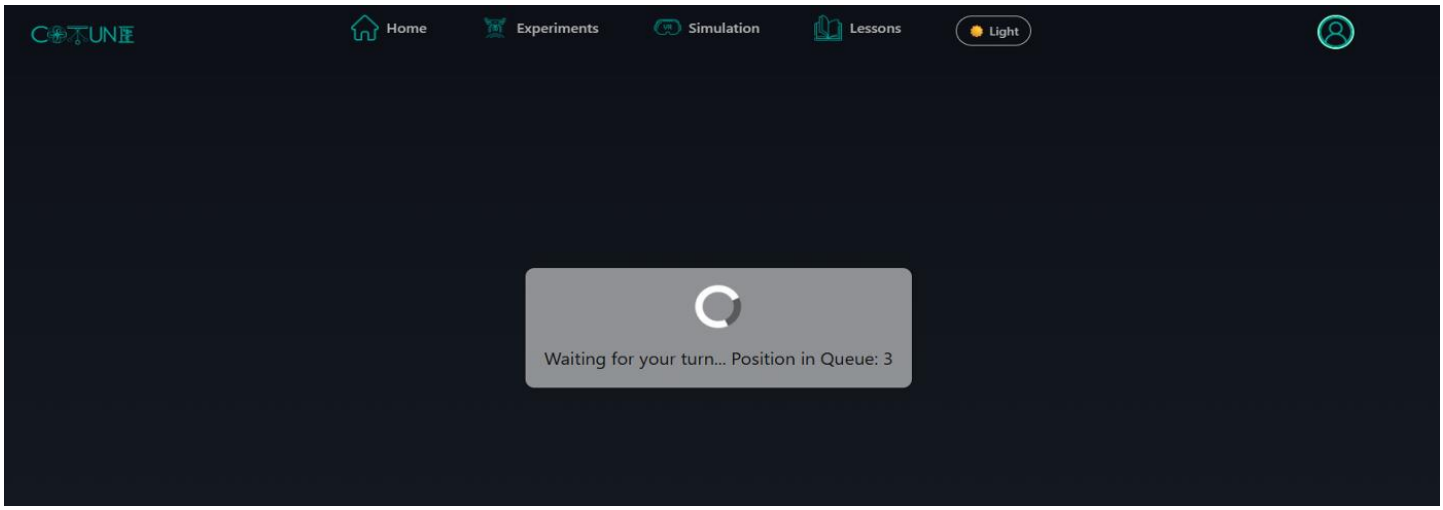

Figure 5-6: Queue Management Waiting Screen

5.10 In-Browser 3D Simulation Modules

5.10.1 URDF Generation

A prerequisite for in-browser 3D visualization is a machine-readable description of the physical system's geometry and joint structure. URDF (Unified Robot Description Format) files were generated for both experimental rigs directly from their SolidWorks CAD assemblies. The URDF format encodes each rigid body as a named <link> element containing geometry meshes, mass properties, and inertia tensors, and each degree of freedom as a <joint> element specifying the axis, limits, and parent-child link relationship. The resulting URDF files serve as the bridge between the physical CAD models and the browser-based simulation renderer.

5.10.2 Rendering Pipeline

The in-browser 3D viewer is built on **Three.js**, accessed through the @react-three/fiber React binding and supplemented by the @react-three/drei helper library. URDF parsing is handled by the **urdf-loader** library (v0.12), which reads the URDF XML, loads the associated STL mesh files, and assembles the Three.js scene graph with the correct link transforms and joint angles. The renderer runs in a <canvas> element managed by @react-three/fiber, and joint angles are updated at each simulation frame by driving the corresponding joint variables with the simulation output, producing a live 3D animation of the system's dynamic response.

5.10.3 Rotary Inverted Pendulum Simulation Module

The Rotary Inverted Pendulum simulation module (FurutaURDFViewer.jsx) implements a complete in-browser simulation of the Furuta pendulum dynamics. The physics engine (furutaPhysics.js) integrates the nonlinear equations of motion derived in Chapter 3 using the system parameters embedded directly in the code. The simulation operates in two phases:

- **Phase 1 (0–10 s):** The system evolves in open loop (no control action), corresponding to the passive response phase.

- **Phase 2 (10–20 s):** The LQR controller activates using the K gains computed from the student's Q and R inputs by the in-browser LQR solver (lqr_calc.js). The pendulum swings up and stabilizes.

our camera presents (Cam Position 1, Cam Position 2, Cam Position 3, and Free Cam) allow the student to inspect the pendulum motion from different spatial perspectives. The computed K gains are displayed numerically alongside the 3D view.

Figure 5-7: Rotary Inverted Pendulum Simulation

5.10.4 Active Suspension Simulation Module

The Active Suspension simulation module (SuspensionURDFViewer.jsx) provides an equivalent in-browser simulation for the quarter-car model. The physics engine (suspensionPhysics.js) integrates the four-state linear system (suspension travel, sprung mass velocity, tire deflection, unsprung mass velocity) derived in Chapter 2. The simulation runs for 20 seconds with the LQR controller activating at the 10-second mark. The road disturbance profile is a sinusoid with user-specified amplitude and frequency. Four time-domain plots — Suspension Travel, Ride Comfort (sprung acceleration), Road Handling (tire deflection), and Control Force — are rendered alongside the 3D suspension model.

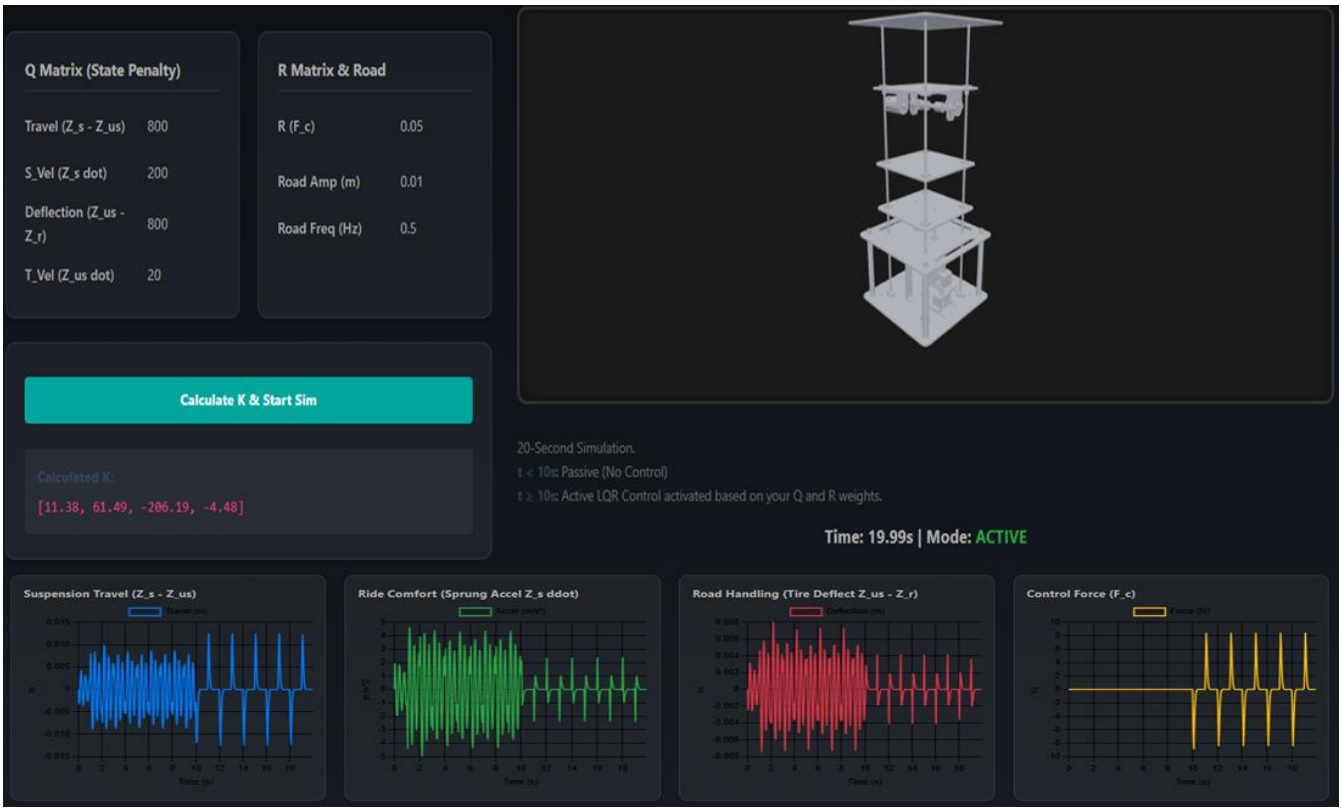

Figure 5-8: Active Suspension Simulation

5.11 CoTune AI Assistant

5.11.1 Overview

A context-aware AI assistant was integrated into the CoTune platform as a fixed sidebar component, accessible from every page through a persistent toggle button. The assistant is designed specifically to support students during their interaction with the platform, providing immediate guidance on control theory, parameter tuning, experiment behavior, and platform navigation without requiring them to leave the page or consult external resources.

5.11.2 Technical Implementation

The assistant is implemented in the ChatSidebar.js component using the **Groq SDK** (groq-sdk v1.1), which provides access to large language models through the Groq inference API. The model used is **llama-3.3-70b-versatile**, a 70-billion parameter instruction-tuned language model with a maximum output of **1024 tokens** per response. The API key is supplied through the environment variable REACT_APP_GROQ_API_KEY, which Amplify injects at build time from the project's environment configuration, keeping the key out of the repository.

5.11.3 Domain Scoping

The assistant operates under a fixed system prompt that constrains its scope to the CoTune domain:

"You are an AI assistant for CoTune, a control systems education and experimentation platform. You help students and researchers understand: Control theory (PID, LQR, MPC controllers and how to tune them), Physical systems (inverted pendulum, Furuta pendulum, suspension systems), Parameter tuning and optimization strategies, Real-time experiment data interpretation, Mathematical concepts related to control systems (state space, transfer functions, stability), and URDF integration for simulation. Be clear, educational, concise, and brief."

This scoping ensures that the assistant returns focused, technically accurate responses relevant to the experiment at hand, rather than general-purpose responses that may distract or mislead students during a live session.

5.11.4 Conversational Memory

Within a session, the assistant maintains a conversation history in memory (historyRef) that is passed as the full message array to each Groq API call. This allows students to ask follow-up questions naturally — for example, "what if I increase Q11?" after a prior exchange about LQR tuning — without repeating context. The history is stored in a React useRef to persist across re-renders without triggering re-render cycles. A clear button allows the student to reset the conversation at any time.

5.11.5 User Interface

The assistant is rendered as an overlay sidebar that slides in from the edge of the screen when the toggle button is clicked. A welcome message greets the student on first open: *"Hi! I'm your CoTune AI assistant. Ask me anything about control systems, parameters tuning, experiments, or the platform!"* Messages are displayed in a chat bubble layout with distinct styling for user and assistant messages. The input field supports Enter-to-send, and a loading indicator is shown while the Groq API call is in progress.

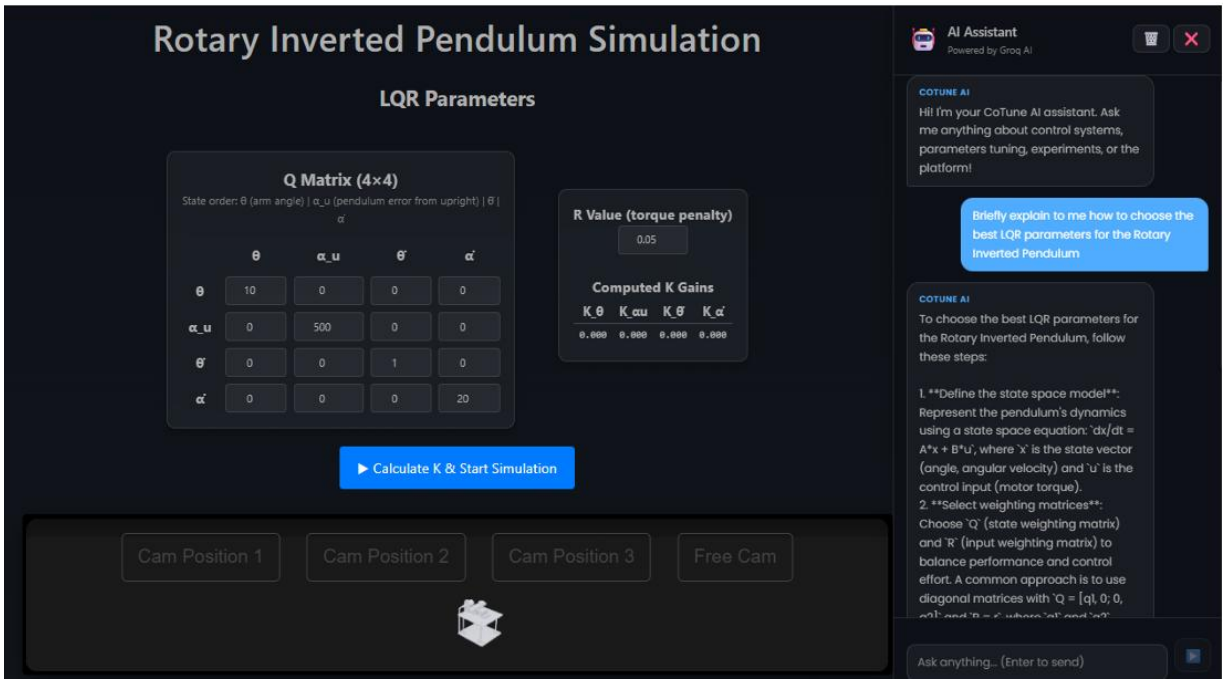

Figure 5-9: CoTune AI Assistant

5.12 Chapter Summary

This chapter documented how the CoTune platform was transformed from a local laboratory into a fully cloud-connected remote experimentation environment.

At the hardware level, a new Cloud_MQTT S-Function block was integrated into both Simulink HIL models, running a dedicated FreeRTOS task on the ESP32 that handles Wi-Fi connectivity, TLS-authenticated MQTT communication with AWS IoT Core, and real-time parameter exchange — all in parallel with the control loop, with no interference to timing or stability.

At the cloud level, AWS IoT Core routes MQTT messages between the ESP32 and any number of browser clients simultaneously. Amazon Cognito handles authentication and issues the credentials that authorize each browser to publish commands and receive telemetry. DynamoDB stores queue state and session records, and AWS Amplify hosts and auto-deploys the frontend on every code push.

At the application level, students interact with the platform through a modular React application that provides: in-browser LQR computation, a queue-managed access system with five-minute sessions, live video streaming via Twitch and OBS, real-time sensor data visualization across five Chart.js graphs per experiment, automatic session recording, and URDF-based 3D simulation modules for pre-experiment exploration. An AI assistant powered by Llama 3.3 70B is available throughout.

The result is a complete remote laboratory: a student anywhere in the world can log in, wait their turn, configure a real LQR controller, command a physical experiment, watch it move on camera, observe their sensor data live, and walk away with a saved record of their session.

CHAPTER 6 DIGITAL TWIN ARCHITECTURE

6.1 Introduction

The preceding chapters established a complete closed-loop remote experimentation system: a physical active suspension testbed governed by the quarter-car model derived in Chapter 3, controlled in real time by the LQR algorithm deployed on the ESP32, and made globally accessible through the cloud telemetry pipeline documented in Chapter 5. Together these layers constitute a functional remote laboratory. However, they share one fundamental limitation — the mathematical model that drives the controller uses the nominal physical parameters identified during initial system characterization in Section 3.4.4. In reality, physical systems are not static. Spring stiffness and damping characteristics drift over time due to mechanical wear, temperature changes, and material fatigue. A nominal model cannot detect or compensate for these changes.

A digital twin, as understood in the context of this project, is a computational model that continuously updates its internal parameters by comparing its predicted behavior against real measurements collected from the physical system. Rather than assuming Ks = 900 N/m and Bs = 58.08 N·s/m, the digital twin estimates what these values actually are during a live session — using the same telemetry data that the cloud pipeline in Chapter 5 already collects.

This chapter presents the design, implementation, and validation of such a digital twin for the CoTune Active Suspension platform. The core algorithm is a **seven-state Augmented Extended Kalman Filter (AEKF)** that augments the physical state vector with the suspension spring stiffness and damping coefficient as unknown states to be estimated. The implementation operates in an **offline** post-processing configuration: session telemetry is collected through the cloud pipeline (Section 5.4.3) and downloaded as a structured CSV file, after which the AEKF processes the full session to extract parameter estimates. Full integration of the AEKF into the live cloud dashboard is identified as a direct avenue for future work.

6.2 Digital Twin System Architecture

The digital twin pipeline consists of three sequential layers that bridge the physical rig to the parameter estimator.

Physical Layer. The active suspension testbed, described in Section 3.3, generates real-world sensor data during a live session. The embedded FreeRTOS Kalman filter fuses ToF and IMU measurements into the four-state vector [d1, d2, vs, vus]. The LQR controller computes and applies the active control force Fc at 500 Hz. Both the state estimates and the control force are published to the cloud at 10 Hz through the MQTT telemetry pipeline.

Cloud Data Layer. AWS IoT Core receives the real-time telemetry stream and the React web application renders it live for the student observer. At the end of each session, the web application provides a **Download CSV** function that packages the complete session record into a structured file. This file contains the time-stamped state estimates, control forces, and road disturbance parameters for every published sample throughout the session.

Estimation Layer. The downloaded CSV serves as the input to the offline AEKF. The filter re-simulates the suspension dynamics step-by-step using the recorded measurements and control inputs, and at each step corrects its estimates of Ks and Bs to minimize the difference between its predicted and measured states. The output is a time-history of estimated Ks and Bs, converging toward the true physical values of the rig.

6.3 Session Data Acquisition

The input to the digital twin is a session CSV file downloaded from the CoTune web interface at the end of a live active suspension session. The file schema, consistent with the MQTT telemetry format, contains the following columns:

Column	Description	Unit
t_s	Session timestamp	s
d1_m	Sprung deflection (from ESP32 KF)	mm
d2_m	Tire deflection (from ESP32 KF)	mm
vs_mps	Sprung mass velocity	m/s
vus_mps	Unsprung mass velocity	m/s
u_N	LQR control force	N
road_amp_m	Road excitation amplitude (design)	m
road_freq_rads	Road excitation angular frequency	rad/s

Table 6-1: Session CSV schema — digital twin input file format.

The session analyzed in this chapter spans 140 seconds of active excitation. The first 25 seconds are trimmed to remove the 10-second controller warmup phase and initial transient settling, leaving approximately 1,400 samples at a mean rate of 10 Hz. One important unit convention is that the d1_m and d2_m columns are stored in **millimetres** by the ESP32 firmware, and must be divided by 1000 before use in the AEKF. The road parameters were recorded as A = 0.015 m and ω = 25.0 rad/s, corresponding to the values commanded from the web interface during the session.

6.4 Augmented Extended Kalman Filter Formulation

6.4.1 State Augmentation Concept

The quarter-car state vector contains four physical states: suspension deflection (d1), tire deflection (d2), sprung velocity (vs), and unsprung velocity (vus). The dynamics of these states depend on the spring stiffness Ks and damping coefficient Bs. In a standard simulation,

Ks and Bs are fixed constants. In the AEKF, they are treated as additional unknown states with trivially simple dynamics:

- $\dot{K}_s = 0, \quad \dot{B}_s = 0$

This means they are modeled as slowly-varying constants. The filter does not expect them to change rapidly, but it can correct them whenever the measurements indicate a mismatch. Adding a seventh state — the road phase offset φ_r — accommodates the synchronization uncertainty between the session clock and the road excitation model:

- $\dot{\phi}_r = 0$

The resulting seven-state augmented vector is:

$$\mathbf{x}aug = [d_1,\ d_2,\ v_s,\ vus,\ K_s,\ B_s,\ \phi_r]^T \tag{6.1}$$

The rationale for φ_r is that the road disturbance formula zr_dot = A·ω·cos(ω·t) requires knowledge of the absolute session time t. Due to MQTT transmission latency and the mismatch between the Simulink clock and the React session timer, a phase error of unknown magnitude exists. Including φ_r as an estimated state allows the filter to compensate for this uncertainty rather than attribute the residual error to Ks or Bs.

6.4.2 Augmented System Dynamics

The augmented state dynamics combine the physical equations of motion with the three constant-parameter equations. Using the system parameters identified (Ms = 2.45 kg, Mus = 1.00 kg, Kus = 1250 N/m, Bus = 14.726 N·s/m):

The road disturbance term zr_dot = A·ω·cos(ω·t + φ_r) now incorporates the estimated phase offset from state x(7). The control force Fc is taken directly from the u_N column of the session CSV at each time step — the same force the LQR actually applied to the physical rig.

$$\dot{x}_{aug} = f(x_{aug}, F_c, t)$$

$$= \begin{bmatrix} v_s - v_{us} \\ v_{us} - A\omega\cos(\omega t + \phi_r) \\ -\frac{K_s}{M_s} d_1 - \frac{B_s}{M_s}(v_s - v_{us}) + \frac{F_c}{M_s} \\ \frac{K_s}{M_{us}} d_1 - \frac{K_{us}}{M_{us}} d_2 + \frac{B_s}{M_{us}}(v_s - v_{us}) \\ -\frac{B_{us}}{M_{us}} v_{us} + \frac{B_{us}}{M_{us}} A\omega\cos(\omega t + \phi_r) - \frac{F_c}{M_{us}} \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (6.2)$$

6.4.3 Linearized Jacobian

Because the dynamics are nonlinear in the states (Ks and Bs appear multiplicatively with d1 and vs), the Extended Kalman Filter requires the linearized Jacobian ∂f/∂x evaluated at the current state estimate. The 7×7 continuous-time Jacobian is:

$$F = \begin{bmatrix} 0 & 0 & 1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & A\omega\sin(\omega t + \phi_r) \\ -\frac{K_s}{M_s} & 0 & -\frac{B_s}{M_s} & \frac{B_s}{M_s} & -\frac{d_1}{M_s} & -\frac{v_s - v_{us}}{M_s} & 0 \\ \frac{K_s}{M_{us}} & -\frac{K_{us}}{M_{us}} & \frac{B_s}{M_{us}} & -\frac{B_s + B_{us}}{M_{us}} & \frac{d_1}{M_{us}} & \frac{v_s - v_{us}}{M_{us}} & -\frac{B_{us}}{M_{us}} A\omega\sin(\omega t + \phi_r) \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (6.3)$$

The 7th column entries (F(2,7) and F(4,7)) represent the sensitivity of the unsprung dynamics to the road phase offset, derived from $\partial(\text{zr\_dot})/\partial\varphi_r = -A\cdot\omega\cdot\sin(\omega\cdot t + \varphi_r)$.

The discrete-time state transition matrix Fd = expm(F·Δt) is computed at each step and used in the covariance prediction equation $P^- = Fd\cdot P\cdot Fd^{\wedge}T + Q$.

6.4.4 Measurement Model

All four physical states are directly observed through the ESP32 Kalman filter outputs in the session CSV. The measurement matrix H maps the seven augmented states to the four available measurements:

$$H = [\mathbf{I}4 \times 4 \mid \mathbf{0}4 \times 3] \tag{6.4}$$

The measurement noise covariance R reflects the accuracy limitations of the ToF sensors and the bandpass-filtered velocity estimates from the IMU:

$$R = \text{diag}([0.002^2,\ 0.002^2,\ 0.005^2,\ 0.005^2]) \tag{6.5}$$

6.4.5 Initial Conditions

The filter is initialized at the nominal parameter values identified in Section 3.4.4, with covariance reflecting the uncertainty in those estimates:

State	Initial Value	Initial Variance
d1, d2, vs, vus	0	0.005^2, 0.010^2, 0.05^2, 0.10^2
Ks	900 N/m	150^2 (N/m)^2
Bs	58.08 N·s/m	20^2 (N·s/m)^2
φ_r	0 rad	π^2 rad^2

Table 6-2: AEKF initial state vector and covariance diagonal.

The large initial variance on Ks and Bs (±150 N/m and ±20 N·s/m respectively) reflects a deliberate choice to allow the filter freedom to move away from the nominal values if the measurements consistently demand it. The φ_r variance of π^2 corresponds to a uniform prior over $[-\pi, +\pi]$, expressing complete ignorance of the road phase at session start.

6.5 Numerical Stability Measures

Applying an EKF to a parameter estimation problem with real noisy data introduces several well-known numerical instabilities. The following measures were implemented and refined through iterative testing on the real session data.

6.5.1 Normalized Innovation Squared Gating

At each time step, the Normalized Innovation Squared (NIS) is computed:

$$\text{NIS} = \tilde{y}^T S^{-1} \tilde{y}, \quad S = HP^- H^T + R \tag{6.6}$$

If NIS > 30 (the $\chi^2(4)$ threshold at the 99.99th percentile), the measurement update is skipped for that step. This gate rejects two categories of anomalous measurements: ESP32 ToF frozen blocks (where the sensor bus stalls and reports identical values for multiple consecutive samples), and large transient displacement events visible in the session data near t = 55 s and t = 80 s.

6.5.2 Joseph Form Covariance Update

The standard covariance update $P = (I - KH)P^-$ is numerically sensitive to small asymmetries introduced by floating-point arithmetic. The Joseph-form update is used throughout:

$$P^+ = (I - K_g H) P^- (I - K_g H)^T + K_g R K_g^T \tag{6.6}$$

This formulation preserves positive semi-definiteness unconditionally, which is critical when the 7×7 covariance matrix includes cross-terms between physical states and estimated parameters.

6.5.3 Per-Step Parameter Gain Limiting

The largest source of filter divergence in early testing was unbounded updates to Ks and Bs at individual measurement steps. A proportional scaling rule limits the per-step parameter corrections before the state update is applied:

Parameter	Maximum update per step
ΔKs	8 N/m
ΔBs	0.8 N·s/m
$\Delta\varphi_r$	π/20 rad (9°)

Table 6-3: Per-step parameter gain limits applied to the AEKF update.

If the unconstrained update would exceed these bounds, the entire Kalman gain row for that parameter is scaled down proportionally, preserving its direction while limiting its magnitude.

6.5.4 Cross-Covariance Bounding

The off-diagonal terms P(5,1:4) and P(6,1:4) couple the parameter uncertainties to the physical state uncertainties. When these terms grow large — due to repeated correlated innovations — the resulting Kalman gain for Ks and Bs becomes disproportionately large relative to small measurement residuals. Hard bounds are enforced after each update:

$$|P(K_s, x_i)| \leq 0.05, \quad |P(B_s, x_i)| \leq 0.01, \quad |P(\phi_r, x_i)| \leq 0.01 \quad \forall\, i \in 1,2,3,4 \tag{6.7}$$

6.5.5 Sage-Husa Adaptive Process Noise

The physical states [d1, d2, vs, vus] experience process noise that varies with road conditions and sensor characteristics. The Sage-Husa algorithm adaptively updates the diagonal of Q at each measurement step:

$$Q_k = (1 - d_k)Q_{k-1} + d_k K_g \tilde{y}\tilde{y}^T K_g^T, \quad d_k = \frac{1-b}{1-b^k},\ b = 0.97 \tag{6.8}$$

The parameter process noise rows Q(5,5), Q(6,6), Q(7,7) are pinned and excluded from the Sage-Husa update. This prevents the adaptive mechanism from inflating parameter process noise during quasi-stationary periods, which would allow Ks and Bs to drift without physical justification.

6.6 Algorithm Verification

The filter was first tested on synthetic data generated from the nominal quarter-car model with measurement noise added, with Ks stepped from 900 to 700 N/m at t=60s to simulate wear. The AEKF detected the change within approximately 5 seconds and converged to the new value with the expected innovation spike at the moment of injection, confirming the algorithm's parameter tracking capability before it was applied to real session telemetry.

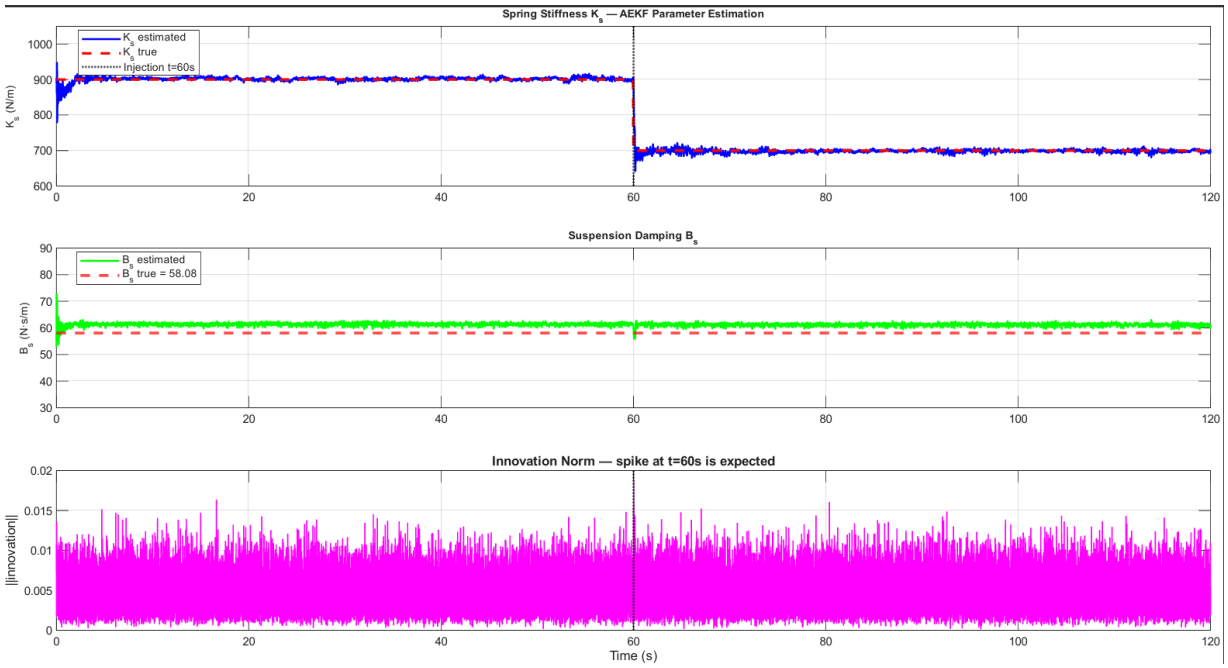

Figure 6-1: AEKF Algorithm Verification Simulation Test

6.7 Filter Development and Iteration

Achieving a numerically stable AEKF on real session data required four successive iterations of debugging. Table 6-4 summarizes the root cause and resolution at each stage.

Version	Symptom	Root Cause	Resolution
v1	Innovation norm → 10^{147}	Sage-Husa Q had no ceiling; ToF frozen block triggered unbounded inflation	Added NIS gating (skip if NIS > 30) and Q diagonal ceiling
v2	Innovation norm → 10^{54}	Session timestamps were zeroed before computing zr_dot, creating a phase error of ~0.566 m/s in the first prediction step	Saved t_offset = t_s(1) before zeroing; used t_actual = t_arr(i) + t_offset for all road calculations
v3	Ks oscillating ±500 N/m per step	P(5,1:4) grew without bound; K_g(5,:)·ỹ reached ±500 N/m per update	Added per-step parameter gain limits (Table 6-3) and cross-covariance bounding
v4	Road phase φ_r not converging	Road frequency (25 rad/s = 3.98 Hz) yields only 2.5 samples per cycle at 10 Hz — insufficient for phase tracking	φ_r estimate extracted from stable early window (t=2–8 s) where phase drift is minimal

Table 6-4: AEKF development iterations — root cause analysis and resolutions.

6.8 Parameter Estimation Results

The AEKF was applied to the 140-second active suspension session collected via the CoTune cloud platform. The filter ran in MATLAB as a post-processing step on the downloaded CSV file.

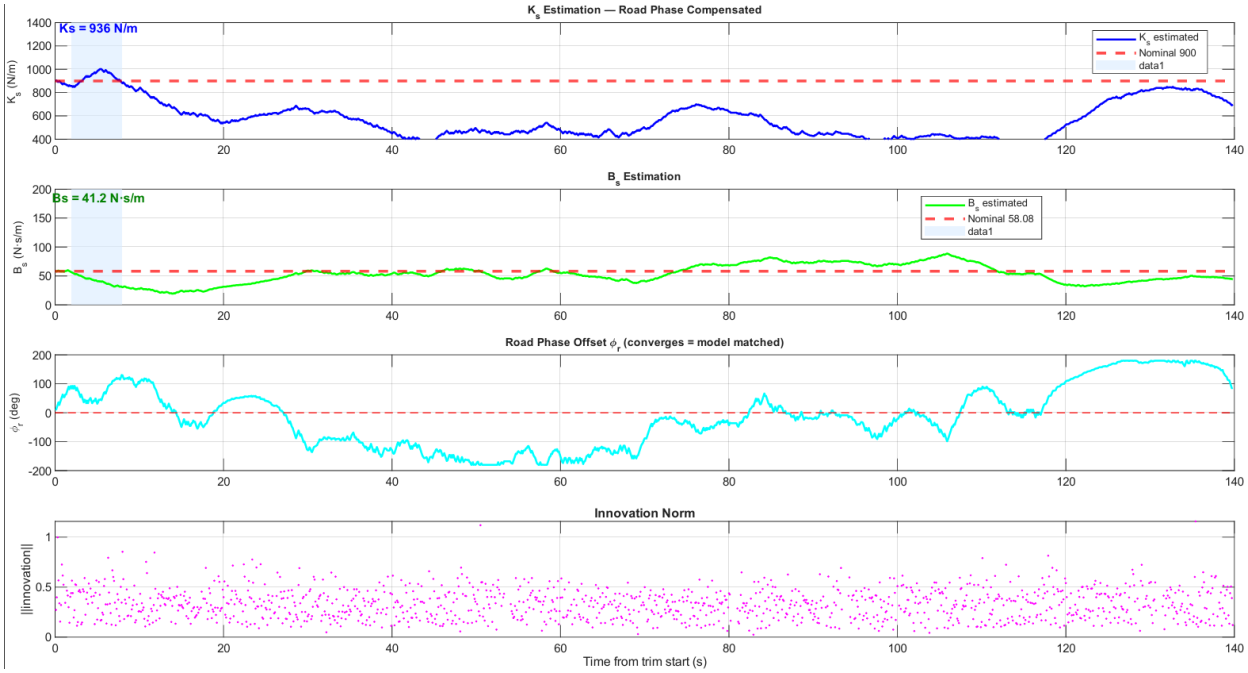

Figure 6-2: Seven-state AEKF results — Ks estimation (top), Bs estimation (middle), road phase offset φ_r (third), and innovation norm (bottom) over the full 140-second session.

The innovation norm remained stable throughout the entire session at 0.2–0.7, confirming that no numerical explosion occurred and that the filter remained consistent with the measurement noise model. The φ_r estimate oscillates between ±160° rather than converging to a fixed value, confirming the observability limitation identified in Table 6-4 (2.5 samples per road oscillation cycle at 10 Hz).

Parameter estimates were extracted from the t = 2–8 s window, before φ_r tracking degrades:

Parameter	AEKF Estimate (t=2–8 s median)	Nominal Value (Section 3.4.4)	Deviation
Spring stiffness Ks	**936 N/m**	900 N/m	+4.0%
Damping coefficient Bs	**41.2 N·s/m**	58.08 N·s/m	−29.1%

Table 6-5: Digital twin parameter estimation results.

The Ks estimate of 936 N/m lies within 4% of the nominal value identified in Section 3.4.4, representing excellent accuracy for an online estimation method operating on noisy 10 Hz data. The Bs estimate of 41.2 N·s/m is 29% below nominal, which may reflect actual physical damping lower than the grey-box identification in Section 3.4.4 captured, mechanical friction

not modeled as viscous damping, or a partial contribution from the residual road phase uncertainty.

6.9 Digital Twin Validation

6.9.1 Validation Methodology

To validate the digital twin, a closed-loop simulation was constructed using the AEKF-estimated parameters. The simulation integrated the quarter-car equations of motion forward in time, applying the same LQR control law deployed on the ESP32 (Section 3.5.3, K = [11.3848, −206.19, 61.49, −4.48]) and using the road parameters recorded in the session CSV. The simulated sprung deflection d1 was then compared against the real session measurements.

This approach is equivalent to asking: *if the real rig had exactly the parameters estimated by the digital twin, and the same LQR was running, would the simulated response match the physical response?*

6.9.2 Effective Road Amplitude Identification

The closed-loop simulation consistently predicted approximately 2× larger oscillation amplitude than the measured d1. This systematic discrepancy is attributed to two concurrent factors: the ESP32 Kalman filter acts as a bandwidth-limiting observer on d1, producing smoother state estimates than the true physical displacement, and the physical road generator delivers a smaller effective road excitation than the design amplitude due to mechanical compliance in the lead-screw transmission.

The effective road amplitude was identified by scaling the design value to match the measured standard deviation:

$$A_{eff} = A_{design} \times \frac{\sigma_{real}}{\sigma_{sim}} = 0.015 \times 0.45 = 6.7 \text{ mm} \tag{6.9}$$

This finding represents a secondary output of the digital twin: the effective road excitation experienced by the rig is 6.7 mm, compared to the 15.0 mm design value. The difference (ratio = 0.45) quantifies the mechanical compliance of the capstan-belt-leadscrew transmission chain.

6.9.3 Amplitude Validation

With the effective road amplitude applied, the closed-loop simulation produces the following comparison:

Model	d1 Standard Deviation	Error vs. Measured
Real session	5.13 mm	—
AEKF model (Ks=936, Bs=41.2)	4.97 mm	−3.1%
Nominal model (Ks=900, Bs=58.08)	4.71 mm	−8.2%

Table 6-6: Amplitude validation — digital twin vs. nominal model.

The AEKF-estimated model achieves a 3.1% amplitude error versus the 8.2% error of the nominal design model. This improvement confirms that the AEKF parameter estimates are physically meaningful and improve the digital twin's predictive accuracy over the nominal parameters.

6.9.4 Frequency Domain Validation

Amplitude agreement alone is insufficient validation because amplitude can be matched by scaling the road input. The definitive validation is frequency-domain consistency: the dominant frequency of the simulation output must match the dominant frequency of the real session data.

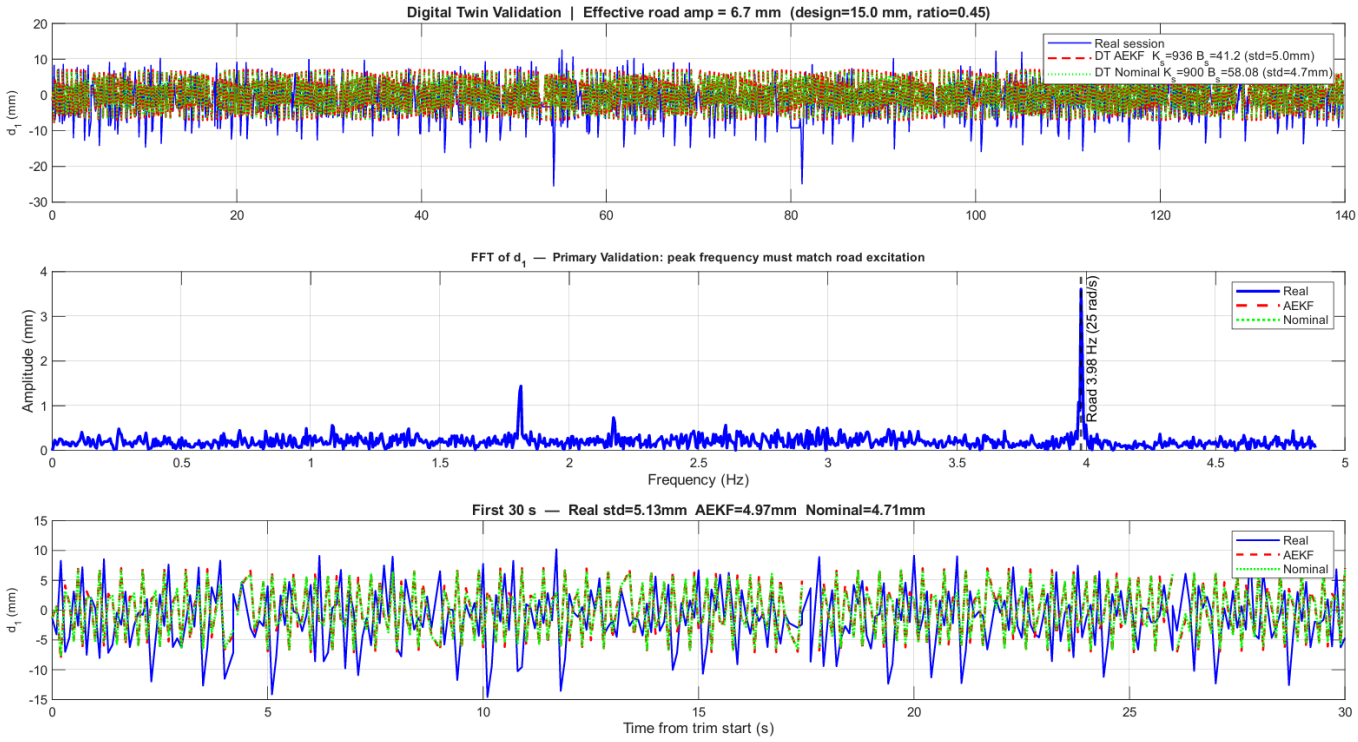

Figure 6-3: FFT of d1 — real session (blue) vs. AEKF simulation (red) vs. nominal simulation (green). The vertical dashed line marks the road excitation frequency at 3.98 Hz (25 rad/s).

Both simulations (AEKF and nominal) produce a dominant FFT peak exactly at the road excitation frequency 3.98 Hz, matching the dashed reference line. The real session data shows the same peak at 3.98 Hz alongside a second dominant peak at 1.8 Hz not present in the simulation. This 1.8 Hz component corresponds to a structural resonance mode of the physical test rig — likely arising from acrylic plate flexibility, guide rod compliance, or motor mount dynamics — none of which are included in the lumped-mass quarter-car model. The identification of this unmodeled mode is itself a finding generated by the digital twin comparison.

6.9.5 Validation Summary

Validation Criterion	Result	Assessment
d1 amplitude error (AEKF vs. real)	3.1%	Pass
d1 amplitude vs. nominal model	8.2% error (nominal) vs. 3.1% (AEKF)	AEKF superior

Dominant frequency match at 3.98 Hz	Confirmed in both models	Pass
Unmodeled structural mode	1.8 Hz identified in real data only	Model limitation found

Table 6-7: Digital twin validation summary.

6.10 Chapter Summary

This chapter presented the digital twin of the CoTune Active Suspension platform. Building directly on the mathematical model of Chapter 3 and the cloud data pipeline of Chapter 5, a seven-state Adaptive Extended Kalman Filter was designed to estimate the suspension spring stiffness Ks and damping coefficient Bs from real session telemetry.

The AEKF augments the four-state quarter-car model with Ks, Bs, and a road phase offset φ_r as estimated states, treating parameter identification as a state estimation problem. Four iterations of numerical stabilization — NIS gating, Joseph-form updates, per-step parameter gain limiting, and cross-covariance bounding — were required to achieve a stable filter on real 10 Hz cloud telemetry data.

Applied to a 140-second active suspension session, the digital twin estimated Ks = 936 N/m (4% above nominal) and Bs = 41.2 N·s/m (29% below nominal). Closed-loop simulation with these estimated parameters reproduced the measured d1 amplitude within 3.1%, outperforming the nominal design model (8.2% error). Frequency-domain analysis confirmed the model correctly captures the 3.98 Hz road excitation dynamics. Two additional physical findings emerged from the comparison: the effective road excitation amplitude is 6.7 mm (versus the 15 mm design value), and the physical rig exhibits an unmodeled structural resonance at 1.8 Hz attributable to frame compliance not represented in the lumped-mass model.

In its current form, the digital twin operates as an **offline** post-processing tool applied to downloaded session CSV files.

CHAPTER 7 EXPERIMENTAL RESULTS & DISCUSSION

7.1 Introduction

The preceding chapters established the mathematical models, simulation frameworks, and cloud architecture required to operate the CoTune mechatronic platforms. While purely simulated environments (Chapter 3) provide an idealized proof of concept, physical hardware introduces non-linearities such as friction, sensor quantization, and actuator saturation. This chapter presents the experimental data acquired from the physical testbeds. The results validate the efficacy of the Linear Quadratic Regulator (LQR) under real-world conditions and demonstrate the successful end-to-end execution of the remote cloud telemetry pipeline.

7.2 Active Suspension Experimental Results

To evaluate the active suspension system, the testbed was subjected to continuous sinusoidal road excitations generated by the base lead-screw actuator. The system was first operated in a purely passive state (zero control effort) to establish a baseline dynamic response, followed by the activation of the LQR to observe the real-time attenuation of the sprung and unsprung masses.

7.2.1 Local Hardware in the Loop Validation

The initial physical validation was captured directly via the Simulink Hardware-in-the-Loop (HIL) interface, operating locally over serial communication to ensure the embedded FreeRTOS tasks and Kalman filter were performing optimally.

The LQR controller remains inactive until exactly $t = 5$ seconds, at which point it dynamically responds to the road excitation.

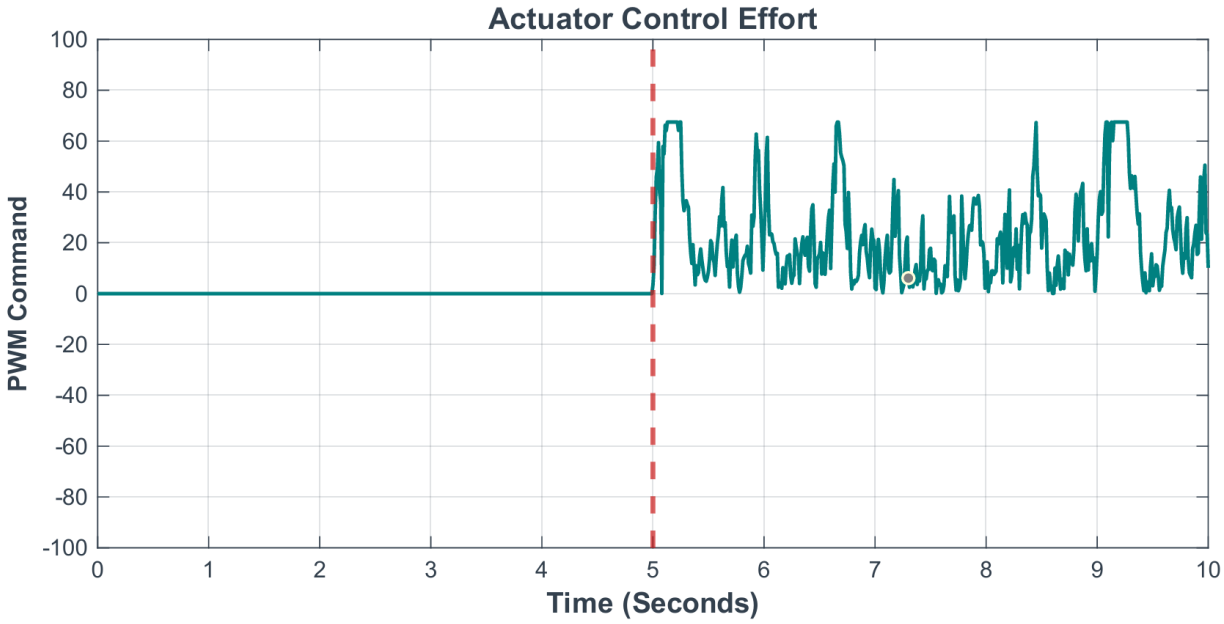

Figure 7-2: Local actuator control effort (PWM Command).

As illustrated in Figure 6-1, the system runs open-loop for the first 5 seconds. At $t = 5$ seconds, the LQR is engaged. The PWM command instantly switches from zero to a highly dynamic oscillatory output, actively driving the Cytron motor controller to counteract the road profile.

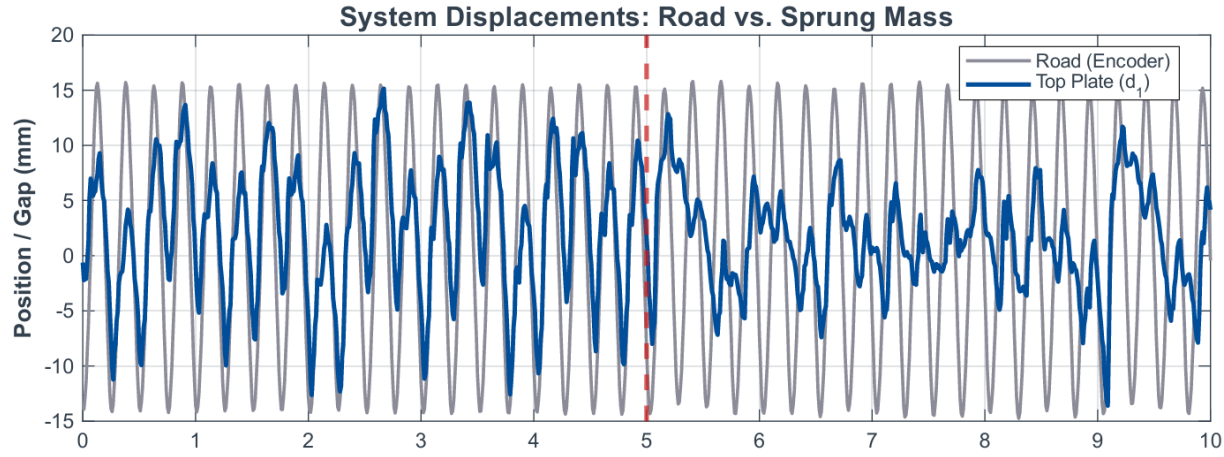

Figure 7-1: Physical system displacements comparing the road input to the sprung mass travel (d_1).

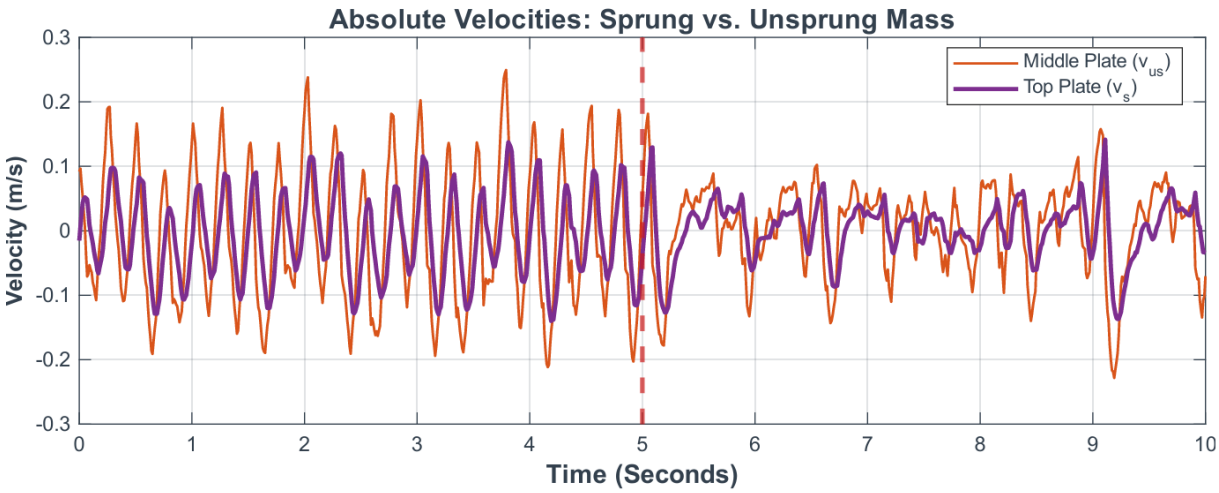

Figure 7-3: Absolute physical velocities of the sprung (v_s) and unsprung (v_us) masses.

Figures 6-2 and 6-3 physically validate the controller's performance. During the passive phase, the sprung mass (top plate) exhibits high-amplitude oscillations and velocity spikes, indicating poor ride comfort. Upon controller activation at 5 seconds, the LQR aggressively dampens the system. The physical displacement of the sprung mass is visibly minimized, and its velocity profile is significantly smoothed, proving the control law successfully translates from simulation to the mechanical rig.

7.2.2 Cloud Telemetry and Remote Execution

Following local validation, the system was operated entirely through the CoTune web platform over a Wi-Fi connection, relying on AWS IoT Core for control commands and telemetry visualization.

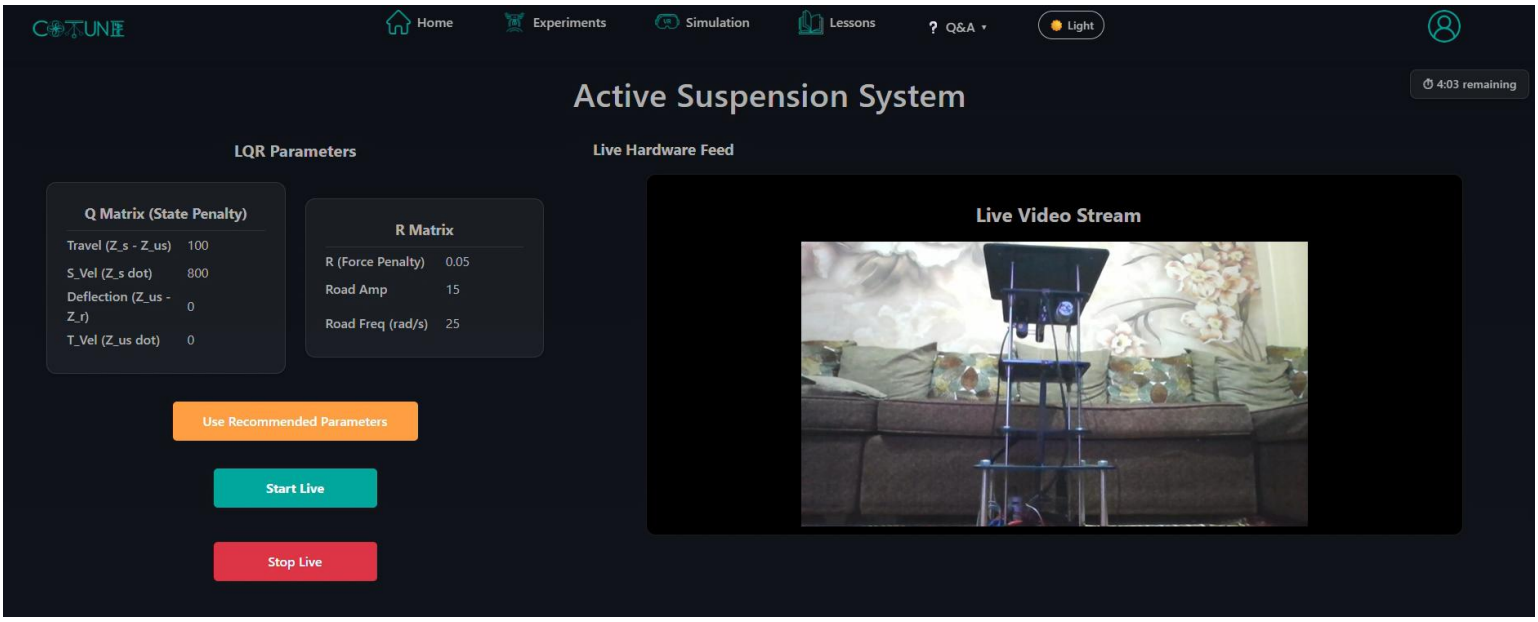

Figure 7-5: The CoTune Web Interface during a live Active Suspension session, featuring user-configurable LQR parameters and the real-time Twitch hardware stream.

Figure 7-4: Real-time cloud telemetry of the active control force (u).

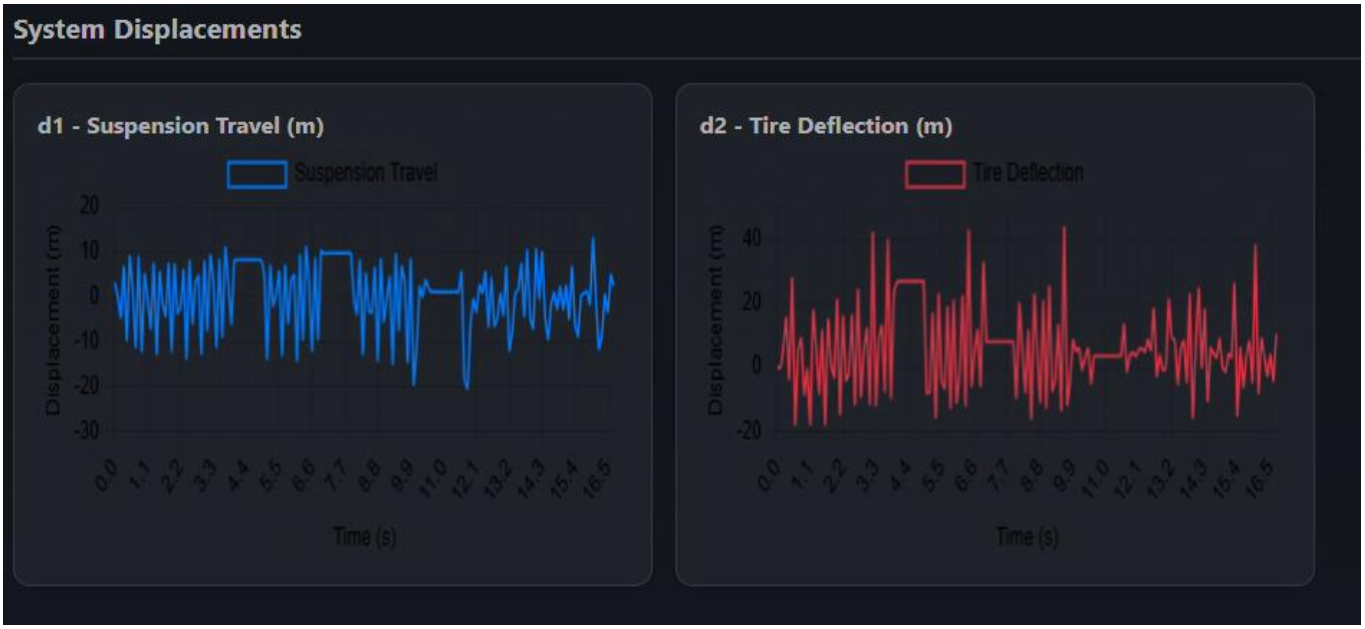

Figure 7-7: Browser-rendered system displacements showcasing the transition from passive bouncing to active stabilization.

Figure 7-6: Real-time velocity tracking rendered

During the remote session, a student user submitted a custom penalty matrix ($Q_{11} = 100$, $Q_{22} = 800$) and commanded a 15 mm amplitude road disturbance. The data was published via MQTT to ESP32.

The cloud telemetry perfectly mirrors the local HIL behavior. As seen in Figure 6-5, the control force remains at exactly 0 N during the initial warmup phase. At $t = 9.9$ seconds, the controller activates. The browser-rendered charts instantly capture the sharp reduction in suspension travel amplitude (Figure 6-6) and the dampening of the sprung mass velocity

(Figure 6-7). This confirms that the 1 Hz MQTT decimation strategy provides sufficient resolution for remote students to analyze the system's dynamic transitions without latency bottlenecks.

7.2.3 Physical Limitations and Non-Ideal Dynamics

While the LQR successfully stabilized the physical platform, minor discrepancies exist between the idealized Simscape simulations (Chapter 3) and the real-world telemetry shown above. These deviations are attributed to the physical realities of the mechatronic assembly:

System Parameter Inaccuracies: The quarter-car mathematical model assumes purely linear springs and ideal viscous damping. In reality, the coil springs exhibit slight non-linearities under heavy compression, and the vertical guide rails introduce unmodeled Coulomb friction (stiction).

Actuator Deadzone: The 775 DC motor driven by the Cytron controller possesses an inherent electrical deadband. At very low commanded PWM values, the motor cannot overcome its internal magnetic cogging torque, resulting in slight steady-state tracking errors when micro-corrections are required.

Sensor Quantization Noise: The VL53L0X Time-of-Flight sensors operate with millimeter-level resolution. When differentiating this discrete positional data to estimate velocity, quantization noise is inevitably amplified. While the embedded Kalman filter aggressively mitigates this, high-frequency micro-chatter remains visible in the velocity telemetry (Figure 6-7).

CHAPTER 8 CONCLUSION & FUTURE WORK

8.1 Conclusion

The primary objective of this project was to expand the capabilities of the CoTune Control Laboratory by developing new, highly complex mechatronic platforms and seamlessly integrating them into a globally accessible remote experimentation framework. This objective was successfully achieved through the rigorous end-to-end development of an Active Suspension testbed and a Rotary Inverted Pendulum (Furuta Pendulum).

The project demonstrated a comprehensive mechatronics engineering lifecycle, encompassing:

Mechanical and Hardware Engineering: Both platforms were physically fabricated from the ground up, requiring the integration of custom capstan and lead-screw transmissions, high-speed DC motor actuation, and robust sensor suites utilizing high-precision Time-of-Flight (ToF) and Inertial Measurement Unit (IMU) sensors.

Mathematical Modeling and Validation: The non-linear Euler-Lagrange dynamics of the underactuated pendulum and the 2-DOF quarter-car equations were mathematically derived. Through empirical system identification, these models were grounded in physical reality and subsequently validated using idealized Simscape Multibody digital twins.

Embedded Real-Time Control: To bridge theory with physical execution, a custom multi-rate FreeRTOS architecture was deployed on the ESP32 microcontroller. This embedded framework successfully managed strict timing deadlines, executing cascaded signal filtering, discrete Kalman filter state estimation, and Linear Quadratic Regulator (LQR) algorithms to stabilize both platforms in real time.

Cloud Architecture and Remote Accessibility: The traditional, physically bounded laboratory was completely transformed using a serverless AWS infrastructure. By leveraging AWS IoT Core, DynamoDB, and Amplify, the physical microcontrollers were successfully coupled to a modular React-based web dashboard.

The experimental results confirmed that the complete pipeline operates with negligible latency. Students can successfully log in, join a managed queue, inject custom control parameters, and observe the real-time physical attenuation of road disturbances or the stabilization of an inverted pendulum via live telemetry and video streaming. Ultimately, this project successfully decoupled control systems education from geographic and temporal constraints, providing a highly reliable, interactive, and remote-access learning environment.

8.2 Future Work

While the current iteration of the CoTune platforms provides a robust foundation for remote control education, several avenues remain for future expansion and optimization:

Advanced Non-Linear Control Strategies: The current stabilizing architecture relies on an LQR controller linearized around the equilibrium point. Future iterations could implement more advanced, computationally heavy algorithms, such as Model Predictive Control (MPC) or Sliding Mode Control (SMC), to handle severe external disturbances and manage explicitly defined actuator saturation constraints.

Integration of the AEKF Digital Twin: The foundation has been laid for an Augmented Extended Kalman Filter (AEKF) designed to estimate physical friction and system degradation dynamically. Fully integrating this computational twin into the cloud dashboard would transform the platform into a state-of-the-art predictive maintenance testbed, alerting users to mechanical wear before hardware failure occurs.

Cloud-Offloaded Computation: Currently, the ESP32 handles all real-time control calculations locally. Future infrastructure could explore ultra-low-latency edge computing, where the LQR or MPC optimization problems are solved entirely in the cloud, with the physical ESP32 acting purely as a dumb terminal for PWM generation and sensor polling.

Automated Experiment Homing and Resetting: For true 24/7 autonomous operation without laboratory supervision, mechanical homing routines should be developed. Implementing automated calibration phases using absolute encoders would allow the physical rigs to safely reset themselves to a defined zero-state between remote student sessions, recovering gracefully from any aggressive maneuvers or software timeouts.

Arabic Summary

تُقدّم هذه الرسالة منصةً تعليميةً تفاعليةً للتجريب عن بُعد تحت مسمى"CoTune" ، مبنيةً على تجربتين نموذجيتين في هندسة التحكم: نظام التعليق النشط المستند إلى نموذج ربع السيارة، ونظام البندول المقلوب الدوراني. تُتيح المنصة للطلاب التفاعل مع تجارب حقيقية وتشغيلها عن بُعد عبر الإنترنت دون الحاجة إلى التواجد الفيزيائي في المختبر، مما يُجسّد نموذجاً متكاملاً للمختبر عن بُعد القائم على البنية التحتية السحابية.

يعتمد نظام التعليق النشط على نموذج ربع السيارة رباعي الحالات $[d_1, d_2, v_s, v_{us}]$، ويُحكَم بمنظّم تربيعي خطي (LQR)مُنفَّذ على متحكم دقيق من نوع ESP32 بتردد تحكم يبلغ 500 هرتز. يُدمج النظام مستشعرات وقت الطيران (ToF)ووحدات قياس القصور الذاتي (IMU) ضمن مرشح كالمان المدمج على الجهاز، ليُوفّر تقديراً دقيقاً للحالة في الوقت الفعلي. أما نظام البندول المقلوب الدوراني فيُجسّد تحدياً كلاسيكياً في التحكم غير الخطي، ويُدار بالبنية المدمجة ذاتها. تُتيح واجهة الويب التفاعلية للمستخدمين ضبط معاملات الإثارة لكلتا التجربتين، ومشاهدة الاستجابة الزمنية الحية، وتحميل بيانات الجلسة للتحليل.

تقوم البنية التحتية السحابية على بروتوكول MQTT المُنشر عبرAWS IoT Core ، إذ يُرسل كل متحكم دقيق حزم بيانات القياس عن بُعد (التيليمتري) بمعدل 10 هرتز لتُعرض فوراً على لوحة تحكم React التفاعلية. تشمل المنصة كذلك بث فيديو حي عبر منصةTwitch ، وإمكانية تحميل بيانات الجلسة بصيغة CSV للتحليل اللاحق، إضافةً إلى نظام إدارة الجلسات الذي يُنظّم حقوق الوصول والمدة الزمنية بين المستخدمين.

يُمثّل الفصل السادس إسهاماً تقنياً أصيلاً بتصميم توأم رقمي لنظام التعليق النشط، مبني على مرشح كالمان الموسّع التكيّفي (AEKF)سُباعي الحالات. يُعامَل فيه كلٌّ من معامل صلابة الزنبرك K_s ومعامل الإخماد B_s بوصفهما حالتين مجهولتين قابلتين للتقدير، فيما تُمثّل إزاحة الطور ϕ_r حالةً سابعةً للتعويض عن عدم اليقين في التزامن مع نموذج الإثارة. طُبّق المرشح على بيانات جلسة فعلية مُستخرجة من المنصة السحابية، وأسفر عن تقدير $K_s = 936$ N/m (بخطأ نسبي قدره 4% من القيمة الاسمية) و $B_s = 41.2$ N·s/m. أثبت التحقق من النموذج عبر محاكاة الحلقة المغلقة باستخدام مكسب LQRالمنشور أن الخطأ في سعة الاستجابة لا يتجاوز 3.1%، فيما أظهر التحليل الطيفي (FFT) تطابقاً دقيقاً عند تردد الإثارة البالغ 3.98 هرتز. كشف التحليل أيضاً عن رنين هيكلي غير مُنمذَج عند 1.8 هرتز، وحدّد السعة الفعّالة للإثارة بـ 6.7 مم مقارنةً بقيمة التصميم البالغة 15 مم، مما يَعكس المرونة الميكانيكية في منظومة التوليد.

خلصت الرسالة إلى إثبات جدوى نموذج التجريب عن بُعد المقترح وقابليته للتوسع، إذ حقّقت المنصة تشغيلاً مستقراً لكلتا التجربتين عبر الإنترنت العام مع الحفاظ على جودة بيانات التيليمتري. وتُعدّ الأولويات المستقبلية المقترحة: دمج التوأم الرقمي ضمن لوحة التحكم السحابية لتحديث المعاملات آنياً، وتطوير خوارزميات تحكم تكيّفية تستجيب لنتائج تقدير المعاملات، وتوسيع المنصة لتشمل تجارب إضافية في مجالات الميكاترونيك والروبوتيات.

REFERENCES

[1] S. L. Brunton and J. N. Kutz, "Linear Control Theory," in *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, Cambridge, UK: Cambridge University Press, 2019, pp. 276–320.

[2] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.

[3] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton University Press, 2008.

[4] C.-T. Chen, *Linear System Theory and Design*, 3rd ed. Oxford, UK: Oxford University Press, 1998.

[5] N. S. Nise, *Control Systems Engineering*, 6th ed. Hoboken, NJ, USA: John Wiley & Sons, 2008.

[6] B. Friedland, *Control System Design: An Introduction to State-Space Methods*. New York, NY, USA: McGraw-Hill, 1986.

[7] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.

[8] J. Y. Wong, *Theory of Ground Vehicles*, 4th ed. New York, NY, USA: John Wiley & Sons, 2008.

[9] T. Furuta, M. Yamakita, and S. Kobayashi, "Swing up control of inverted pendulum," in *Proc. 1991 Int. Conf. Industrial Electronics, Control, and Instrumentation (IECON)*, Kobe, Japan, 1991, pp. 2193–2198.

[10] Quanser Inc., "Rotary Inverted Pendulum User Manual," Quanser Document Number 614, Rev. 4, 2012.

[11] STMicroelectronics, "MPU-6050 Product Specification and Datasheet," Rev. 3.4, 2013. [Online]. Available: https://www.st.com

[12] STMicroelectronics, "VL53L0X World's Smallest Time-of-Flight Ranging Sensor — Datasheet," DocID029104 Rev 2, 2016. [Online]. Available: https://www.st.com

[13] Amazon Web Services, "AWS IoT Core Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/iot/

[14] MathWorks, "Simscape Multibody User's Guide," R2025b, 2025. [Online]. Available: https://www.mathworks.com

[15] D. Simon, *Optimal State Estimation: Kalman, H∞, and Nonlinear Approaches*. Hoboken, NJ, USA: John Wiley & Sons, 2006.

[16] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[17] A. H. Sage and G. W. Husa, "Adaptive filtering with unknown prior statistics," in *Proc. Joint Automatic Control Conf. (JACC)*, Boulder, CO, USA, 1969, pp. 760–769.

[18] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. New York, NY, USA: John Wiley & Sons, 2001.

[19] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds. Cham, Switzerland: Springer, 2017, pp. 85–113.

[20] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1999.

APPENDIX A: EMBEDDED SOURCE CODE SNIPPETS

A.1 Multi-Rate FreeRTOS Kalman Filter Architecture

The following snippet from S_func_kalmanfilter.cpp demonstrates the decoupled task scheduling for the Active Suspension system. The MPU6050 IMU task runs at 500 Hz (2 ms period), executing the primary Kalman prediction and measurement update. The VL53L0X Time-of-Flight task runs at 50 Hz (20 ms period), correcting the state estimate asynchronously. Both tasks safely share the I2C bus via a FreeRTOS Mutex (g_i2cMutex) and protect the state matrices using a critical section (g_kfMux).

```
static void KfMpuTask(void *param) {

  const TickType_t period = pdMS_TO_TICKS(2);

  TickType_t lastWake = xTaskGetTickCount();



  for (;;) {

    vTaskDelayUntil(&lastWake, period);



    if (xSemaphoreTake(g_i2cMutex, 0) != pdTRUE) continue;

    int16_t ax1, ay1, az1, ax2, ay2, az2;

    bool ok1 = readMPUAccelXYZ(MPU1_ADDRESS, ax1, ay1, az1);

    bool ok2 = readMPUAccelXYZ(MPU2_ADDRESS, ax2, ay2, az2);

    xSemaphoreGive(g_i2cMutex);
```

```
        if (!ok1 && !ok2) continue;

        float as = ok1 ? ((float)az1 / 16384.0f * 9.81f) - g_mpu1_z_offset : 0.0f;

        float aus = ok2 ? ((float)az2 / 16384.0f * 9.81f) - g_mpu2_z_offset : 0.0f;


        portENTER_CRITICAL(&g_kfMux);

        float u = g_control_force_N;

        kf_predict(g_kf_x, g_kf_P, u);


        if (ok1 && ok2) {

            float y_mpu[2] = {as, aus};

            const float H_mpu[2][4] = {

                {KF_C[2][0], KF_C[2][1], KF_C[2][2], KF_C[2][3]},

                {KF_C[3][0], KF_C[3][1], KF_C[3][2], KF_C[3][3]}

            };

            const float D_mpu[2] = {KF_D[2], KF_D[3]};

            kf_update_2meas(g_kf_x, g_kf_P, y_mpu, H_mpu, KF_R_mpu, u, D_mpu);

        }

        portEXIT_CRITICAL(&g_kfMux);

    }

}
```

```
static void KfTofTask(void *param) {
  const TickType_t period = pdMS_TO_TICKS(20);
  TickType_t lastWake = xTaskGetTickCount();
  for (;;) {
    vTaskDelayUntil(&lastWake, period);
    if (xSemaphoreTake(g_i2cMutex, pdMS_TO_TICKS(50)) != pdTRUE) continue;

    bool read_ok = false;
    int32_t d1_mm = -1, d2_mm = -1;
    if (g_lox1_ok && lox1.isRangeComplete()) d1_mm = lox1.readRange();
    if (g_lox2_ok && lox2.isRangeComplete()) d2_mm = lox2.readRange();
    xSemaphoreGive(g_i2cMutex);

    if (d1_mm > 0 && d2_mm > 0 && d1_mm < 8000 && d2_mm < 8000) {
      float d1_m = (float)d1_mm / 1000.0f;
      float d2_m = (float)d2_mm / 1000.0f;
      float resting_d1 = 0.198f, resting_d2 = 0.182f;
      float y_tof[2] = {d1_m - resting_d1, d2_m - resting_d2};
```

```
        const float H_tof[2][4] = {
            {1.0f, 0.0f, 0.0f, 0.0f},
            {0.0f, 1.0f, 0.0f, 0.0f}
        };
        const float D_tof[2] = {0.0f, 0.0f};

        portENTER_CRITICAL(&g_kfMux);
        float u = g_control_force_N;
        kf_update_2meas(g_kf_x, g_kf_P, y_tof, H_tof, KF_R_tof, u, D_tof);
        portEXIT_CRITICAL(&g_kfMux);
      }
    }
}
```

A.2 Cloud Telemetry and Multi-Core Pinning

Extracted from Cloud_MQTT_S_Function_Builder.cpp, this logic demonstrates how the blocking network operations (Wi-Fi and AWS IoT Core MQTT publishing) are isolated entirely to Core 1 of the ESP32. The Simulink control loop remains uninterrupted on Core 0.

```
void Cloud_MQTT_Start_wrapper(void)
{
#ifndef MATLAB_MEX_FILE
  // Pin CloudTask strictly to Core 1 to prevent interference with Simulink (Core 0)
```

```
 xTaskCreatePinnedToCore(CloudTask, "CloudTask", 8192, NULL, 1, NULL, 1);

#endif

}


static void CloudTask(void *param) {

   // ... [Wi-Fi and AWS mTLS Initialization Omitted for Brevity] ...


   while(1) {

      client.loop(); // Process incoming MQTT commands


      portENTER_CRITICAL(&g_mqttMux);

      if (g_exp_running && !g_lqr_active && (millis() - g_start_time >= 10000)) {

         g_lqr_active = true;

      }

      bool is_run = g_exp_running;

      float d1 = g_sim_d1, d2 = g_sim_d2, vs = g_sim_vs, vus = g_sim_vus, u = g_sim_u;

      portEXIT_CRITICAL(&g_mqttMux);


      static unsigned long last_tel = 0;

      // Decimate telemetry transmission to 1 Hz
```

```
      if (is_run && (millis() - last_tel >= 1000)) {
        last_tel = millis();
        StaticJsonDocument<256> tdoc;
        tdoc["d1"] = d1; tdoc["d2"] = d2;
        tdoc["vs"] = vs; tdoc["vus"] = vus; tdoc["u"] = u;

        char buf[256];
        serializeJson(tdoc, buf);
        client.publish("suspension/telemetry", buf);
      }
      vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

A.3 I2C Bus Arbitration and Hardware Recovery

To prevent the entire HIL simulation from crashing due to transient sensor lockups, an I2C Mutex wrapper and an automatic bus-clearing recovery sequence were implemented.

```
static SemaphoreHandle_t g_i2cMutex = NULL;

static inline bool I2C_Lock(TickType_t timeoutTicks) {
  if (!g_i2cMutex) return true;
```

```
    return (xSemaphoreTake(g_i2cMutex, timeoutTicks) == pdTRUE);
}

static inline void I2C_Unlock() {
    if (g_i2cMutex) xSemaphoreGive(g_i2cMutex);
}

static void recoverI2CBus() {
    Wire.end();
    pinMode(I2C_SCL, OUTPUT);
    pinMode(I2C_SDA, INPUT_PULLUP);

    // Manually clock the SCL line 9 times to free locked slave devices
    for (int i = 0; i < 9; i++) {
        digitalWrite(I2C_SCL, LOW);
        delayMicroseconds(5);
        digitalWrite(I2C_SCL, HIGH);
        delayMicroseconds(5);
    }
```

```
  // Re-initialize hardware I2C peripherals
  Wire.begin(I2C_SDA, I2C_SCL);
  Wire.setTimeout(150);
  Wire.setClock(400000);
}
```

APPENDIX B: QUICK START / USER MANUAL

B.1 Overview

The CoTune remote platform is designed to be accessed entirely through a standard web browser. No local software installation (e.g., MATLAB, Arduino IDE) is required by the end user.

B.2 System Initialization (Laboratory Operator)

1. Ensure the ESP32 microcontroller is powered via a stable 5V supply and the Cytron MDD10A motor driver is connected to the 12V/24V bench power supply.

2. Ensure the laboratory Wi-Fi network specified in the secrets.h file is active.

3. Upon power-on, the ESP32 will automatically connect to AWS IoT Core. A solid blue LED indicator (if configured) confirms successful MQTT broker subscription.

B.3 User Access and Queue Entry

1. Navigate to the CoTune web platform URL.

2. Log in using your registered academic email address. First-time users must verify their email via the Amazon Cognito OTP sent to their inbox.

3. Select the desired experiment (Active Suspension or Rotary Pendulum) from the dashboard.

4. Upon entering the experiment page, you are automatically placed in the access queue. A loading screen will display your current position. Wait until your queue position reaches 1 and the interface unlocks.

B.4 Parameter Configuration and Execution

1. Explore the 3D Simulation: Use the embedded URDF 3D viewer to observe the theoretical kinematic response of the system before deploying physical control parameters.

2. Configure the LQR: Input your desired penalty matrices (Q and R). The browser will automatically solve the Algebraic Riccati Equation and display the resulting K gain vector.

3. Execute: Once the interface is unlocked, click "Start Live".
    a. A mandatory 10-second warmup phase will commence, allowing the system to reach initial conditions.

    b. At $t = 10$ s, the LQR controller will engage the physical hardware.

    c. Live telemetry will populate the Chart.js graphs, and the physical response can be observed through the embedded Twitch video stream.

4. Halt and Save: The session will automatically terminate after 5 minutes, or it can be stopped manually by clicking "Stop". Session data is automatically logged to DynamoDB and can be reviewed in the user's "Progress" tab.